

Efficient LLM Serving: A Comprehensive Survey on KV Cache Optimization, Speculative Decoding, and Quantization Strategies

1 Introduction and the Efficiency Trilemma in LLM Serving

1.1 The Memory-Bound Nature of Autoregressive Generation

1.1 The Memory-Bound Nature of Autoregressive Generation

The deployment of Large Language Models (LLMs) has fundamentally altered the landscape of deep learning inference, shifting the primary performance bottleneck from computational capacity to memory bandwidth and capacity. In traditional deep learning workloads, particularly during the training phase or the prefill stage of inference, operations are predominantly compute-bound. These phases involve massive matrix-matrix multiplications (GEMM) where the arithmetic intensity—the ratio of floating-point operations to bytes of memory access—is sufficiently high to saturate the GPU’s tensor cores. However, the autoregressive decoding phase, which generates tokens sequentially one by one, presents a drastically different computational profile. As the sequence length increases, the process transitions into a memory-bound regime where the speed of generation is dictated not by how fast the GPU can calculate, but by how quickly it can fetch data from global memory. This phenomenon, often referred to as the “memory wall” in AI serving, arises from the widening disparity between the scaling of compute power and memory bandwidth, creating a severe efficiency gap [1].

The root cause of this memory-bound nature lies in the mechanics of autoregressive generation and the indispensable role of the Key-Value (KV) cache. To avoid re-computing the attention keys and values for all preceding tokens at every generation step, LLM serving systems store these intermediate states in a KV cache. While this optimization reduces the computational complexity of attention from quadratic to linear with respect to sequence length, it introduces a substantial memory overhead. The size of the KV cache grows linearly with both the batch size and the sequence length. Consequently, for long-context applications or high-concurrency serving scenarios, the memory footprint of the KV cache can easily surpass the size of the model weights themselves [2]. In many modern deployment scenarios, the KV cache becomes the dominant consumer of GPU memory, strictly constraining the maximum achievable batch size and, by extension, the system throughput.

During each decoding step, the GPU must load the entire model weights and the relevant portions of the KV cache from High Bandwidth Memory (HBM) into on-chip SRAM to compute attention scores and generate the next token. Because only a single token is generated per step, the amount of computation performed per byte of data transferred is extremely low. This low arithmetic intensity means that the GPU’s computational units often sit idle, waiting for data to be fetched from memory. Research has shown that for decoder-only transformer models, memory bandwidth becomes the dominant bottleneck, limiting the utilization of expensive GPU hardware [1]. The situation is exacerbated by hardware scaling trends: peak server FLOPS have been increasing at a rate of approximately 3.0x every two years, whereas DRAM and interconnect bandwidth have only scaled at 1.6x and 1.4x, respectively [1]. This widening gap ensures that without specific architectural or algorithmic interventions, LLM serving will remain strictly limited by memory bandwidth rather than compute power.

The linear growth of the KV cache imposes hard constraints on GPU utilization that directly impact system efficiency. As the context length extends, the memory required to store the cache

increases proportionally, forcing serving systems to reduce the batch size to avoid out-of-memory errors. A smaller batch size further reduces arithmetic intensity, creating a vicious cycle where the GPU operates even further from its theoretical peak performance. Existing systems that reserve maximum sequence length memory for the KV cache to guarantee completion often suffer from significant memory fragmentation and underutilization, leading to lower throughput [2]. Furthermore, the inefficiency is not merely a matter of capacity but also of access patterns; the need to read large, dynamically growing KV caches for every token generation results in inefficient matrix-vector products that fail to leverage the parallel processing capabilities of modern accelerators [3].

To mitigate these bottlenecks, recent research has focused heavily on optimizing the management and compression of the KV cache, forming the foundation for many efficiency enhancements discussed later in this survey. Techniques such as PagedAttention have been introduced to eliminate memory fragmentation and allow for flexible sharing of KV caches across requests, thereby increasing the effective batch size and throughput [4]. Similarly, methods like H2O and Scissorhands propose eviction policies to retain only the most critical tokens, significantly reducing the memory footprint without substantial accuracy loss [5; 6]. Other approaches, such as KIVI and QAA, explore quantization strategies specifically tailored for the KV cache to reduce the number of bytes that must be transferred per token, directly addressing the bandwidth constraint [7; 8].

Moreover, the memory-bound nature of decoding has spurred innovations in system architecture, including the disaggregation of prefill and decode phases and the use of heterogeneous computing resources. For instance, FastDecode proposes decomposing the transformer model to offload memory-bound KV cache accessing to CPUs, leveraging their aggregated memory capacity and bandwidth to boost GPU throughput [9]. Additionally, specialized compression frameworks like GEAR and SubGen aim to achieve near-lossless compression or sublinear memory complexity by exploiting the clustering tendencies and low-rank structures within the KV cache [10; 11]. These efforts underscore the critical realization that overcoming the memory-bound bottleneck is essential for scalable and cost-efficient LLM serving. Without addressing the linear growth and bandwidth-hungry nature of the KV cache, the potential of LLMs in real-time, long-context applications will remain severely constrained by the physical limitations of current memory hierarchies, making the navigation of the resulting trade-offs between latency, throughput, and cost the central challenge of modern serving systems.

1.2 The Efficiency Trilemma: Latency, Throughput, and Cost

1.2 The Efficiency Trilemma: Latency, Throughput, and Cost

While Section 1.1 established that the autoregressive decoding phase is fundamentally constrained by memory bandwidth rather than compute capacity, translating this physical limitation into a practical serving system introduces a complex set of competing objectives known as the efficiency trilemma: minimizing latency, maximizing throughput, and reducing hardware costs. In an ideal landscape, a serving infrastructure would deliver instant responses to users, process massive volumes of concurrent requests, and operate on minimal infrastructure. However, the memory-bound nature of decoding creates inherent tensions among these goals, forcing system designers to make difficult trade-offs. Optimizing for one metric frequently entails significant degradation in the others, necessitating a nuanced, scenario-dependent approach to resource allocation and algorithmic design that directly addresses the bandwidth bottlenecks identified previously.

Latency, particularly Time to First Token (TTFT) and Time Per Output Token (TPOT), serves as the primary user-experience metric for interactive applications where immediate feedback is

paramount. Achieving low latency often requires dedicating substantial compute resources to individual requests to avoid queuing delays, a strategy that directly undermines throughput. Conversely, throughput, measured in tokens generated per second across the system, relies heavily on batching multiple requests together to amortize the fixed cost of loading model weights and KV cache data from memory, thereby saturating the limited memory bandwidth. As noted in recent analyses, while batching is highly effective for improving overall system throughput in memory-bound regimes, it can be detrimental to the latency of individual requests if not managed carefully [12]. When requests are aggregated to maximize GPU utilization, shorter requests may suffer from head-of-line blocking caused by longer prompts or slower decoding sequences, leading to increased tail latency and potential Service Level Objective (SLO) violations [13].

This conflict is further exacerbated by the distinct computational profiles of the prefill and decode phases discussed in the context of arithmetic intensity. The prefill phase, which processes the input prompt, is predominantly compute-bound and benefits from large batch sizes, whereas the decode phase is strictly memory-bound and sensitive to batch size increases due to the linear growth of the KV cache. Colocating these phases on the same hardware often leads to severe interference, where the resource demands of one phase starve the other, forcing a compromise between TTFT and TPOT [14]. Systems that prioritize low latency must often under-utilize hardware to ensure immediate scheduling, thereby inflating the cost per token. Conversely, systems optimized for maximum throughput may introduce unacceptable delays for latency-sensitive users. To mitigate this, advanced scheduling strategies like chunked prefills have been proposed to create stall-free schedules that allow new requests to enter the batch without pausing ongoing decodes, attempting to decouple these conflicting goals [12].

The third vertex of the trilemma, cost, introduces additional complexity by linking hardware utilization to economic efficiency. Cost efficiency is not merely a function of hardware selection but of how effectively that hardware is utilized under varying workload constraints imposed by the memory wall. The market offers a heterogeneous landscape of GPUs with varying price-to-performance ratios, yet there is no linear relationship between GPU cost and serving efficiency. The optimal hardware configuration depends heavily on the specific interplay of request size, arrival rate, and latency SLOs [15]. For instance, using high-end GPUs for low-concurrency, latency-critical tasks may be wasteful, while using commodity hardware for high-throughput batch processing might fail to meet strict SLOs due to insufficient memory bandwidth. Furthermore, the total cost of ownership is increasingly driven by power consumption. As data centers expand to meet LLM demands, energy availability has become a bottleneck, and power oversubscription strategies are being explored to deploy more servers within fixed power envelopes, though this risks performance instability if not managed with robust telemetry and controls [16].

Addressing the trilemma often requires moving beyond static resource allocation to dynamic, adaptive systems that can navigate the memory-bound constraints identified in Section 1.1. Traditional approaches that rely on hardware isolation or static partitioning are limited by the finite number of resource partitions available, such as last-level cache slices, which restricts the number of latency-critical applications that can be co-located without interference [17]. Consequently, modern solutions are shifting towards software-defined orchestration that can dynamically adjust resource allocation based on real-time workload characteristics. For example, disaggregating prefill and decode computations onto different hardware units allows for independent optimization of resources tailored to the specific needs of each phase, significantly improving the maximum rate of requests served within strict latency constraints [14]. Similarly, leveraging heterogeneous pipelines that offload memory-bound components to CPUs can boost GPU throughput without incurring the

prohibitive costs of additional GPU memory [9].

Ultimately, navigating the efficiency trilemma requires a holistic view that considers the specific deployment scenario and the underlying hardware limitations. Interactive chatbots may prioritize latency over raw throughput, accepting higher costs to ensure responsiveness, while batch processing for data analysis might prioritize throughput and cost, tolerating higher latency. Emerging frameworks are beginning to automate these trade-offs, using cost-aware bin-packing algorithms to select the most efficient mix of hardware resources that adhere to configurable latency SLOs [15]. Additionally, the push towards “greener” LLMs emphasizes that energy efficiency must be treated as a first-class citizen alongside latency and throughput, requiring providers to explore various optimization knobs that balance ecological impact with performance guarantees [18]. As the field evolves, the ability to dynamically navigate these trade-offs will define the viability and scalability of LLM serving infrastructures. However, to move from empirical trade-offs to rigorous optimization, it is essential to adopt a theoretical framework that quantifies these bottlenecks, a topic explored in the following section through the lens of Roofline models and arithmetic intensity.

1.3 Theoretical Foundations: Roofline Models and Arithmetic Intensity

1.3 Theoretical Foundations: Roofline Models and Arithmetic Intensity

To rigorously analyze and optimize the performance of Large Language Model (LLM) serving, it is imperative to move beyond the empirical trade-offs of the “Efficiency Trilemma” discussed previously and adopt a theoretical framework that elucidates the fundamental bottlenecks inherent in hardware execution. The Roofline model serves as this critical diagnostic tool, providing a visual representation of the upper bound on application performance based on the interplay between computational throughput and memory bandwidth. Originally developed for high-performance computing, this model has been adapted to deep learning contexts to systematically identify whether a specific kernel or phase of inference is compute-bound or memory-bound [19]. In the realm of LLM serving, the Roofline model acts not merely as a static performance ceiling but as a dynamic lens through which the conflicting goals of latency, throughput, and cost can be understood, revealing why improvements in algorithmic Floating Point Operations (FLOPs) do not always translate to reduced wall-clock latency.

The core of the Roofline model rests on the concept of Arithmetic Intensity (AI), defined as the ratio of total floating-point operations performed to the total bytes of data transferred from global memory to the compute units. Mathematically, if P represents performance in FLOPs per second, B represents memory bandwidth in bytes per second, and I represents arithmetic intensity in FLOPs per byte, the performance is bounded by $P \leq \min(P_{peak}, B \times I)$. Here, P_{peak} denotes the peak theoretical compute capacity of the hardware. When the arithmetic intensity is high, the application operates in the compute-bound regime, where performance is limited by the processor’s ability to execute instructions. Conversely, when arithmetic intensity is low, the application enters the memory-bound regime, where performance is constrained by the rate at which data can be fetched from memory, regardless of how powerful the compute units are [20].

In the context of autoregressive LLM generation, the decoding phase exhibits notoriously low arithmetic intensity, placing it firmly in the memory-bound region of the Roofline plot. This theoretical insight directly explains the tensions described in the efficiency trilemma: during token generation, the model must load the entire set of weights for each layer to generate a single new token. Since the number of operations required to generate one token scales linearly with the model size, but the data movement also scales linearly with the model size, the ratio of computation

to data movement remains small. This stands in stark contrast to the prefill phase or training, where large batch sizes or long sequences allow for significant data reuse within on-chip caches, thereby increasing arithmetic intensity and pushing the workload toward the compute-bound limit. Research indicates that for single-batch inference, which is common in latency-sensitive serving scenarios, the memory bandwidth becomes the primary bottleneck rather than the compute capacity [21]. Consequently, simply adding more computational cores or utilizing faster tensor cores yields diminishing returns if the memory subsystem cannot feed data to these units fast enough, forcing system designers to choose between under-utilized hardware (to reduce latency) or massive batching (to improve throughput).

The implications of this low arithmetic intensity are profound for system design and optimization strategies, providing the theoretical justification for the key techniques explored in this survey. It explains why techniques that reduce the volume of data transferred, such as quantization, often yield disproportionate speedups compared to techniques that merely reduce the number of operations. For instance, reducing weight precision from 16-bit to 4-bit reduces the memory bandwidth requirement by a factor of four, effectively shifting the Roofline ceiling upward for memory-bound workloads, whereas reducing FLOPs via pruning might have a negligible impact on latency if the memory access pattern remains unchanged [22]. Furthermore, the growth of the Key-Value (KV) cache exacerbates this memory bottleneck. As the sequence length increases, the KV cache grows linearly, requiring additional memory bandwidth to fetch historical context for attention calculations, further depressing the arithmetic intensity and saturating the memory bus [5]. This understanding drives the necessity for efficient KV cache management and speculative decoding, the latter of which amortizes memory costs over multiple tokens to improve effective throughput.

Advanced variations of the Roofline model have been proposed to capture the nuances of modern distributed and heterogeneous systems, bridging the gap between theoretical limits and practical deployment. For example, the “Ridgeline” model extends the traditional 2D Roofline to include network bandwidth as a third dimension, which is essential for analyzing distributed LLM serving where inter-node communication can become the limiting factor [23]. Similarly, time-based extensions incorporate kernel launch overheads and cache locality effects, providing a more granular view of where time is spent during execution [20]. These refined models help practitioners understand that the “framework tax”—the overhead introduced by software stacks that prevents hardware from reaching theoretical peaks, as will be detailed in the following section—is often a symptom of inefficient memory access patterns that fail to maximize arithmetic intensity [24].

Moreover, the Roofline framework guides the development of specialized dataflows and kernel optimizations designed to push systems closer to their theoretical limits. Techniques such as FlashDecoding++ and FLAT are explicitly designed to mitigate attention bottlenecks by optimizing data movement and increasing data reuse, effectively attempting to raise the arithmetic intensity of the attention mechanism [25][26]. By fusing operations and minimizing off-chip memory accesses, these methods strive to shift the operational point of the LLM inference closer to the compute roofline. Without such optimizations, the system remains stuck in the bandwidth-limited slope of the Roofline model, where any increase in model size or sequence length directly degrades Tokens-per-second throughput.

Ultimately, the Roofline model provides the theoretical backbone for the integrated optimization strategies discussed in this survey. It clarifies that addressing the efficiency trilemma requires a holistic approach: KV cache management reduces the data volume to be transferred, speculative decoding amortizes memory costs over multiple tokens, and quantization directly increases the effective bandwidth by compressing data. Understanding the arithmetic intensity of each phase

allows researchers to target the correct bottleneck, ensuring that optimizations yield tangible improvements in latency and throughput rather than theoretical FLOP counts that never materialize in practice [19]. As hardware evolves with newer architectures like H100s offering higher bandwidth and specialized tensor cores, the Roofline model remains an indispensable tool for navigating the shifting landscape of LLM serving efficiency [27]. However, even with perfect algorithmic alignment to the Roofline model, realizing these theoretical gains depends heavily on the efficiency of the underlying software stack, a challenge that leads directly to the discussion of the framework tax.

1.4 The Framework Tax and Hardware-Software Disparities

1.4 The Framework Tax and Hardware-Software Disparities

While the Roofline model and Arithmetic Intensity provide a robust theoretical framework for understanding hardware bottlenecks, realizing these theoretical limits in practice is frequently obstructed by the software stack itself. This phenomenon, known as the “framework tax,” represents the substantial disparity between the peak performance predicted by hardware specifications and the actual wall-clock latency achieved during inference. As established in the previous section, LLM serving is predominantly memory-bound, suggesting that optimizations like quantization and KV cache management should yield direct latency reductions. However, empirical evidence reveals that advancements in model architectures and hardware accelerators often fail to translate proportionally into end-to-end speedups due to overheads introduced by deep learning frameworks. These frameworks manage memory, schedule tasks, and interface with hardware drivers in ways that frequently prevent the full exploitation of silicon potential, creating a bottleneck that algorithmic reductions in FLOPs alone cannot resolve [24]. As hardware speeds continue to increase exponentially, this gap between theoretical efficiency and realized performance widens, necessitating a critical examination of the software execution engine.

The root of the framework tax often lies in inefficient task scheduling and the serialization of operations within the software execution engine. Ideally, a deep learning framework should orchestrate GPU utilization such that runtime is strictly bound by the volume of computation. In reality, standard frameworks often serialize operations that could theoretically execute in parallel, leading to significant underutilization of available compute resources. For instance, the inability to effectively overlap kernel launches or manage dependencies dynamically results in idle GPU cycles. Research has demonstrated that introducing ahead-of-time (AoT) scheduling and leveraging multiple GPU streams within a single device can eliminate most runtime scheduling overheads. Such optimizations have been shown to accelerate inference by up to 22.34 times compared to standard framework implementations like PyTorch, highlighting the sheer magnitude of the latency penalty imposed by suboptimal software scheduling [28]. This inefficiency directly counteracts the benefits of high arithmetic intensity sought in the prefill phase, as the hardware remains starved of work despite available computational capacity.

Furthermore, a structural mismatch exists between the fine-grained requirements of neural network architectures and the coarse-grained abstractions provided by general-purpose frameworks. The relationship between network form (architecture design) and function (hardware acceleration) is complex; while hardware-specific acceleration can theoretically achieve average speedups of 380%, the realized improvement varies drastically depending on how well the macro-architecture aligns with the target hardware’s memory hierarchy and cache efficacy [29]. Designs not optimized for specific hardware primitives fail to benefit from acceleration features, a problem exacerbated by

the “one-size-fits-all” nature of many frameworks. These frameworks often lack the dynamic algorithm mapping and fine-grained tuning necessary to adapt to the diverse computation and memory requirements across different layers of deep networks, resorting to suboptimal strategies that leave hardware resources idle or force inefficient data movements [30]. This disconnect is particularly detrimental in LLM serving, where the distinct patterns of the attention mechanism and feed-forward networks require specialized handling to approach the Roofline ceiling.

Memory management constitutes another critical component of the framework tax, particularly regarding data ingestion, I/O pipelining, and the handling of intermediate states. During operation, frameworks must load, pre-process, and transfer numerous small files from CPUs to accelerators. If the framework fails to effectively overlap this I/O pipeline with computation, the accelerator stalls, waiting for data. Studies characterizing I/O workloads in frameworks like TensorFlow have shown that increasing thread counts and utilizing dedicated prefetchers can eliminate the effective cost of I/O by completely overlapping it with accelerator computation [31]. Without such optimizations, the framework’s inability to hide latency results in a direct increase in wall-clock time, regardless of the model’s theoretical efficiency. Additionally, the memory footprint of intermediate outputs often restricts the scale of architectures that can be deployed. Automated frameworks that jointly optimize checkpointing schedules and operator implementations have demonstrated the ability to reduce memory requirements by 3x with minimal computational overhead, proving that intelligent software management is essential to overcome hardware memory constraints and enable the large batch sizes necessary for throughput optimization [32].

The disparity is further evident when comparing native framework performance against hand-tuned, hardware-specific implementations, especially in the context of the optimization techniques discussed later in this survey. Specialized hardware such as FPGAs or custom ASICs offers the potential for ultra-low latency and high energy efficiency, but realizing this potential requires overcoming the abstraction layers of general frameworks. For example, deploying models on low-power devices often reveals that popular deep learning frameworks lack support for highly optimized low-precision operators, forcing the system to rely on generic, slower libraries. Automated workflows that generate high-performance, low-precision operators tailored to specific CPU architectures have shown speedups of up to 16x over optimized integer baselines, illustrating how framework limitations can stifle the benefits of quantization [33]. Similarly, full-stack search techniques that optimize hardware datapaths, software scheduling, and compiler passes simultaneously have achieved significant performance gains over standard TPU implementations, underscoring the necessity of co-designing the software stack alongside the hardware to fully leverage techniques like speculative decoding and KV cache compression [34].

Ultimately, the framework tax signifies that reducing floating-point operations or model parameters is insufficient for achieving true efficiency if the software layer introduces redundant overheads. The gap between the algorithmic innovations described in the theoretical foundations and deployment reality is bridged only by addressing these software bottlenecks. Whether through automated end-to-end optimizing compilers that handle operator fusion and memory latency hiding [35], or through dynamic runtime management systems that adapt to hardware availability [36], the future of efficient LLM serving demands a rigorous re-evaluation of the software foundations. Ignoring the framework tax risks rendering advanced algorithmic optimizations—such as the KV cache management, speculative decoding, and quantization strategies explored in the following section—moot, as their theoretical gains are consumed by the inefficiencies of the execution environment. Therefore, a holistic approach to LLM efficiency must treat the software stack not as a passive conduit, but as a critical optimization target equal in importance to the model architecture and

the underlying hardware, paving the way for the synergistic co-design strategies discussed next.

1.5 Motivation for Integrated Optimization Strategies

1.5 Synergistic Co-Design: Integrating KV Cache, Speculative Decoding, and Quantization

The preceding analysis of the “framework tax” reveals a critical insight: even with optimal algorithmic reductions in floating-point operations, end-to-end latency is often dictated by software overheads and hardware-software mismatches. This reality underscores that isolated optimization strategies targeting Key-Value (KV) cache management, speculative decoding, or quantization are increasingly insufficient to meet the demands of scalable Large Language Model (LLM) serving. While individual techniques have demonstrated significant standalone improvements, the compounding constraints of modern deployment scenarios—exacerbated by the very framework inefficiencies previously discussed—necessitate a holistic, integrated approach. The motivation for such synergy arises from the interdependent nature of these bottlenecks; addressing memory capacity without considering bandwidth constraints, or reducing sequential dependencies without accounting for quantization-induced errors, often leads to suboptimal system performance where theoretical gains are consumed by execution overheads. Therefore, the future of efficient LLM serving lies not in the siloed application of these technologies, but in their co-design and joint optimization to simultaneously alleviate memory footprint, maximize throughput, and maintain fidelity within the constraints of the underlying software stack.

The primary driver for this integration is the limitation of single-dimension solutions in handling the “long-context” revolution, a challenge where framework inefficiencies often magnify memory pressure. As sequence lengths extend, the KV cache grows linearly, rapidly saturating GPU memory and becoming the dominant factor in latency. Methods that rely solely on eviction policies, such as [6] or [5], effectively reduce memory usage by retaining only pivotal tokens. However, these approaches risk discarding information crucial for complex reasoning tasks, potentially leading to hallucinations or context loss, while also introducing irregular memory access patterns that further strain the framework’s scheduler. Conversely, pure quantization strategies like [7] or [10] can compress the cache significantly but may introduce accumulation errors that degrade generation quality over long sequences. An integrated strategy mitigates these individual weaknesses; for instance, combining importance-aware eviction with mixed-precision quantization, as suggested by [37], allows systems to retain a broader context in low precision while keeping critical tokens in high precision. This balance not only saves memory but also regularizes data layouts, helping to alleviate the memory management bottlenecks identified in the framework tax.

Furthermore, the interplay between memory optimization and speculative decoding reveals another layer of necessity for integrated designs, particularly in overcoming the sequential execution penalties inherent in many frameworks. Speculative decoding aims to break the sequential dependency of token generation by employing a draft-then-verify paradigm. However, the efficiency of this process is heavily contingent on the speed of the verification step, which is itself memory-bound due to KV cache access. If the KV cache is not optimally managed, the theoretical speedups from speculation are nullified by memory bandwidth saturation and framework-induced stalls. Recent works like [38] explicitly address this by leveraging dynamic sparse KV caches alongside hierarchical speculation, demonstrating that speculation and cache compression are not mutually exclusive but rather mutually reinforcing. Similarly, [39] illustrates that combining these methods yields superior latency

reductions compared to applying them in isolation. Without integrating cache management into the speculative framework, the draft model may generate tokens faster than the target model can verify them due to memory stalls, rendering the speculation ineffective and wasting the compute resources that the framework fails to schedule efficiently.

Quantization further complicates this landscape, acting as both a solution and a potential source of instability when combined with other techniques, especially given the sensitivity of low-level operator implementations. While ultra-low-bit quantization, such as the 4-bit end-to-end scheme proposed in [40], drastically reduces memory traffic and enables larger batch sizes, it can alter the probability distributions that speculative decoding relies upon for token acceptance. If the draft and target models exhibit divergent behaviors due to quantization noise, the acceptance rate drops, negating the benefits of speculation and exacerbating the framework’s inability to hide latency. Therefore, an integrated approach must jointly optimize quantization parameters with speculative strategies. For example, adaptive quantization schemes like [8] which tailor precision based on sensitivity, can be aligned with speculative verification thresholds to ensure that the compressed representation does not unduly penalize the acceptance of correct draft tokens. Moreover, systems like [9] demonstrate that decomposing model parts and offloading memory-bound operations to CPUs requires careful coordination with quantization to minimize data transfer overheads, highlighting the need for cross-layer optimization that bridges the gap between algorithmic design and hardware-specific execution.

The synergy of these techniques also directly addresses the hardware-software disparity known as the “framework tax” by enabling more efficient kernel implementations and memory layouts. Isolated optimizations often fail to translate into wall-clock speedups due to inefficient kernel implementations, memory fragmentation, or the inability of general-purpose frameworks to handle irregular computation graphs. Integrated frameworks like [4] show that system-level architectural changes, such as paged attention, are prerequisites for realizing the full potential of algorithmic innovations by abstracting away physical memory fragmentation. When combined with compression techniques like [11], which employs clustering to achieve sublinear complexity, or [41], which optimizes budgets across layers, the system can achieve near-hardware-limit throughput. The motivation is clear: only by treating KV cache management, speculative decoding, and quantization as interconnected variables in a unified optimization problem can we overcome the diminishing returns of individual methods and the structural inefficiencies of the software stack.

Finally, the reliability and safety of compressed and accelerated models demand an integrated perspective that accounts for the cumulative effects of these optimizations. As noted in [37], aggressive eviction can lead to safety breaches and hallucinations, risks that are amplified when framework overheads force trade-offs between precision and speed. An integrated strategy ensures that speedup mechanisms do not compromise the model’s parametric knowledge. By jointly tuning the aggressiveness of eviction, the length of speculative drafts, and the bit-width of quantization, serving systems can operate within a safe “efficiency envelope” that guarantees both performance and trustworthiness. This holistic view is essential for deploying LLMs in critical applications where latency, cost, and accuracy must be balanced precisely, marking the transition from component-level tweaking to system-level co-design as the defining challenge of the next generation of efficient LLM serving.

2 Advanced Key-Value Cache Management and Memory Optimization

2.1 Virtual Memory and Paged Attention Architectures

2.1 Virtual Memory and Paged Attention Architectures

The deployment of Large Language Models (LLMs) has fundamentally shifted the bottleneck of autoregressive generation from computational power to memory bandwidth and capacity. As sequence lengths grow, the Key-Value (KV) cache required to store intermediate attention states expands linearly, often exceeding the available high-bandwidth memory on accelerators. Traditional deep learning frameworks rely on contiguous memory allocation strategies that are ill-suited for the dynamic and fragmented nature of KV cache growth during inference. This rigidity leads to severe internal and external fragmentation, where usable memory exists but cannot be allocated due to a lack of contiguous blocks, forcing systems to underestimate their true capacity or suffer from Out-Of-Memory (OOM) errors. To address these systemic inefficiencies, recent innovations have turned to operating system principles, specifically virtual memory management, to decouple logical address spaces from physical memory layouts. This architectural foundation is prerequisite for the dynamic token management strategies discussed in the following section, as efficient eviction policies require a flexible underlying memory substrate.

The cornerstone of this architectural shift is the concept of Paged Attention, which adapts the paging mechanisms found in modern operating systems to the specific requirements of GPU memory management in LLM serving. By treating the KV cache as a collection of non-contiguous physical blocks mapped to a contiguous logical space via page tables, systems can eliminate the fragmentation overhead inherent in pre-allocated contiguous buffers. This approach allows the memory allocator to utilize any available free block, regardless of its physical location, thereby maximizing memory utilization. The inspiration for such optimizations draws heavily from broader research into memory virtualization and fragmentation mitigation. For instance, studies on GPU memory defragmentation have highlighted how native allocators degrade under irregular allocation patterns common in advanced training and inference techniques, proposing virtual memory stitching to fuse non-contiguous blocks into a logical whole [42]. Similarly, the challenges of physical memory fragmentation in production environments, often exacerbated by file caching and reclamation systems, underscore the necessity of management layers that can handle discontinuous free memory effectively [43].

Beyond simple paging, advanced hierarchical caching strategies are essential for managing the disparity between fast on-device memory and slower, larger capacity pools such as CPU RAM or NVMe storage. The “memory wall” necessitates intelligent data movement policies that keep hot KV blocks on the GPU while evicting colder blocks to lower tiers without stalling the generation process. This tiered approach mirrors developments in hybrid memory systems where data placement is dynamically optimized based on access patterns. Research into transparent page placement for tiered memory systems demonstrates that proactively demoting cold pages to slower memory tiers while keeping hot pages in fast memory can achieve performance comparable to ideal baselines with all data in the fastest tier [44]. Furthermore, mechanisms that enable non-exclusive memory tiering, where pages can exist in both fast and slow memory simultaneously to prevent thrashing under pressure, offer robust solutions for handling fluctuating workloads [45].

The integration of these virtual memory concepts with hardware-specific optimizations is critical for realizing their full potential. For example, leveraging large pages or superpages can significantly reduce address translation overheads, a bottleneck that becomes pronounced when managing millions

of small KV blocks. However, traditional superpage management often conflicts with the need for fine-grained migration in hybrid systems. Innovations like “Rainbow” propose managing memory at superpage granularity while using fast memory to cache hot sub-pages, effectively bridging the gap between translation efficiency and migration flexibility [46]. Additionally, the efficiency of these systems relies on minimizing the latency of page table walks and data movement. Techniques that flatten page tables or bias cache replacement algorithms to retain page table entries during high miss rate periods have shown substantial performance gains in virtualized and high-throughput environments [47].

Moreover, the interaction between the memory subsystem and the compute engine requires careful orchestration to avoid interference. In heterogeneous systems, memory bandwidth contention between multiple applications or between the CPU and GPU can degrade performance. Architectural support for efficient data movement, such as bandwidth partitioning and adaptive granularity selection, ensures that the overhead of managing paged KV caches does not negate the benefits of increased capacity [48]. The use of unified virtual memory (UVM) further simplifies programming models but introduces challenges related to page fault handling and prefetching. Intelligent frameworks that utilize machine learning to predict access patterns and manage oversubscription in CPU-GPU unified memory can significantly reduce page thrashing and improve throughput [49].

In summary, the adoption of virtual memory and paged attention architectures represents a paradigm shift in LLM serving by abstracting away from physical contiguity constraints. By leveraging hierarchical, OS-inspired management strategies, these systems achieve near-optimal memory utilization, creating the necessary infrastructure for more granular optimization techniques. While paged attention solves the problem of *where* and *how* to store KV blocks efficiently, it does not inherently determine *which* tokens are most valuable to retain when memory pressure persists. This distinction sets the stage for the next layer of optimization: dynamic token eviction policies and heavy hitter identification, which operate atop these flexible memory architectures to selectively prune the cache based on semantic importance rather than just access recency. As memory technologies evolve towards more disaggregated and heterogeneous configurations, the principles of paged attention will remain central to unlocking the efficiency required for next-generation generative AI applications.

2.2 Token Eviction Policies and Heavy Hitter Identification

2.2 Token Eviction Policies and Heavy Hitter Identification

Building upon the flexible memory substrate established by paged attention architectures in Section 2.1, the next critical challenge lies in determining *which* tokens to retain when memory pressure persists. While paged attention solves the problem of physical fragmentation and enables efficient tiered storage, it does not inherently address the semantic value of the stored data. As sequence lengths continue to grow, the Key-Value (KV) cache scales linearly, often surpassing the memory footprint of the static model parameters and becoming the primary bottleneck for long-context applications like dialogue systems and story generation. To overcome this capacity constraint without resorting to costly hardware upgrades, research has shifted toward dynamic token eviction policies. These strategies operate on the premise that not all tokens contribute equally to the attention mechanism’s output; by identifying and discarding less informative tokens while preserving critical context, systems can achieve substantial memory savings with minimal impact on generative quality.

A pioneering approach in this domain is the Heavy-Hitter Oracle (H₂O), which introduces an

eviction policy grounded in the observation that a small subset of tokens, termed “Heavy Hitters,” contributes the majority of the value during attention score computation [5]. The authors of H₂O found that the emergence of these heavy hitters is a natural phenomenon strongly correlated with the frequent co-occurrence of tokens in the text. Crucially, their investigation revealed that removing these pivotal tokens results in significant performance degradation, whereas evicting non-heavy hitters has a negligible impact. Consequently, H₂O formulates KV cache eviction as a dynamic submodular problem, theoretically guaranteeing that the algorithm effectively balances the retention of recent tokens and identified heavy hitters. Empirical validation demonstrates that implementing H₂O with only 20% of tokens retained as heavy hitters can improve throughput by up to 29× compared to leading inference systems like DeepSpeed Zero-Inference and Hugging Face Accelerate, while reducing latency by up to 1.9× [5].

Expanding on the concept of token importance, Scissorhands proposes a system grounded in the “Persistence of Importance” hypothesis [6]. This hypothesis posits that tokens which exerted a substantial influence at one generation step are highly likely to significantly influence future generations. Based on this empirical verification, Scissorhands manages the KV cache by probabilistically storing pivotal tokens, thereby maintaining memory usage within a fixed budget without requiring model fine-tuning. The system has been shown to reduce inference memory usage of the KV cache by up to 5X without compromising model quality. Furthermore, Scissorhands exhibits strong compatibility with other compression techniques; when combined with 4-bit quantization traditionally used for model weights, it can achieve a cumulative compression ratio of up to 20X [6].

Complementing these persistence-based strategies, Keyformer leverages the observation that approximately 90% of the attention weight in generative inference focuses on a specific subset of “key” tokens [50]. Keyformer employs a novel score function to identify and retain only these crucial tokens in the KV cache, effectively reducing both the cache size and the associated memory bandwidth usage. This approach is particularly effective in applications requiring long-context processing, such as summarization and conversation tasks. Evaluations across foundational models like GPT-J and Cerebras-GPT indicate that Keyformer can reduce inference latency by 2.1× and improve token generation throughput by 2.4× while preserving model accuracy [50].

Beyond identifying heavy hitters solely based on cumulative attention scores, recent works have explored more nuanced criteria for eviction derived from structural and temporal patterns. For instance, the CORM algorithm capitalizes on the high similarity between adjacent tokens’ query vectors, observing that current query attention calculations can often rely on a small portion of preceding queries [51]. By dynamically retaining important key-value pairs based on these structural similarities, CORM reduces KV cache memory usage by up to 70% with no noticeable performance degradation. Similarly, RoCo addresses deficiencies in prior policies regarding importance score calculation and eviction scope construction by introducing a robust cache omission policy based on temporal attention scores and robustness measures [52].

However, the binary nature of “keep-or-drop” decisions inherent in the aforementioned methods carries the risk of irreversible context loss, potentially leading to safety breaches, hallucinations, or degraded coherence. To mitigate this, hybrid approaches like MiKV (Mixed-precision KV cache) argue that completely discarding evicted KV pairs is suboptimal [37]. MiKV proposes retaining evicted pairs in low precision while keeping important pairs in high precision, thus recovering degraded generation quality while still achieving significant compression. This highlights a critical evolution in eviction policies: moving from rigid binary decisions to more granular, importance-aware strategies that safeguard the integrity of the generative process.

Collectively, these token eviction policies and heavy hitter identification mechanisms transform the KV cache from a rigid, linearly growing structure into a dynamic, adaptive memory system capable of supporting ultra-long contexts on commodity hardware. While these methods effectively cap memory usage by pruning the cache, they fundamentally rely on a binary logic that discards data permanently. This limitation sets the stage for the complementary strategy discussed in Section 2.3: sparse and selective attention mechanisms. Unlike eviction policies that reduce the *size* of the cache, sparse attention alters the *computation* and *access patterns*, allowing models to interact with the full context selectively without permanently removing information, thereby offering a different avenue to decouple inference cost from sequence length.

2.3 Sparse and Selective Attention Mechanisms

2.3 Sparse and Selective Attention Mechanisms

While the token eviction policies discussed in Section 2.2 effectively cap memory usage by discarding less critical tokens, they inherently operate on a binary “keep-or-drop” logic that risks irreversible context loss. In contrast, sparse and selective attention mechanisms offer a complementary strategy: rather than permanently removing data from the cache, these approaches fundamentally alter the *computation* and *access patterns* of the attention mechanism. By selectively fetching or computing attention scores for only a critical subset of the context, these methods decouple inference cost from sequence length while striving to preserve the full informational fidelity of the original sequence. Unlike structural sparsity methods that impose static patterns (e.g., local windows or strided attention) which may discard globally relevant information, modern selective mechanisms are often dynamic, adapting to the input data to identify and retain only the most salient tokens during the inference pass.

A prominent category of selective attention leverages the observation that attention distributions in pre-trained LLMs are inherently sparse, with specific “heavy hitter” tokens dominating the attention scores across various heads and layers. [53] capitalizes on this insight by introducing a fine-tuning-free approach that dynamically compresses the KV cache access. The method operates on the premise that attention heads consistently focus on specific prompt features that can be identified via an observation window at the end of the prompt. By clustering and selecting only these important KV positions for each attention head, [53] significantly reduces the memory overhead and computational burden associated with long sequences. Empirical evaluations demonstrate that this strategy yields substantial efficiency gains, achieving an 8.2x enhancement in memory efficiency and a 3.6x increase in generation speed for inputs of 16K tokens, all while maintaining performance comparable to full-attention baselines across diverse long-sequence datasets. Furthermore, [53] enables the processing of context lengths up to 380K tokens on a single GPU with negligible accuracy degradation, highlighting the potential of selective caching to extend the practical limits of existing models.

Complementing cache selection strategies, other techniques focus on optimizing the data movement and bandwidth utilization during the attention computation itself, addressing the memory-bandwidth bottleneck that typically constrains token generation in large-batch and long-sequence settings. [54] addresses this by employing a selective fetching mechanism that retrieves only the most relevant portions of the KV cache based on the current query, rather than computing attention over the entire cached history. This approach allows for a drastic reduction in data transfers, reporting up to 8x savings in attention data traffic without requiring modifications to the pre-training setup or additional fine-tuning. By aligning memory access patterns with the intrinsic

sparsity of attention distributions, [54] effectively increases inference throughput, making it particularly suitable for deployment scenarios where memory bandwidth is the primary constraint rather than compute capacity.

Beyond token-level selection, some methodologies explore sparsity at the level of attention heads or through learned connectivity patterns to further reduce computational load. [55] proposes a comprehensive algorithm-architecture co-design that exploits both token and head sparsity. Recognizing the redundancy in human language, [55] introduces cascade pruning techniques that dynamically remove unimportant tokens and non-essential heads on the fly. This is supported by specialized hardware engines designed to rank importance scores with high throughput, alongside progressive quantization strategies that further reduce memory access. Similarly, [56] challenges the limitations of fixed spatial attention patterns by learning instance-dependent connectivity. By employing a lightweight predictor to estimate relevance scores between token pairs, [56] constructs unstructured sparse masks that capture semantic relationships beyond mere spatial proximity, achieving significant FLOPs reduction with minimal accuracy loss.

The concept of selective attention also extends to modifying the fundamental interaction mechanisms within the transformer block to avoid quadratic costs entirely. [57] argues against treating all tokens equally, proposing a dual-attention encoder that scores tokens based on localization and semantic information to focus computation on informative regions. This selective focus allows the model to abandon background queries efficiently, optimizing the trade-off between computational efficiency and detection accuracy. In the realm of general sequence modeling, [58] introduces flexibility by replacing the standard softmax activation with α -entmax, a differentiable function that allows low-scoring words to receive precisely zero weight. This enables attention heads to learn context-dependent sparsity patterns automatically, fostering greater head diversity and interpretability without sacrificing performance.

Furthermore, innovative approaches like [59] leverage the spatial coherence and sparse structure inherent in real-world data to approximate attention maps in $O(n \log n)$ time. By utilizing randomized correspondence algorithms to pinpoint compatible keys before constructing a sparse approximation, [59] offers a broadly applicable acceleration technique that differs from static sparse approximations. Additionally, [60] posits that small, input-dependent sets of attention heads and parameters can approximate the dense model’s output. [60] implements a system to predict this contextual sparsity on the fly, achieving significant latency reductions on massive models like OPT-175B without compromising quality. Collectively, these sparse and selective mechanisms represent a paradigm shift from brute-force computation to intelligent, data-adaptive processing. While eviction policies (Section 2.2) manage memory capacity by shrinking the cache size, and the clustering strategies discussed next (Section 2.4) compress the representation of retained tokens, sparse attention optimizes the *access* and *computation* over the available context, forming a crucial pillar in the triad of efficient LLM serving.

2.4 Context Compression and Clustering Strategies

2.4 Context Compression and Clustering Strategies

While sparse and selective attention mechanisms (Section 2.3) optimize the *access* and *computation* over the context by filtering which tokens to process, they often retain the original representation of the selected tokens. In contrast, context compression and clustering strategies take a more aggressive approach: they fundamentally alter the *representation* of the retained context itself. Where token eviction policies (Section 2.2) risk irreversible information loss by discarding data, and

sparse attention merely skips computation on less relevant parts, compression techniques aim to distill the essential information of the entire context into a compact, sublinear memory footprint. By employing sophisticated methods such as online clustering, low-rank approximation, and semantic summarization, these approaches preserve the fidelity of long-range dependencies while drastically reducing the storage requirements of the Key-Value (KV) cache.

A foundational insight driving many of these methods is the observation that key embeddings within the attention module often exhibit significant clustering tendencies. Exploiting this structural redundancy, [11] proposes a novel caching method that employs online clustering on key tokens combined with online ℓ_2 sampling on values. By grouping similar keys and representing them with cluster centers, SubGen effectively reduces the number of stored KV pairs, achieving a provably accurate attention decoding algorithm with sublinear time and memory complexity. The authors establish tight error bounds for this approach, demonstrating that it significantly outperforms existing state-of-the-art KV cache compression methods in long-context question-answering tasks. This work highlights the potential of treating the KV cache not as a rigid list of vectors but as a dynamic distribution that can be approximated efficiently.

Complementing clustering-based approaches, low-rank approximation and hybrid quantization schemes offer another avenue for near-lossless compression. [10] introduces a framework that synergistically integrates ultra-low precision quantization, low-rank matrix approximation, and sparse error correction. GEAR operates on the premise that uniform quantization or simple token dropping often incurs high approximation errors, particularly when compounded over autoregressive steps. Instead, GEAR quantizes the majority of entries with similar magnitudes to ultra-low precision, utilizes a low-rank matrix to approximate the resulting quantization error, and employs a sparse matrix to remedy individual outliers. This multi-faceted strategy enables near-lossless 4-bit KV cache compression, yielding substantial throughput improvements and peak memory reductions without the critical deviation in model generation observed in simpler methods. Similarly, [61] advances this direction by proposing a two-dimensional quantization strategy specifically designed for the distribution of the KV cache, incorporating cross-block reconstruction regularization to optimize parameters and balance accuracy with efficiency.

In the domain of interactive applications, such as multi-turn dialogues, the structure of the context offers unique opportunities for compression that bridge the gap between single-pass inference and persistent conversation history. [62] observes that dialogue contexts are highly structured and that specific tokens, such as the “End-of-Utterance” (EoU) markers, act as “conversational attention sinks” capable of aggregating information. Leveraging this, StreamingDialogue compresses long dialogue histories into these compact EoU representations, reducing computational complexity quadratically with the number of utterances. The method incorporates short-memory reconstruction and long-memory reactivation strategies to minimize information loss, enabling the handling of over 200,000 utterances with an 18-fold reduction in memory usage compared to dense attention re-computation. This approach demonstrates that semantic understanding of the input modality can drive highly efficient compression algorithms tailored to specific use cases, serving as a conceptual precursor to the modular caching strategies discussed in the following section.

Beyond clustering and low-rank methods, some strategies focus on learning to compress the context directly within the model architecture. [63] presents a technique that enables lossless compression of the KV cache through a specialized training procedure and an optimized autoregressive generation algorithm. LoMA compresses the cache after a fixed number of generated tokens, achieving significant reductions in computational consumption and memory usage within a single inference pass. Furthermore, [64] proposes a method where the model learns to apply different compression

rates across different heads and layers via continued pre-training. This adaptive approach allows for fine-grained control over the compression trade-off, retrofitting existing models like Llama 2 to achieve up to 3.7x throughput increases while preserving downstream performance.

Theoretical advancements in clustering also support these practical implementations. [65] explores the combination of dimensionality compression and vector quantization, proposing a Low-Rank Representation Vector Quantization method that optimizes vectors for clustering performance. Although initially focused on vision models, the principles of reducing vector dimensionality to improve clustering efficiency are directly applicable to optimizing the high-dimensional key vectors in LLM attention mechanisms. Additionally, [66] introduces a VIP-token centric compression scheme that selectively compresses sequences based on their impact on approximating the representation of “Very Important” tokens, scaling effectively to ultra-long sequences of 128K tokens or more.

Collectively, these context compression and clustering strategies represent a paradigm shift from merely managing memory limits via selection to fundamentally altering how context is encoded. By grouping similar tokens, approximating residuals with low-rank structures, and leveraging semantic hierarchies in dialogues, these methods achieve sublinear memory complexity while mitigating the information loss associated with hard eviction. They form a critical bridge in the efficiency triad: whereas Section 2.3 optimizes *which* tokens to access, and the subsequent Section 2.5 explores how to *reuse* cached states across multiple turns, this section focuses on *how* to represent those states more densely. The integration of these techniques into serving frameworks promises to unlock new capabilities for processing massive contexts that were previously infeasible due to memory bottlenecks, setting the stage for architectures that treat conversation history as a persistent, shareable resource.

2.5 Attention Reuse and Modular Caching for Multi-Turn Interactions

2.5 Attention Reuse and Modular Caching for Multi-Turn Interactions

While the preceding section explored strategies to compress the internal representation of context within a single inference pass, a distinct challenge arises in conversational AI applications where the context spans multiple interaction turns. In standard autoregressive serving frameworks, systems are typically stateless across requests, necessitating the re-computation of the Key-Value (KV) cache for the entire preceding dialogue history with every new turn. As conversations lengthen, this redundancy induces a linear increase in latency and computational waste, creating a bottleneck that compression alone cannot resolve. To address this inefficiency, recent research has pivoted towards attention reuse and modular caching strategies. These approaches treat the KV cache not as a transient byproduct of generation, but as a persistent, shareable resource that can be archived, retrieved, and reused across the lifecycle of a conversation.

A primary avenue of innovation involves the development of hierarchical storage systems designed to persist KV caches beyond the immediate GPU memory limits. [67] proposes a comprehensive solution that leverages cost-effective memory and storage mediums to archive KV caches for all historical turns. By decoupling positional encoding from the cache content, [67] ensures that saved caches remain valid even when the context window shifts or overflows, effectively preventing the invalidation of previously computed states. Furthermore, [67] employs sophisticated scheduler-aware fetching and eviction schemes, alongside layer-wise pre-loading and asynchronous saving, to overlap data movement with GPU computation. This architectural shift allows the system to reduce the Time to First Token (TTFT) by up to 87% and lower end-to-end inference costs by 70% for multi-turn scenarios, demonstrating the profound impact of treating conversation history as a

first-class citizen in memory management.

Parallel to hierarchical storage for dialogue history, the concept of modular caching has emerged to exploit structural redundancies inherent in prompt engineering across different user sessions. Many LLM applications utilize shared system prompts, instruction templates, or reference documents that appear repeatedly. [68] introduces a schema-based approach where these frequently occurring text segments are explicitly defined as “prompt modules.” By precomputing and storing the attention states of these modules, [68] enables the inference server to instantly retrieve and reuse these states whenever the corresponding text segment appears in a new prompt. This modularity not only accelerates inference but also ensures positional accuracy during state reuse without requiring any modifications to the underlying model parameters. The efficacy of this approach is particularly pronounced in document-based question answering and recommendation systems, where [68] reports latency reductions ranging from 8x on GPUs to an astonishing 60x on CPUs.

Beyond explicit modular definitions, efficient prefix sharing mechanisms have been developed to dynamically detect and exploit commonalities in input sequences, particularly in multi-tenant serving scenarios. [69] addresses the optimization of environments where multiple requests often share system prompt prefixes. By breaking monolithic key/value tensors into smaller chunks and organizing them into an auxiliary prefix tree, [69] enables the runtime sharing of KV tensors across requests. This structure is complemented by a two-phase partition algorithm within the self-attention kernel, which improves data locality during computation. Similarly, [3] decomposes the attention operation to compute shared prefixes and unique suffixes separately. [3] batches queries together across sequences for the shared prefix portion, transforming inefficient matrix-vector products into hardware-friendly matrix-matrix multiplications. This decomposition allows [3] to achieve throughput improvements of up to 32x, with performance gains scaling alongside batch size and prefix length.

The challenge of maintaining statefulness in traditionally stateless serving engines is further addressed by systems that generalize memory management to support non-contiguous storage and optimized dataflows. [70] designs a multi-tier caching strategy that utilizes both GPU and CPU memory to store conversation history, thereby avoiding duplicate processing of historical logs. [70] extends the PagedAttention kernel to support attention mechanisms over GPU caches spread across non-contiguous memory regions, achieving significant throughput improvements over state-of-the-art stateless systems. Additionally, for scenarios involving extremely long system prompts that cause redundant memory accesses in batched requests, [71] proposes an algorithm that reads hidden states from DRAM exactly once for a batch of input tokens. [71] achieves this through a mathematical reformulation of causal attention, eliminating the redundancy of transferring cached states for each individual request in a batch.

Collectively, these advancements signify a paradigm shift from ephemeral, request-local caching to persistent, global, and modular memory architectures. By effectively reusing precomputed KV caches through hierarchical storage [67], modular schemas [68], prefix trees [69], and decomposed attention mechanisms [3], modern serving systems can drastically reduce the computational overhead of multi-turn interactions. While the previous section focused on compressing the representation of context to fit within memory limits, and the following section will explore restructuring the attention mechanism itself via hierarchical and tree-based models to reduce algorithmic complexity, this section highlights how intelligent memory management and reuse can unlock efficiency without altering the fundamental attention computation or model architecture. These strategies enable the practical deployment of LLMs in complex, long-context conversational environments that were previously prohibitive due to memory and compute constraints.

2.6 Hierarchical and Tree-Based Attention Structures

2.6 Hierarchical and Tree-Based Attention Structures

While the preceding section demonstrated how intelligent memory management and reuse can alleviate the redundancy of recomputing KV caches in multi-turn interactions, a more fundamental bottleneck remains when dealing with ultra-long sequences within a single context window: the intrinsic quadratic complexity of standard self-attention. As sequence lengths extend into the hundreds of thousands or millions of tokens, the $O(n^2)$ scaling of traditional transformers renders them impractical for applications such as long-document understanding, genomic analysis, and high-resolution video processing. Even with optimized caching strategies, the sheer volume of pairwise interactions creates insurmountable computational and memory demands. To address this, a distinct class of architectural modifications has emerged that replaces the flat, all-to-all attention matrix with hierarchical or tree-based structures. These approaches leverage divide-and-conquer strategies to reduce complexity to linear or near-linear time while striving to maintain the global receptive field essential for coherent language generation. By restructuring the algorithmic foundation of attention itself, these methods complement the memory-centric optimizations discussed previously, offering a pathway to scalability that compression and reuse alone cannot achieve.

A foundational concept in this domain is the adaptation of fast summation methods from computational physics, specifically the Fast Multipole Method (FMM), to deep learning. The [72] paper introduces a novel mechanism that organizes queries, keys, and values into a hierarchy of resolution levels. By grouping tokens into clusters where distant interactions are approximated at coarser resolutions, this approach achieves a time and memory complexity of $O(n \log n)$ or even $O(n)$, depending on specific implementation choices regarding query down-sampling. This hierarchical grouping allows the model to consider global context without explicitly computing every pairwise interaction, effectively mitigating the computational explosion that constrains standard attention. Similarly, the theoretical underpinnings of these methods are reinforced by research into the communication complexity of such hierarchical algorithms, as discussed in [73], which highlights how tree-like data structures can achieve asymptotically optimal computational and communication complexity on distributed architectures.

Parallel to physics-inspired methods, decision tree-based approaches have been developed to reframe attention as a nearest-neighbor retrieval problem. The [74] architecture exemplifies this strategy by utilizing hierarchical navigation through decision trees to reduce the retrieval cost per query token from linear to nearly logarithmic relative to the sequence length. Treeformer introduces two specific attention layers: TF-Attention for fine-grained computation and TC-Attention, a coarse layer designed to ensure dense gradients during training. Through a specialized two-level bootstrapped training method, Treeformer demonstrates the ability to match the accuracy of baseline Transformers while utilizing significantly fewer FLOPs, reportedly up to 30 times less in the attention layer. This indicates that tree-based navigation can effectively prune the search space for relevant context without sacrificing model performance.

Further advancements in hierarchical structures include the integration of Hierarchical Matrices (H-Matrices) from numerical analysis into transformer architectures. The [75] proposes a method that exploits matrix structures similar to H-Matrices to achieve linear run time and memory complexity. This approach embeds an inductive bias that is particularly effective for capturing the inherent hierarchical structures found in natural language and vision tasks. Empirical evaluations show that such hierarchical attention mechanisms can outperform alternative sub-quadratic proposals by significant margins on benchmarks like the Long Range Arena, while also setting new state-of-

the-art perplexity scores with fewer parameters.

In the realm of vision and multimodal tasks, where spatial hierarchy is intrinsic, quadtree-based structures have proven highly effective. The [76] method constructs token pyramids to compute attention in a coarse-to-fine manner. At each level of the hierarchy, only the top-K patches with the highest attention scores are selected for finer-grained evaluation in subsequent levels. This dynamic pruning ensures that computational resources are focused on relevant regions, achieving linear complexity and substantial reductions in FLOPs while improving accuracy on tasks such as object detection and semantic segmentation. Complementary to this, the [77] addresses the abrupt granularity changes in traditional sparse attention by constructing a multi-scale tree that provides a smoother transition from local to global context, thereby enhancing the model’s ability to capture long-range dependencies in text summarization tasks.

Beyond static hierarchical definitions, recent works have explored dynamic and learnable hierarchical factorizations. The [78] proposes treating self-attention as a conditional expectation, approximating the distribution through structured factorization where tokens attend to abstractions of local regions. This allows for full attention capability with sub-quadratic costs, effectively bridging the gap between sparse efficiency and expressiveness. Additionally, the [79] utilizes a multi-stage chunking strategy that progressively applies different chunk sizes, allowing the model to learn both local fluctuations and global trends in time-series data without altering the total input length.

Collectively, these hierarchical and tree-based structures represent a paradigm shift from brute-force computation to intelligent data organization. By restructuring the flow of information through trees, pyramids, or hierarchical clusters, these models decouple performance from sequence length, addressing the algorithmic complexity limitations that persist even after applying the caching and reuse strategies outlined in the previous section. However, realizing the full potential of these architectures in production serving environments requires more than just algorithmic innovation; it demands tight integration with the underlying hardware. The irregular memory access patterns and complex data dependencies introduced by tree traversals and hierarchical clustering can exacerbate bandwidth bottlenecks if not carefully managed. For instance, the [26] work, while not strictly a tree architecture, emphasizes the critical importance of tailored dataflows and tiling to mitigate off-chip bandwidth bottlenecks—a principle that becomes even more vital when implementing efficient hierarchical attention on physical hardware. Consequently, the transition from theoretical algorithmic efficiency to real-world throughput necessitates the hardware-aware dataflows and IO optimization strategies that will be explored in the following section, ensuring that the benefits of hierarchical attention are not lost to memory latency.

2.7 Hardware-Aware Dataflows and IO Optimization

2.7 Hardware-Aware Dataflows and IO Optimization

While hierarchical and tree-based structures discussed in the previous section address algorithmic complexity by restructuring attention computations, their practical efficacy in serving environments is ultimately bounded by physical hardware constraints. The deployment of Large Language Models (LLMs) is increasingly constrained not by computational FLOPs, but by the “memory wall,” where the latency and energy costs of moving data between off-chip memory and on-chip compute units dominate execution time. In the context of autoregressive generation, the Key-Value (KV) cache grows linearly with sequence length, exacerbating bandwidth bottlenecks during the decoding phase. To bridge the gap between theoretical algorithmic efficiency and real-world performance, recent research has shifted towards hardware-aware dataflows and algorithm-architecture co-design

strategies that fundamentally restructure how attention mechanisms interact with the memory hierarchy. These approaches prioritize minimizing off-chip data movement through fused kernels, advanced tiling schemes, and specialized memory layouts tailored to specific hardware substrates such as GPUs and FPGAs.

A primary strategy in overcoming memory bandwidth limitations is the fusion of operations to eliminate intermediate memory writes. Traditional attention implementations often materialize large $N \times N$ attention matrices in global memory before applying softmax and value multiplication, leading to quadratic memory growth and excessive I/O. Hardware-aware dataflows mitigate this by fusing the matrix multiplication, softmax, and value projection into a single kernel execution, ensuring that intermediate results remain in fast on-chip SRAM or registers. For instance, the development of FlashAttention-2 exemplifies this paradigm by optimizing work partitioning across thread blocks and warps on NVIDIA GPUs. By reducing non-matmul FLOPs and parallelizing computation within thread blocks, FlashAttention-2 achieves significant speedups and higher FLOPs utilization compared to previous iterations, effectively addressing the suboptimal work partitioning that previously limited performance [80]. Furthermore, implementing such fused kernels requires careful tuning of tile sizes to balance register pressure and shared memory utilization, as demonstrated in optimized implementations targeting modern architectures like NVIDIA Hopper, which leverage specific hardware features such as the Tensor Memory Accelerator (TMA) to overlap memory copies with computation [81].

Building upon the principles of kernel fusion, tiling approaches have emerged as a critical mechanism for transforming the memory footprint of attention mechanisms from quadratic to linear, directly enabling the processing of the ultra-long sequences targeted by hierarchical methods. The FLAT (Fused Linear Attention Tile) dataflow represents a significant advancement in this domain, specifically designed to mitigate attention bottlenecks without altering the underlying mathematical functionality. FLAT employs a unique fusion mechanism combined with a bespoke tiling approach that enhances data reuse across attention operations. This method effectively reduces the off-chip bandwidth bottleneck and lowers on-chip memory requirements, enabling the processing of sequences up to 64K tokens on hardware with scarce on-chip resources. Evaluations indicate that FLAT delivers substantial speedups and energy savings compared to state-of-the-art accelerators lacking such customized dataflow optimizations, proving that algorithmic restructuring can unblock transformer models for ultra-long contexts [26]. The success of FLAT underscores the necessity of moving beyond generic deep learning dataflows, which often hit efficiency limits for inputs larger than 512 elements, toward attention-specific optimizations that maximize temporal locality.

In heterogeneous and reconfigurable computing environments, such as FPGAs, IO optimization extends beyond kernel logic to the physical layout of data in memory. The efficiency of data transfers is heavily dependent on memory access patterns; irregular or strided accesses can severely underutilize available bandwidth. Research into FPGA accelerators has shown that modifying the data layout to enable burst-friendly memory accesses can dramatically increase effective bandwidth. By employing compiler-based approaches to reorganize data layouts for contiguous access and utilizing runtime compression techniques, it is possible to decrease I/O cycles significantly compared to un-optimized access patterns [82]. Similarly, techniques that ensure data is contiguous in external memory allow accelerators to utilize burst transfers rather than element-wise accesses, pushing up memory throughput with minimal logic overhead [83]. These hardware-specific optimizations are crucial for LLM serving on edge devices or specialized accelerators where memory bandwidth is a hard constraint.

Furthermore, the co-design of algorithms and hardware extends to the management of data move-

ment in spatial architectures and near-memory computing paradigms, offering scalable solutions for distributed serving. In tiled spatial architectures, the efficiency of interconnections between tiles is paramount. Software-hardware co-design frameworks like METRO decouple traffic scheduling policies from hardware fabrics, moving scheduling logic to the software level to coordinate traffic more efficiently. This approach has been shown to achieve significant communication speedups and processing time reductions compared to traditional on-chip network designs, highlighting the potential of synergistic software-hardware optimization [84]. Additionally, the concept of Processing-in-Memory (PIM) offers a radical solution to the data movement bottleneck by placing computation capabilities close to or inside memory units. While naively employing PIM can lead to sub-optimal performance due to design constraints, co-designing specialized PIM accelerators with algorithms tailored for data-intensive workloads can yield high-performance and energy-efficient architectures [85]. The integration of logic layers in 3D-stacked memory further enables near-data acceleration, provided that challenges regarding virtual memory abstractions and coherence are addressed through efficient mechanisms that minimize unnecessary off-chip communication [86].

Finally, advanced scheduling and mapping techniques play a vital role in optimizing IO for complex neural network structures, ensuring that the benefits of hierarchical attention and fused kernels are fully realized across diverse hardware topologies. Frameworks that explore the intrinsic connection between network structures and memory features, such as Cocco, utilize graph-level execution schemes to minimize communication overhead and bandwidth requirements with smaller memory capacities [87]. Similarly, for streaming architectures on FPGAs, smart off-chip eviction mechanisms allow for the mapping of modern CNNs with long skip connections to devices with limited on-chip memory, effectively utilizing off-chip memory as a buffer to maintain high throughput [88]. Collectively, these hardware-aware dataflows and IO optimization strategies form the bedrock of efficient LLM serving. By tightly coupling algorithmic innovations with the physical realities of hardware memory hierarchies, these methods ensure that the growing demands of KV cache management do not stall the generative process, paving the way for the scalable deployment of large models across diverse computing platforms.

3 Accelerating Token Generation via Speculative and Parallel Decoding

3.1 Foundations of Speculative Decoding and Draft-then-Verify Frameworks

3.1 Foundations of Speculative Decoding and Draft-then-Verify Frameworks

The deployment of Large Language Models (LLMs) in real-world applications has revealed a critical inefficiency inherent to their autoregressive nature: the sequential generation of tokens. In standard decoding, each new token depends entirely on the previously generated sequence, forcing the model to perform a full forward pass through all parameters for every single token produced. This process creates a severe memory bandwidth bottleneck, as the system must repeatedly load massive weight matrices from high-bandwidth memory to compute relatively small amounts of arithmetic operations per step. Consequently, modern GPU accelerators often remain underutilized, waiting for memory transfers rather than performing compute-intensive operations. To address this fundamental latency constraint, the paradigm of speculative execution, borrowed from computer architecture, has been adapted for LLM inference. The core principle of this approach is the “draft-then-verify” framework, which aims to break the strict sequential dependency of token generation by parallelizing the verification of multiple candidate tokens [89].

At the heart of speculative decoding lies the utilization of a smaller, faster “draft” model to generate a sequence of candidate tokens. This draft model, possessing significantly fewer parameters than the target LLM, can generate a block of tokens autoregressively at a fraction of the computational cost and time required by the larger model. Once this candidate sequence is drafted, the target LLM does not simply accept these tokens; instead, it performs a single, parallel forward pass to verify the entire sequence simultaneously. This verification step involves computing the probability distribution for each drafted token given the true context and applying a modified rejection sampling scheme. If a drafted token aligns with the target model’s distribution, it is accepted; if not, it is rejected, and the generation process resumes from the last accepted token. Crucially, this methodology preserves the exact output distribution of the target model, ensuring that acceleration does not come at the expense of generation quality or fidelity [90].

The theoretical foundation of this framework relies on the observation that the latency of scoring a short sequence of continuations in parallel is often comparable to, or only marginally higher than, the latency of scoring a single token sequentially, due to the memory-bound nature of the operation. By shifting the workload from multiple sequential target model passes to a single parallel verification pass, speculative decoding effectively amortizes memory access costs over multiple tokens. However, the efficacy of this approach is heavily dependent on the acceptance rate of the draft tokens, which is a function of the alignment between the draft and target models. Research indicates that performance gain is not strictly correlated with the draft model’s standalone language modeling capabilities but rather with its latency and its ability to predict the specific output distribution of the target model under given decoding conditions [91]. This reliance on acceptance rates establishes a clear motivation for advancing beyond simple linear chains, a topic explored in depth in the following section regarding tree-based mechanisms.

To optimize the draft-then-verify process, various strategies have been proposed to enhance the quality of the initial drafts. One prominent direction involves knowledge distillation, where the draft model is explicitly trained to align its output distribution with that of the target model. Techniques such as DistillSpec utilize on-policy data generation and tailored divergence functions to minimize the discrepancy between the two models, resulting in significantly higher acceptance rates and substantial speedups compared to standard speculative decoding approaches [92]. Furthermore, the concept of online adaptation has been introduced to dynamically update draft models based on observed user query distributions. By leveraging surplus computational power in serving clusters to retrain draft models in real-time, systems can mitigate distribution shifts and maintain high prediction accuracy across diverse input domains [93].

Beyond the use of separate small models, architectural innovations have emerged to integrate drafting capabilities directly within the target model or through lightweight additions. For instance, methods like Medusa introduce multiple decoding heads attached to the backbone LLM, allowing the model to predict several future tokens in parallel without requiring a distinct draft model. This approach simplifies the deployment pipeline while maintaining the benefits of parallel verification [94]. Similarly, Hydra heads propose a sequentially dependent mechanism where draft predictions condition on preceding drafted tokens within the same step, enhancing speculation accuracy over independent head architectures [95]. Other frameworks, such as PaSS, explore parallel speculative sampling using a single model by marking specific input tokens for simultaneous generation, thereby eliminating the need for a separate model entirely while achieving notable speed improvements [96].

The verification mechanism itself has also undergone rigorous refinement to maximize throughput. The standard rejection sampling algorithm ensures that the final sequence follows the target model’s distribution exactly, but variations have been developed to handle non-greedy sampling and to op-

timize the trade-off between latency and task performance. For example, some systems incorporate lossy speculative decoding to allow for fine-grained control over this trade-off, accepting a negligible drop in quality for significant latency reductions in specific scenarios [92]. Additionally, the interaction between speculative decoding and batching strategies is critical; the optimal length of the speculative draft often varies with batch size, necessitating adaptive control mechanisms that dynamically adjust speculation parameters to sustain high hardware utilization across different workloads [97].

In summary, the draft-then-verify framework represents a transformative shift in LLM serving, converting a memory-bound sequential process into a more compute-efficient parallel workflow. By decoupling the generation of candidate tokens from their final validation, speculative decoding leverages the complementary strengths of fast, approximate drafting and accurate, parallel verification. While these foundational linear approaches provide significant speedups, they are inherently limited by the probability of accepting an entire sequential chain; a single rejection invalidates all subsequent tokens. This limitation motivates the evolution toward more sophisticated structures capable of recovering from partial mismatches, setting the stage for the tree-based speculative inference and verification mechanisms discussed in the next subsection. As the field evolves, the integration of advanced distillation techniques, adaptive online learning, and novel architectural heads continues to push the boundaries of inference speed, making large-scale language models increasingly viable for latency-sensitive applications [89].

3.2 Tree-Based Speculative Inference and Verification Mechanisms

3.2 Tree-Based Speculative Inference and Verification Mechanisms

While the foundational draft-then-verify frameworks discussed in Section 3.1 successfully parallelize verification, they remain constrained by the fragility of linear speculation. In standard speculative decoding, the acceptance of a draft sequence is binary for any given position: if a single token in the linear chain is rejected, all subsequent tokens in that chain are discarded, regardless of their potential validity. This “all-or-nothing” dynamic often leads to wasted computation, particularly when the draft model’s confidence diverges from the target model further down the sequence. To overcome this inefficiency, recent research has pivotally shifted toward tree-based speculative inference and verification mechanisms. By organizing draft tokens into branching structures rather than rigid linear chains, these approaches maximize token diversity and significantly increase the expected number of accepted tokens per verification step, effectively recovering from partial mismatches that would terminate a linear draft.

The core principle of tree-based speculative decoding is the simultaneous exploration of multiple potential continuations. Instead of generating a single sequence of k tokens, the draft model constructs a tree where each node represents a token and edges represent conditional probabilities. This topology allows the target model to evaluate multiple hypotheses in a single forward pass using specialized attention masks, such as sparse or tree attention. A critical advancement in managing the computational overhead of these expanding structures is dynamic token tree pruning, exemplified by the ProPD framework. ProPD addresses the challenge of balancing tree breadth with verification costs by introducing an advanced early pruning mechanism. This strategy efficiently eliminates unpromising token sequences before they consume significant verification resources, dynamically adjusting the tree generation algorithm to balance computation and parallelism in real-time. Empirical evaluations demonstrate that such adaptive management allows ProPD to consistently outperform existing decoding algorithms by factors ranging from 1.1x to 3.2x across varying batch

sizes and task complexities [98].

Beyond structural management, the methodology used to construct the draft tree is paramount for maximizing diversity. Traditional methods often generate tokens at each level independently, which can lead to redundant paths that fail to cover the target distribution effectively. Recursive Speculative Decoding (RSD) offers a novel solution by sampling draft tokens without replacement, a technique that inherently maximizes the diversity of the generated tree. RSD employs sophisticated sampling strategies, such as the Gumbel-Top- k trick or Stochastic Beam Search, to draw tokens in parallel while avoiding redundancy. This ensures that the branching factor covers a broader spectrum of the probability distribution, thereby increasing the aggregate acceptance rate during verification. Furthermore, RSD incorporates mechanisms for the early truncation of unlikely draft sequences, reducing the computational burden on the target model. Experimental results on models such as Llama 2 and OPT confirm that RSD outperforms baseline tree-based methods, particularly under fixed computational budgets, making it highly suitable for resource-constrained devices [99].

The theoretical underpinnings of these verification mechanisms have also been refined through the lens of optimal transport, providing a rigorous mathematical foundation for tree construction. The SpecTr framework frames speculative decoding as an optimal transport problem with membership cost, generalizing the method to allow for a set of k candidates at the token level. This perspective leads to an improved optimal membership cost and enables a draft selection algorithm that achieves near-optimal acceptance probability with almost linear time complexity relative to the token domain size. By optimizing the transport plan between the draft and target distributions, SpecTr demonstrates that mathematically principled tree construction can yield substantial wall-clock speedups without compromising output quality [100].

Building upon these structural and theoretical advances, multi-candidate speculative decoding approaches further extend the utility of tree structures by organizing sampled candidates into batches for verification. This method leverages the branching concept to sample multiple diverse candidates from a draft model and verify them simultaneously, strictly maintaining the target model’s distribution. The design of efficient verification algorithms for these multi-candidate batches is essential to realizing the theoretical benefits of increased acceptance rates. Studies indicate that organizing candidates in this manner consistently outperforms standard linear speculative decoding across various datasets and models, validating the hypothesis that branching structures provide a more robust defense against the stochastic nature of language generation [101].

Additionally, the integration of semantic awareness and feedback loops into the drafting process has shown promise in enhancing tree quality. The SDSAT approach improves the initial quality of branches by incorporating semantic adaptive tokens. While not strictly a tree-pruning algorithm, its “two-step-draft-then-verify” strategy complements tree-based mechanisms by reducing the need for aggressive pruning and increasing the depth of accepted paths. When combined with tree-based verification, such semantic enhancements can yield speed increases of over 3.0x to 3.5x, illustrating the synergy between improved draft quality and advanced verification architectures [102]. Similarly, the Ouroboros framework constructs a phrase candidate pool from the verification process to inform future draft generation, effectively creating a feedback loop that refines the tree structure over time and achieves speedups of up to 2.8x compared to standard speculative decoding [103].

In conclusion, tree-based speculative inference and verification mechanisms represent a decisive evolution from the linear constraints of early speculative decoding. By transitioning to branching structures and employing sophisticated techniques such as dynamic pruning [98], sampling without replacement [99], and optimal transport-based verification [100], these methods effectively mitigate

the waste associated with partial rejections. The ability to dynamically adapt tree structures to hardware constraints and input characteristics ensures robustness across diverse deployment scenarios. These advancements in organizing and verifying diverse token paths naturally lead to further innovations in parallelism, specifically the multi-candidate and blockwise strategies explored in the following section, which seek to extend speculation beyond hierarchical trees to maximize arithmetic intensity and hardware utilization.

3.3 Multi-Candidate and Blockwise Parallel Decoding Strategies

3.3 Multi-Candidate and Blockwise Parallel Decoding Strategies

Building upon the tree-based verification mechanisms discussed in the previous section, which maximize token diversity through branching structures, further innovations have emerged to extend speculation beyond linear sequences and optimize hardware utilization. While tree-based approaches focus on organizing draft tokens into hierarchical structures for parallel verification, this subsection explores two complementary paradigms: multi-candidate speculative decoding, which leverages batched diversity to increase acceptance probabilities, and Blockwise Parallel Decoding (BPD), which targets the parallel generation of token blocks to maximize arithmetic intensity. These strategies collectively address the limitations of single-path speculation, particularly when significant capability gaps exist between draft and target models, by better aligning draft distributions with the target model’s preferences.

A primary avenue of innovation involves multi-candidate speculative decoding, designed to overcome the diminishing acceptance rates inherent in single-path speculation as sequence lengths grow or contextual complexity increases. Unlike traditional methods that generate a single candidate segment, recent works propose sampling multiple diverse candidates from a draft model and organizing them into batches for simultaneous verification. This approach capitalizes on the statistical likelihood that among several candidate paths, at least one will closely align with the target model’s preferred trajectory. For instance, the work titled [101] explicitly designs algorithms to efficiently verify these batched candidates while rigorously maintaining the output distribution of the target model. Empirical results demonstrate that this multi-candidate strategy yields significant improvements in acceptance rates across various datasets and models, consistently outperforming standard speculative decoding schemes reliant on a single candidate stream. Furthermore, the synergy between such speculative mechanisms and batching is critical for real-world serving scenarios. Research presented in [97] highlights that the optimal length for speculation is not static but dependent on the batch size utilized during inference. By implementing an adaptive strategy that dynamically selects the optimal speculation length based on batch characteristics, systems can achieve performance metrics equal to or better than state-of-the-art fixed-length schemes, ensuring high GPU utilization without compromising latency constraints.

Parallel to the multi-candidate approach is the refinement of Blockwise Parallel Decoding (BPD), a technique originally proposed to predict multiple time steps in parallel before reverting to the longest validated prefix. Early iterations of BPD, such as those described in [104], demonstrated the potential for substantial theoretical speedups, achieving iteration reductions of up to 2x over greedy decoders with no loss in quality, and even higher speedups with minor performance trade-offs. However, the efficacy of BPD is heavily contingent on the quality of the initial drafts. Recent advancements have focused on refining these drafts using auxiliary information to boost the average verified prefix length. The paper [105] contributes significantly to this area by analyzing the token distributions produced by BPD prediction heads. Based on this analysis, the authors introduce

algorithms that refine BPD drafts using small n-gram models or neural language models. These refined drafts result in a higher average verified prefix length, directly translating to improved inference speeds across diverse tasks. Additionally, lossless acceleration methods have been developed to further enhance BPD. The approach known as Adaptive N-gram Parallel Decoding (ANPD), detailed in [106], employs a two-stage process involving a rapid drafting phase with a context-adaptive N-gram module followed by a verification phase. This method preserves the integrity of the original LLM output while eliminating the need for retraining or additional GPU memory, achieving speed improvements of up to 3.67x on models like LLaMA.

Another promising direction within this domain is single-model self-speculation, which eliminates the overhead of managing separate draft models entirely, bridging the gap between external drafting and internal architectural modifications. Techniques such as Parallel Speculative Sampling (PaSS), introduced in [96], propose using parallel decoding within a single model to draft multiple tokens. This approach requires only an additional input token to mark words generated simultaneously, avoiding the complexity of aligning tokenizers between different models and requiring merely $O(d_{emb})$ additional parameters. Similarly, the concept of “hidden transfer” explored in [107] decodes multiple successive tokens in a single forward pass by transferring intermediate hidden states to pseudo hidden states of future tokens. This method utilizes a novel tree attention mechanism to generate and verify multiple candidates simultaneously, ensuring lossless generation while outperforming other single-model acceleration techniques. Furthermore, recurrent architectures have been adapted for this purpose; the [108] introduces a lightweight draft head with recurrent dependency, allowing for efficient beam search filtering without the complex tree structures required by other methods like Medusa.

Collectively, these multi-candidate and blockwise strategies represent a paradigm shift from rigid, sequential drafting to flexible, parallel exploration of the token space. Whether through the diversification of candidate paths to hedge against prediction errors, the intelligent refinement of blocks using n-gram guides, or the integration of drafting mechanisms directly into the target model architecture, these methods effectively address the memory-bound nature of autoregressive generation. By increasing the number of tokens accepted per verification step and maximizing the parallel compute capacity of modern accelerators, they offer a robust pathway toward ultra-low latency LLM serving. These algorithmic and methodological advances set the stage for the next layer of optimization: fundamental architectural innovations that restructure the model itself to facilitate inherent parallelism, as detailed in the following section on Tandem Models and Early Exiting.

3.4 Architectural Innovations for Parallelism: Tandem Models and Early Exiting

3.4 Architectural Innovations for Parallelism: Tandem Models and Early Exiting

While the multi-candidate and blockwise strategies discussed in Section 3.3 optimize token exploration through algorithmic rearrangement and external drafting, they largely operate within the constraints of standard autoregressive architectures. To fundamentally dismantle the sequential bottleneck inherent in token generation, recent research has pivoted toward intrinsic architectural innovations that restructure the model itself. This section examines two such paradigms that facilitate parallel generation from the ground up: **Tandem Transformers**, which decouple prediction and validation through heterogeneous model pairing, and robust **Early-Exiting** frameworks that dynamically allocate computational depth based on input complexity. These approaches transition the optimization focus from *how* tokens are speculated to *where* and *when* computation occurs within the network topology.

Tandem Transformers represent a novel architectural evolution designed to resolve the accuracy-speed trade-off often encountered in traditional speculative decoding. Whereas standard methods rely on a distinct, smaller draft model that may lack the contextual richness of the target, the Tandem architecture uniquely couples a small autoregressive model with a large model operating in block mode. Crucially, this design grants the small predictor attention access to the rich representations generated by the large model, transforming it from a weak approximator into a highly informed predictor capable of processing multiple tokens simultaneously. Empirical evaluations on the PaLM2 pretraining dataset demonstrate the efficacy of this synergy, where a tandem configuration of PaLM2-Bison and PaLM2-Gecko achieved a 3.3% improvement in next-token prediction accuracy compared to a standalone PaLM2-Gecko [109]. Furthermore, when integrated into speculative decoding frameworks such as SPEED, this arrangement yields a substantial speedup of approximately 1.14x over vanilla baselines while maintaining identical downstream task accuracy [109]. By breaking sequential dependencies more effectively than standard draft-then-verify methods, Tandem architectures lay the groundwork for the inter-request parallelism discussed in the subsequent section.

Parallel to the development of tandem architectures, early-exiting mechanisms have evolved to provide dynamic computation paths tailored to the semantic complexity of specific inputs. The core premise posits that not all tokens require the full depth of the neural network for high-confidence prediction; simpler tokens can be generated using only initial layers, thereby conserving significant computational resources. However, conventional early-exiting frameworks often suffer from performance degradation due to state copying overheads, sensitivity to confidence thresholds, and the latency penalties of managing numerous exit paths. To address these shortcomings, the **Fast and Robust Early-Exiting (FREE)** framework introduces a shallow-deep module coupled with synchronized parallel decoding. By synchronizing the decoding process of the current token with previously stacked early-exited tokens, FREE enables faster inference while mitigating the latency penalties associated with traditional adaptive computation [110]. A key innovation in FREE is the introduction of a novel adaptive threshold estimator utilizing a Beta mixture model, which dynamically determines suitable confidence thresholds by observing predictions from both shallow and deep models, effectively balancing the trade-off between speed and accuracy without manual tuning [110].

The scalability of these architectural innovations is further supported by frameworks like **EE-LLM**, which extends early-exiting capabilities to massive models through 3D parallelism. Built upon Megatron-LM, EE-LLM implements algorithmic optimizations that leverage idle resources in pipeline schedules to compute early-exit layers, ensuring that acceleration benefits do not compromise training efficiency or output quality [111]. Similarly, the **Confident Adaptive Language Modeling (CALM)** framework addresses the theoretical challenges of connecting sequence-level constraints to local per-token exit decisions, demonstrating potential speedups of up to 3x while provably maintaining high performance [112]. These systems collectively illustrate that structural adaptations, when combined with sophisticated control mechanisms, can transform the rigid sequential nature of autoregressive generation into a flexible, parallelizable process. Moreover, the integration of these techniques highlights a broader trend toward hardware-aware co-design; synchronization mechanisms in FREE and block-mode operations in Tandem Transformers are explicitly engineered to maximize arithmetic intensity and minimize memory bandwidth saturation. The success of methods like **MuE**, which enables dynamic layer skipping in unified vision-language models, further validates the versatility of early exiting across modalities, reducing expected inference time by up to 50% with negligible performance loss [113].

In conclusion, architectural innovations such as Tandem Transformers and advanced early-exiting frameworks like FREE represent a significant leap forward in efficient LLM serving by restructuring the inference process to allow for intrinsic parallel execution and dynamic resource allocation. These approaches effectively decouple generation speed from model size, offering a robust solution to the efficiency trilemma while maintaining the fidelity of large-scale models. While these methods optimize the internal mechanics of individual requests, they primarily address single-stream latency. As the field progresses, the synergy between heterogeneous model pairing and adaptive computation depth sets the stage for the next layer of optimization: system-level orchestration. The following section explores how these intra-request efficiencies can be compounded through **temporal fusion**, which parallels the generation processes of multiple concurrent requests to maximize cluster-wide hardware utilization.

3.5 Temporal Fusion and System-Level Orchestration for Parallel Requests

3.5 Temporal Fusion and System-Level Orchestration for Parallel Requests

While architectural innovations like Tandem Transformers and early-exiting mechanisms (discussed in Section 3.4) effectively mitigate sequential dependencies within individual sequences by restructuring model depth or pairing heterogeneous models, they primarily address the efficiency of single-request latency. However, in real-world deployment scenarios, the challenge extends beyond a single stream; requests arrive stochastically with varying generation lengths, creating a systemic bottleneck where memory bandwidth utilization and compute resource allocation remain suboptimal. Standard inference pipelines, even those optimized architecturally, often struggle to maintain high hardware utilization when handling concurrent requests due to the serial nature of token generation. Traditional mitigation strategies, such as dynamic batching or running concurrent independent instances, frequently fall short; dynamic batching introduces latency penalties due to waiting for batch formation, while concurrent instances suffer from severe bandwidth contention and fragmented memory access patterns. To address these systemic inefficiencies, recent research has pivoted toward system-level orchestration strategies that employ **temporal fusion** to parallelize the generation processes of multiple concurrent requests, thereby transforming the memory-bound decoding workload into a more compute-efficient operation.

A pioneering framework in this domain is **Flover**, which introduces a novel temporal fusion mechanism designed to deconstruct the standard generation pipeline and reassemble it for optimal parallel execution [114]. The core insight driving Flover complements the intra-request parallelism of Section 3.4 by focusing on inter-request parallelism: while tokens within a single sequence must be generated sequentially, the generation steps of different requests can be interleaved or fused temporally to maximize hardware throughput. By equipping the framework with a dedicated work scheduler, Flover orchestrates token-level parallelism across all active requests, effectively fusing their generation processes. This approach allows the system to maintain a high density of active arithmetic operations, significantly sparing system resources that would otherwise remain idle due to memory latency. The framework separates the inference process into distinct pre-processing and token generation phases, enabling the scheduler to manage the flow of data and computation with granular precision. This temporal fusion strategy is particularly effective in alleviating the memory bandwidth bottleneck, as it consolidates memory access patterns across multiple requests, reducing the overhead associated with loading model weights for individual token generations.

The efficacy of Flover is demonstrated through its ability to deliver substantial performance improvements over state-of-the-art solutions. Experimental evaluations indicate that by leveraging

advanced tensor parallel techniques and its unique temporal fusion architecture, Flover achieves over an 11x inference speedup on GPT models and a remarkable 16x speedup on LLAMA models compared to NVIDIA FasterTransformer [114]. These gains are attributed not only to the algorithmic reordering of tasks but also to the framework’s robust adaptability across diverse computational landscapes, ranging from single-GPU setups to complex distributed scenarios. A critical component enabling this performance is the integration of a fast buffer reordering algorithm. This algorithm facilitates the efficient memory eviction of finished tasks, ensuring that the memory pool remains optimized for active requests and preventing the accumulation of stale data that could degrade performance [114]. By dynamically managing the lifecycle of requests within the fused timeline, the system maintains high hardware efficiency even under variable load conditions.

The principles underlying temporal fusion in LLM serving share conceptual similarities with broader efforts in heterogeneous computing and task scheduling, where the goal is to maximize resource utilization through intelligent orchestration. For instance, frameworks designed for heterogeneous systems often employ schedulers that dynamically distribute tasks among diverse processing resources, such as CPUs and FPGAs, to minimize load imbalance and exploit all available computing devices [115]. Similarly, the challenge of managing concurrent tasks on accelerators has led to the development of runtime schedulers that utilize temporal execution models to predict task durations and determine optimal execution orders, thereby reducing total execution time and increasing accelerator usage [116]. These parallels highlight a growing consensus in the systems community: that breaking away from rigid, static execution models in favor of dynamic, temporally aware scheduling is essential for unlocking the full potential of modern hardware architectures.

Furthermore, the success of temporal fusion relies heavily on the ability to manage memory and compute resources in a coordinated fashion, a challenge that extends beyond LLMs to general parallel computing environments. Approaches that enable I/O awareness in task-based programming models have shown that creating special tasks for executing I/O workloads, which can overlap with compute tasks, significantly mitigates performance degradation caused by congestion [117]. In the context of LLM serving, the “I/O” bottleneck is analogous to the memory bandwidth constraint during weight loading, and the “compute” corresponds to the matrix multiplication operations. Flover’s temporal fusion can be viewed as a specialized application of this overlap principle, where the memory access for one request is overlapped with the computation of another, effectively hiding latency. Additionally, the importance of compiler-assisted scheduling in managing GPU resources cannot be overstated. Frameworks that utilize compiler guidance to construct GPU tasks and instrument probes for resource requirements allow schedulers to place tasks on appropriate devices in a memory-safe and resource-aware manner, improving throughput and limiting performance degradation [118]. Such compiler-level insights are crucial for implementing the fine-grained control required by temporal fusion frameworks to operate efficiently on complex GPU architectures.

In conclusion, temporal fusion and system-level orchestration represent a paradigm shift in efficient LLM serving, moving beyond the model-centric optimizations discussed in Section 3.4 to holistic system design. By treating the generation of multiple requests as a unified, temporally fused workload rather than a collection of independent serial streams, frameworks like Flover successfully address the memory-bound nature of autoregressive decoding at the cluster level. The integration of dedicated schedulers, fast buffer reordering algorithms, and tensor parallelism enables these systems to achieve unprecedented speedups while maintaining robustness across different hardware configurations. As the demand for low-latency, high-throughput LLM inference continues to grow, the principles of temporal fusion will likely become foundational to next-generation serving architectures, bridging the gap between theoretical hardware capabilities and practical deployment perfor-

mance. This system-level efficiency sets the stage for the adaptive control mechanisms explored in Section 3.6, where the focus shifts from orchestrating fixed requests to dynamically regulating the decoding strategy and sampling budgets of individual generations based on real-time complexity and self-evaluation.

3.6 Adaptive Control, Self-Evaluation, and Cost-Efficient Sampling

3.6 Adaptive Control, Self-Evaluation, and Cost-Efficient Sampling

Building upon the system-level orchestration and temporal fusion strategies discussed in Section 3.5, which optimize throughput by parallelizing multiple concurrent requests, the focus now shifts to the granular optimization of individual generation trajectories. While temporal fusion addresses the macroscopic challenge of hardware utilization across a batch, it operates largely on fixed decoding pipelines. However, the inherent variability in token generation difficulty across different contexts implies that a static decoding strategy is often suboptimal for specific input sequences. Consequently, recent research has pivoted toward adaptive control mechanisms, self-evaluation metrics, and cost-efficient sampling strategies that dynamically balance computational expenditure with output accuracy at the sequence level. These approaches aim to refine the trade-off between draft length and verification overhead—concepts central to speculative decoding—by ensuring that resources are allocated adaptively only where necessary, thereby complementing the inter-request parallelism of Section 3.5 with sophisticated intra-request intelligence.

A critical component of this adaptive paradigm is the capability of models to self-regulate their decoding strategies based on immediate input characteristics. Traditional pipelines rely on manual, heuristic adjustments of hyperparameters such as temperature, top-k, and beam width, which often fail to generalize across diverse tasks. To overcome this rigidity, Hyperparameter Aware Generation (HAG) proposes a paradigm where LLMs are trained to autonomously determine the optimal decoding configuration for each specific input [119]. By leveraging hyperparameter-aware instruction tuning, models internalize the relationship between task complexity and optimal generation settings, effectively eliminating the need for cumbersome manual tuning. This self-regulation allows the system to dynamically switch between aggressive sampling for creative tasks and conservative, deterministic paths for reasoning-heavy queries, optimizing the latency-accuracy curve in real-time without external intervention.

Complementing self-regulation is the concept of self-evaluation, which acts as an intelligent gating mechanism for selective generation. Recognizing that not all tokens or reasoning paths require the full computational budget of the target model, self-evaluation techniques reformulate open-ended generation into token-level prediction problems. These methods leverage the model’s superior calibration to assess the quality of its own outputs [120]. By instructing the LLM to perform multi-way comparisons or point-wise evaluations of generated candidates, the system can identify low-confidence segments that necessitate further refinement or verification. This approach is particularly synergistic with speculative decoding frameworks; a “self-checking” module can decide whether to accept a draft token or trigger a fallback to the full model, streamlining the draft-then-verify process. Furthermore, methods like “Step CoT Check” demonstrate that meticulously designed training data can enhance a model’s intrinsic ability to detect errors in its reasoning chains without relying on external ground truth labels [121]. Such internal verification loops reduce dependency on expensive external validators, aligning perfectly with the efficiency goals of modern serving architectures.

Adaptive control also extends to the dynamic management of sampling budgets in complex reasoning tasks. Standard self-consistency approaches typically generate a fixed number of reasoning

paths, a strategy that is computationally wasteful for simple queries yet insufficient for complex ones. Adaptive-Consistency introduces a lightweight stopping criterion that dynamically adjusts the number of samples per question based on the agreement observed in generated paths thus far [122]. This method non-uniformly distributes the computational budget, halting generation early when consensus is reached and extending the search space only when uncertainty persists. Similarly, Early-Stopping Self-Consistency (ESC) derives control schemes to dynamically choose the performance-cost balance, significantly reducing the average number of sampling steps required for multi-step reasoning tasks without sacrificing accuracy [123]. These strategies exemplify how dynamic resource allocation can achieve substantial efficiency gains, often reducing sample budgets by several folds while maintaining near-identical performance levels.

However, the efficacy of self-correction and adaptive refinement is not universal and depends heavily on the quality of the feedback loop. Research indicates that LLMs often exhibit a “self-bias,” tending to favor their own generated outputs even when incorrect, which can amplify errors in iterative self-refinement pipelines [124]. To mitigate this, adaptive systems must incorporate mechanisms to detect stagnation or bias amplification. For instance, Self-Contrast addresses the instability of intrinsic reflection by adaptively exploring diverse solving perspectives and contrasting their discrepancies to form a robust checklist for re-examination [125]. This diversity-driven adaptive control helps overcome the overconfidence and randomness that plague single-perspective self-evaluation, ensuring that fallback mechanisms are triggered only when genuine uncertainty or conflict is detected.

Furthermore, the integration of specialized verifiers plays a pivotal role in cost-efficient sampling. The V-STaR framework illustrates how training verifiers on both correct and incorrect self-generated solutions can significantly improve the selection of optimal candidates during inference [126]. By iteratively refining both the reasoner and the verifier, the system learns to distinguish high-quality drafts from plausible but incorrect ones, thereby optimizing the acceptance rate in speculative decoding scenarios. This symbiotic relationship between generation and verification allows for longer draft sequences to be attempted with higher confidence, directly reducing the frequency of costly full-model invocations and enhancing the overall throughput of the serving system.

In summary, the evolution of efficient LLM serving extends beyond the system-level orchestration of Section 3.5 to include the seamless integration of adaptive control, self-evaluation, and cost-efficient sampling. By moving away from static decoding protocols toward systems that can self-regulate [119], dynamically adjust sampling budgets [122], and robustly evaluate their own outputs [120], next-generation serving frameworks achieve a more favorable balance between latency, throughput, and accuracy. These strategies ensure that computational resources are expended judiciously, adapting to the nuanced demands of each generation step while mitigating the risks of bias and error propagation. This shift toward intelligent, content-aware inference sets the foundation for subsequent discussions on hardware-specific optimizations, such as quantization and memory management, which further compress the computational footprint of these adaptive models.

4 Comprehensive Quantization Paradigms: Algorithms, Calibration, and Error Correction

4.1 Foundations of Post-Training Quantization and Calibration Metrics

4.1 Foundations of Post-Training Quantization and Calibration Metrics

Post-Training Quantization (PTQ) serves as a foundational pillar for efficient Large Language Model (LLM) serving, offering a pragmatic pathway to drastically reduce memory footprint and computational latency without the prohibitive costs of retraining. At its core, PTQ maps high-precision floating-point weights and activations to low-bit integer representations using a small, unlabeled calibration dataset. The central challenge lies in determining optimal quantization parameters—such as scaling factors and zero-points—to minimize performance degradation. Historically, methodologies have bifurcated into two primary philosophies: local feature-matching approaches that preserve intermediate layer representations, and global prediction-difference metrics that optimize directly for final output fidelity. Understanding the interplay between these paradigms is critical, as the choice of calibration strategy directly influences a model’s resilience to the distributional anomalies discussed in the following section.

Local feature-matching approaches operate on the premise that minimizing reconstruction error between full-precision and quantized activations at each layer cumulatively preserves overall model behavior. These methods typically employ metrics like Mean Squared Error (MSE) to align feature distributions. However, this localized view often fails to account for error propagation across deep architectures, a limitation that becomes acute when addressing the heavy-tailed activation distributions characteristic of LLMs. To address these shortcomings, recent advancements have introduced sophisticated reconstruction mechanisms. For instance, [127] proposes a novel algorithm that sequentially conducts coordinate-wise minimization of layer-wise reconstruction errors. By treating scaling factors and bit-codes as variables within a fixed layer and optimizing them through a carefully designed greedy order, [127] achieves near-lossless accuracy in 4-bit quantization without requiring hyper-parameter tuning or backpropagation. Similarly, [128] frames layer-wise quantization as a discrete-structured non-convex optimization problem, utilizing Coordinate Descent techniques to provide high-quality solutions that rely solely on matrix operations, thereby circumventing the need for complex matrix inversions while significantly outperforming traditional methods like GPTQ.

In contrast to local reconstruction, global prediction-difference metrics argue that minimizing intermediate feature distortion does not necessarily correlate with minimizing final task loss. This philosophy posits that quantization parameters should be selected based on their direct impact on the network’s output predictions rather than internal state alignment. [129] explicitly addresses the limitation of local information by proposing a method that determines quantization parameters using the differences between network predictions before and after quantization. By incorporating global information, [129] demonstrates superior performance in extremely low-bit settings, such as 2-bit weight and activation quantization, where local methods often fail due to overfitting on small calibration sets. Furthermore, [130] highlights the inadequacy of fixed metrics like the standard Lp-norm for complex tasks. The authors propose DetPTQ, a framework that adaptively assigns different p-values for the Lp metric across layers based on an Object Detection Output Loss, effectively bridging the gap between feature perturbation and task-specific performance degradation.

The optimization landscape of PTQ is further complicated by the non-differentiable nature of the

quantization operation, which traditionally necessitates gradient approximations or expensive re-training. However, a new class of backpropagation-free algorithms has gained traction by leveraging mathematical properties to optimize parameters directly. [131] introduces a unit-wise feature reconstruction algorithm grounded in the second-order Taylor series expansion of unit-wise error. By defining adjacent layers as a “Basic-Unit,” [131] leverages inter-layer parameter interactions to compensate for errors that layer-by-layer calibration misses, achieving near-original accuracy even at INT3 precision. Another innovative approach is presented in [132], which deviates from typical element-wise addition rounding mechanisms. [132] utilizes element-wise division to jointly learn a common quantization grid size and individual weight scales, allowing the method to flexibly quantize pre-trained weights based on their magnitudes with negligible performance impact on large language models.

Moreover, the stability of the optimization process itself has become a critical area of study, particularly as models scale. [133] identifies and theoretically proves that oscillation in PTQ methods arises from differences in module capacity between adjacent layers. The proposed solution involves selecting top-k differentials to jointly optimize corresponding modules, thereby stabilizing the quantization process and reducing performance drops significantly. Additionally, [134] rethinks the parameterization of quantized networks to allow for the unified, end-to-end finetuning of all degrees of freedom, including step sizes and preconditioning factors, in a single step. This holistic view contrasts sharply with multi-step chaining methods, offering a more efficient path to high-accuracy 4-bit quantization.

The choice of calibration metric also extends to the specific characteristics of the data and model architecture. [135] notes that activation distributions in Vision Transformers vary drastically across instances, rendering standard CNN-oriented PTQ methods ineffective. By dynamically splitting channels into groups with similar statistical properties for each input instance, this approach minimizes discrepancies under bit-operation constraints. Similarly, [136] borrows from self-supervised learning to propose a contrastive loss function that not only rewards similarity between quantized and full-precision outputs but also enhances the discriminability of features, proving particularly robust in extreme quantization scenarios. These diverse methodologies underscore that there is no singular “best” metric; rather, the efficacy of PTQ depends on aligning the calibration strategy—whether local, global, or hybrid—with the structural nuances of the target model. While these foundational techniques establish the mechanics of low-bit inference, they face a formidable adversary in the form of activation outliers, a challenge that necessitates the specialized mitigation strategies detailed in the subsequent section.

4.2 Mitigating Activation Outliers and Distribution Shifts

4.2 Mitigating Activation Outliers and Distribution Shifts

While the foundational PTQ methodologies discussed in Section 4.1 establish robust mechanisms for parameter calibration, their efficacy is frequently compromised by a distinct structural anomaly inherent to Large Language Models (LLMs): activation outliers. Unlike weights, which typically exhibit relatively uniform distributions amenable to standard quantization schemes, LLM activations are characterized by heavy-tailed distributions where a small subset of channels contains values with magnitudes orders of magnitude larger than the rest. These extreme values force the quantization range to expand significantly, thereby increasing the quantization step size and causing severe precision loss for the majority of non-outlier values. This phenomenon creates a bottleneck that local reconstruction and global prediction metrics alone cannot resolve, necessitating specialized mitiga-

tion strategies that manipulate the distribution of activations through channel-wise shifting, scaling, reassembly, and rotation. By smoothing the data landscape prior to quantization, these techniques bridge the gap between the theoretical optimality of Section 4.1 and the practical stability required for the advanced training regimes explored in Section 4.3.

A primary category of solutions involves equivalent affine transformations that strategically redistribute the quantization difficulty from activations to weights. The core insight driving these methods is that the output of a linear layer remains invariant if scaling or shifting factors are migrated between the activation and the subsequent weight matrix. [137] proposes a framework known as OS+, which explicitly targets the asymmetry and concentration of outliers through channel-wise shifting and scaling. By aligning the center of each channel to remove asymmetry and quantitatively evaluating the changes brought by migration, OS+ achieves near-floating-point performance even at 6-bit and 8-bit precisions. Similarly, [138] identifies the LayerNorm scaling parameter (γ) as a “sinful amplifier” that exacerbates outlier magnitudes. Their approach employs Gamma Migration to equivalently shift these amplification factors to subsequent modules, coupled with a token-wise clipping strategy that leverages the variance in token ranges to minimize quantization loss without incurring extra computational burdens during inference.

Beyond simple shifting and scaling, more advanced techniques focus on restructuring the channel dimensions themselves to isolate or dilute the impact of outliers. [139] introduces an adaptive channel reassembly technique designed specifically for ultra-low bitwidth scenarios. This method operates by dissecting outlier channels into multiple sub-channels to balance the distribution of activation magnitudes, followed by merging similar channels to maintain the original dimensionality for hardware efficiency. This disassembly and assembly process allows QLLM to autonomously determine the optimal number of sub-channels, significantly mitigating the performance drop typically observed in 4-bit quantization. Complementing this, [140] argues that the challenge stems not just from the existence of outliers but from the varying ranges across different channels. RPTQ addresses this by rearranging channels and quantizing them in clusters, effectively mitigating the impact of inter-channel range differences. By fusing the reorder operation into the LayerNorm and linear layers, RPTQ achieves substantial memory reductions, enabling 3-bit activation quantization for the first time in large-scale models.

Another promising direction involves altering the geometric structure of the representation space to eliminate outliers entirely through rotation. [40] presents a groundbreaking approach that rotates the LLM’s hidden states, feed-forward activations, and KV cache in a manner that removes outliers without changing the model’s output. This rotational invariance ensures that the resulting quantized model can perform all matrix multiplications in 4 bits without needing to retain any specific channels in higher precision, achieving negligible perplexity loss compared to full-precision baselines. This contrasts with methods that merely suppress outliers, as QuaRot fundamentally transforms the representation to be outlier-free, facilitating end-to-end low-bit inference. Furthermore, the interaction between outlier dimensions and model functionality has led to nuanced strategies that weigh the importance of specific dimensions. Research indicates that outlier dimensions often encode crucial task-specific knowledge, and their removal can be detrimental [141]. Consequently, methods like [142] adopt a mixed-precision scheme that prioritizes storing sensitive weights associated with outliers in high precision while aggressively quantizing the remaining dense weights. This sensitivity-aware approach acknowledges that not all outliers are noise; some are signal. Additionally, [143] proposes per-input-channel (per-IC) quantization, grouping weights based on the input dimension to isolate outliers within specific groups, thereby preventing them from distorting the quantization scale of the entire output channel.

Finally, hardware-aware innovations are emerging to handle outliers dynamically during inference, complementing the algorithmic transformations described above. [144] proposes a lightweight hardware technique that opportunistically increases bitwidth for activation outliers by overwriting nearby zeros, utilizing range and precision overwrite modes to handle the vast majority of outliers with minimal area overhead. This provides a systematic fallback for extreme values that software clipping might discard. Collectively, these strategies—ranging from mathematical transformations like those in [145] which uses learnable equivalent transformations, to structural reorganizations—demonstrate that mitigating activation outliers is a multifaceted challenge requiring a combination of statistical analysis, geometric manipulation, and hardware co-design. While these distribution smoothing techniques significantly enhance PTQ performance, they often reach a ceiling in ultra-low-bit regimes where information loss becomes irreversible without adapting the model’s internal representations. This limitation necessitates the transition to Advanced Quantization-Aware Training and Parameter-Efficient Fine-Tuning, as detailed in the following section, to further push the boundaries of model compression.

4.3 Advanced Quantization-Aware Training and Parameter-Efficient Fine-Tuning

4.3 Advanced Quantization-Aware Training and Parameter-Efficient Fine-Tuning

While the previous section detailed post-training strategies to mitigate activation outliers through mathematical transformations and hardware-aware clipping, these methods often reach a performance ceiling in ultra-low-bit regimes (below 4-bit) where information loss becomes irreversible without retraining. To bridge the gap between the distribution smoothing techniques discussed earlier and the extreme compression targets of vector quantization explored in the following section, research has increasingly turned to Advanced Quantization-Aware Training (QAT) integrated with Parameter-Efficient Fine-Tuning (PEFT). This hybrid paradigm addresses the instability of standard straight-through estimators and the prohibitive memory costs of full-model retraining, enabling models to adapt their internal representations to aggressive quantization constraints while preserving critical semantic information.

A primary challenge in low-bit QAT, particularly when transitioning from the outlier-suppressed distributions of Section 4.2 to uniform low-precision formats, is the distortion of information within attention mechanisms. Standard approaches that quantize weights while retaining full-precision activations fail to unlock the full potential of inference acceleration on edge devices. To address this, novel frameworks now guide the training process using entropy and distribution metrics to align quantized representations with their full-precision counterparts. For instance, the [146] framework identifies that performance degradation stems largely from divergences in quantized query and key distributions. By introducing entropy and distribution-guided objectives, this method mitigates such distortions and employs a token importance-aware adaptive mechanism to dynamically assign bit widths, achieving substantial on-device speedups. This represents a significant evolution from the static, layer-wise uniform schemes, moving towards context-aware policies that complement the dynamic outlier handling strategies previously discussed.

Parallel to these adaptive training objectives, the integration of Low-Rank Adaptation (LoRA) with QAT has revolutionized the efficiency of fine-tuning large language models, directly addressing the memory bottlenecks that hinder the deployment of the compressed models described in Section 4.2. Traditional fine-tuning requires storing full-precision optimizer states, which is infeasible on commodity hardware. The [147] approach overcomes this by backpropagating gradients through a frozen, 4-bit quantized pretrained model into low-rank adapters. By leveraging innovations such as

4-bit NormalFloat (NF4) data types and double quantization, QLoRA drastically reduces memory footprints without sacrificing performance. Building on this foundation, recent works have combined LoRA with distillation to further compress generative models. The [148] study demonstrates that integrating LoRA with model distillation reduces inference time and mitigates memory overhead, achieving a 50% reduction in consumption while preserving quality. Similarly, for LLMs, the [149] method proposes IR-QLoRA, which utilizes statistics-based information calibration and elastic connection strategies to ensure quantized parameters retain original fidelity, significantly improving accuracy in the 2-to-4-bit regimes that serve as a precursor to the ultra-low-bit strategies discussed next.

Despite these architectural advances, stability during the training of quantized networks remains a critical hurdle, often exacerbated by the non-differentiable nature of quantization operators. Many QAT methods rely on the Straight-Through Estimator (STE), which can introduce biased gradients and sub-optimal convergence, particularly when the weight distributions have been altered by the rotation or shifting techniques mentioned in Section 4.2. To counteract this, researchers have developed advanced regularization and correction techniques. The [150] paper provides empirical insights into gradient propagation vulnerabilities in low-bit KD-QAT, proposing the “ov-freeze” technique to stabilize training and achieve near floating-point precision with 4-bit weights. Addressing oscillations in efficient architectures, the [151] study introduces oscillation dampening and iterative weight freezing algorithms to prevent accuracy degradation caused by erroneously estimated batch-normalization statistics. Furthermore, the [152] work proposes an Exponentially Moving Average (EMA) update and a single-epoch correction method to rectify errors induced by oscillating weights, ensuring robust convergence even under aggressive quantization constraints.

Regularization plays a pivotal role in shaping the weight landscape to be inherently more amenable to quantization, effectively acting as a proactive counterpart to the reactive outlier mitigation strategies of the previous section. Techniques that explicitly constrain the distribution of weights or activations have shown remarkable success in preventing the re-emergence of outliers during training. The [153] study emphasizes the need to regularize both layer inputs and outputs, preventing the migration of quantization difficulty from activations to weights and enabling competitive W4A4 models. Similarly, the [154] introduces auxiliary losses during pre-training to restrict the range of weights, effectively molding the distribution into a tight shape that facilitates lower-bit quantization. In the realm of binary and extreme quantization, the [155] method employs orthonormal transformations and progressive layer-wise quantization to disentangle inter-weight dependencies, thereby alleviating performance degeneration in compact backbones.

Finally, the pursuit of data efficiency has led to the development of data-free and self-supervised QAT frameworks, which are essential for scenarios where labeled data is scarce or privacy-sensitive. The [156] framework achieves QAT-level performance with PTQ-like efficiency by distilling denoising capabilities into a quantized counterpart without requiring training data. Complementing this, the [157] approach formulates QAT as a co-optimization problem that minimizes KL-divergence between full-precision and low-bit models alongside discretization errors, all without supervision from labeled data. These advancements collectively signify a maturing field where QAT is no longer a standalone procedure but a synergistic component of a broader optimization pipeline. By integrating PEFT, distillation, and robust regularization, these methods prepare models for the extreme compression regimes discussed in the subsequent section, enabling the deployment of high-performance models on the most constrained edge devices while maintaining the structural integrity required for vector quantization and minifloat representations.

4.4 Ultra-Low-Bit Strategies: Vector Quantization, Codebooks, and Minifloats

4.4 Ultra-Low-Bit Strategies: Vector Quantization, Codebooks, and Minifloats

Building upon the Advanced Quantization-Aware Training (QAT) and Parameter-Efficient Fine-Tuning (PEFT) methodologies discussed in Section 4.3, which stabilize models for aggressive compression through adaptive training and regularization, research has increasingly pivoted toward extreme compression regimes targeting 2 to 4 bits per parameter. While the hybrid paradigms of the previous section address the instability of straight-through estimators and memory bottlenecks, traditional scalar quantization methods often reach a performance ceiling in these ultra-low-bit domains due to their inability to capture complex weight distributions and persistent outlier patterns. Consequently, advanced strategies leveraging vector quantization, sophisticated codebook designs, and alternative number formats such as minifloats have emerged as critical solutions to bridge the gap between the training-aware optimizations of Section 4.3 and the dynamic, memory-bound constraints of KV cache management explored in the subsequent section.

Vector quantization (VQ) represents a paradigm shift from quantizing individual scalars to encoding blocks of weights as single indices pointing to a shared codebook. This approach is particularly effective in extreme compression scenarios because it exploits the correlation within weight vectors, allowing for a more efficient representation of the underlying data distribution than the scalar-focused methods limited by the outliers addressed in Section 4.2. A prime example of this advancement is the application of Additive Quantization, a classic method from the multi-codebook quantization family, adapted specifically for LLMs. The method known as AQLM demonstrates that by utilizing additive quantization, it is possible to achieve state-of-the-art accuracy at 2 to 3 bits per parameter, significantly outperforming recent techniques on models ranging from 7B to 70B parameters [158]. Similarly, the introduction of lattice-based codebooks has shown remarkable promise. The QuIP# method enhances post-training quantization by employing vector quantization techniques that leverage the ball-shaped sub-Gaussian distribution of incoherent weights. By utilizing hardware-efficient codebooks based on the highly symmetric E_8 lattice, which achieves optimal unit ball packing in eight dimensions, QuIP# attains superior results in regimes of 4 bits or less [159]. These lattice codebooks are often combined with randomized Hadamard transforms to improve incoherence, ensuring that the weight distributions align better with the geometric properties of the codebooks, thereby minimizing quantization error and complementing the distribution-smoothing techniques discussed earlier.

Parallel to developments in vector quantization, the debate between integer and floating-point representations has gained traction, leading to the exploration of minifloat formats. While low-bit integer formats (e.g., INT4, INT2) have been the conventional standard, they often struggle with the dynamic range required to represent the heavy-tailed distributions typical of LLM weights and activations, a challenge that static outlier suppression in Section 4.2 only partially resolves. Minifloats, or reduced-precision floating-point formats, offer a compelling alternative by sacrificing mantissa precision to gain exponent range, thus better handling outliers without complex outlier removal mechanisms. Research into design-space exploration for minifloats indicates that formats with as few as 3 to 8 bits can approach full-precision accuracy while reducing memory footprints and energy costs [160]. Furthermore, comparative analyses suggest that the optimal format is not uniform across the model; different layers exhibit distinct tensor distributions that favor either integer or floating-point representations. This insight has led to the proposal of Mixture of Formats Quantization (MoFQ), which dynamically selects the optimal format on a layer-wise basis. MoFQ has demonstrated state-of-the-art performance in 4-bit weight-only quantization, surpassing integer-

only methods like GPTQ without requiring complex hyperparameter tuning [161].

To further optimize for hardware efficiency, researchers have investigated adaptive floating-point formats that dynamically adjust their dynamic range, addressing the rigidity of fixed formats used in previous sections. *AdaptivFloat*, for instance, introduces a floating-point inspired representation that optimally clips its available dynamic range at a layer granularity. This method has been shown to produce higher inference accuracies than block floating-point or IEEE-like floats at very low precisions (≤ 8 -bit), and in some cases, it even surpasses FP32 baseline performance in tasks like machine translation [162]. Additionally, the concept of transprecision computing advocates for the use of multiple floating-point types to enable fine-grained control over approximation, allowing up to 90% of operations to be safely scaled down to 8-bit or 16-bit formats, thereby reducing energy consumption significantly [163].

Another avenue for ultra-low-bit optimization involves power-of-two quantization methods, which replace multiplication operations with efficient bit-shift operations. This approach is particularly attractive for hardware implementation on FPGAs and specialized accelerators where logical shifts are virtually free compared to arithmetic multiplications, offering a distinct advantage over the arithmetic-heavy approaches of standard QAT. While pure power-of-two quantization can be restrictive, hybrid approaches that decouple parameters into integer and floating-point parts, or those that utilize specific coding theories, offer a balance between hardware friendliness and accuracy. For instance, *decoupleQ* transforms the quantization problem into a mathematical optimization task by decoupling parameters, achieving near full-precision accuracy even at 2-bit uniform quantization, which is highly beneficial for hardware deployment [164]. Moreover, the theoretical underpinnings of lattice transform coding suggest that employing lattice quantization in the latent space can recover optimal vector quantization performance, approaching asymptotically achievable rate-distortion functions [165].

In summary, the landscape of ultra-low-bit quantization is evolving beyond simple scalar rounding, extending the capabilities of the training-aware and regularization techniques from Section 4.3 into the realm of extreme compression. The integration of vector quantization with lattice codebooks, the strategic adoption of minifloat and adaptive floating-point formats, and the exploration of power-of-two schemes constitute a robust toolkit for extreme model compression. These strategies collectively address the limitations of dynamic range and distribution mismatch, enabling the deployment of powerful LLMs on devices with severe memory and compute constraints. However, while these methods effectively compress static model weights, the inference of long-context sequences introduces a distinct bottleneck in the dynamic Key-Value (KV) cache, necessitating the specialized quantization strategies for attention mechanisms discussed in the following section.

4.5 Specialized Quantization for Key-Value Caches and Attention Mechanisms

4.5 Specialized Quantization for Key-Value Caches and Attention Mechanisms

While the ultra-low-bit strategies discussed in the preceding section primarily target static model weights to reduce storage footprints, the deployment of Large Language Models (LLMs) for long-context inference introduces a distinct bottleneck: the Key-Value (KV) cache. Unlike immutable weights, the KV cache grows linearly with sequence length and batch size, often surpassing the model’s parameter count during generation. This dynamic expansion shifts the inference constraint from compute-bound to memory-bound, where latency is dominated by the bandwidth required to fetch cache entries rather than arithmetic operations. Consequently, specialized quantization strategies tailored explicitly for KV caches have emerged as a critical research frontier. These

approaches must address unique distributional characteristics inherent to attention mechanisms, such as channel-specific outliers and the asymmetric sensitivity of keys versus values, which differ significantly from the weight distributions addressed by vector quantization or minifloats.

A fundamental insight driving recent advancements is the recognition that keys and values exhibit divergent sensitivities to quantization errors, necessitating separate, non-uniform quantization strategies. Standard uniform quantization often fails to preserve attention fidelity because extreme values, or outliers, disproportionately distort the scale factors used for the entire tensor. To mitigate this, researchers have developed quality-adaptive schemes that allocate bit-widths based on the statistical properties of the cache entries. For instance, the [8] framework theoretically demonstrates that key caches and value caches possess distinct sensitivity profiles. By formulating separate quantization strategies for these components and integrating dedicated outlier handling mechanisms, [8] achieves compression ratios of up to 10x with negligible impact on model performance. Similarly, the [7] study provides a comprehensive analysis of element distributions, revealing that key caches benefit significantly from per-channel quantization, whereas value caches are better suited for per-token quantization. This asymmetric approach allows [7] to realize tuning-free 2-bit quantization, reducing peak memory usage by 2.6x and enabling up to 4x larger batch sizes without compromising generation quality.

Beyond adaptive bit allocation, hybrid compression frameworks that combine low-bit quantization with error correction mechanisms have shown remarkable efficacy in achieving near-lossless compression. These methods acknowledge that while the majority of KV cache entries can be represented with ultra-low precision, a small subset of critical information requires higher fidelity to prevent error accumulation during autoregressive generation. The [10] framework exemplifies this synergistic approach. [10] applies ultra-low precision quantization to the bulk of the cache entries while employing a low-rank matrix to approximate the resulting quantization error and a sparse matrix to correct individual outliers. This tripartite strategy allows [10] to achieve near-lossless 4-bit compression, delivering throughput improvements of up to 2.38x. Complementing this, the [37] paper argues against the complete eviction of tokens, a common practice in sparsification methods. Instead, it proposes a mixed-precision approach where evicted KV pairs are retained in low precision rather than discarded entirely, while pivotal tokens are maintained in high precision. This method effectively recovers the safety breaches and hallucinations often induced by aggressive eviction policies, offering a robust trade-off between compression ratio and generation reliability.

Furthermore, the integration of quantization with structural modifications to the attention mechanism itself has opened new avenues for optimization. Techniques that leverage block-based formats and channel rearrangement have proven effective in handling the outliers that typically plague low-bit inference. Research presented in [166] focuses on Block Floating Point (BFP) formats, proposing a novel channel rearrangement technique at compile-time to isolate outliers. This allows the use of low-precision BFP formats without the accuracy degradation usually caused by extreme values affecting the shared scale factor of a block, yielding a 2x savings in memory footprint. Additionally, the [167] study identifies a specific type of outlier associated with initial “pivot” tokens that attract disproportionate attention scores. [167] proposes generating the KV cache for these pivot tokens losslessly from the full-precision model while quantizing the rest, a simple yet effective strategy that achieves lossless weight-only INT4 quantization across various downstream tasks.

Joint optimization frameworks that simultaneously address weight and KV cache quantization represent the next evolution in this domain, moving away from treating them as isolated problems. The [61] framework introduces a two-dimensional quantization strategy specifically designed to handle the complex distribution of KV caches alongside model weights. By incorporating past-only

quantization and cross-block reconstruction regularization, [61] achieves memory savings comparable to weight-activation quantization while approaching the performance levels of weight-only quantization. Moreover, the concept of dynamic memory compression, as explored in [64], suggests that models can be retrofitted to learn different compression rates across heads and layers, effectively combining quantization principles with architectural adaptability. These holistic approaches underscore the necessity of viewing KV cache quantization not merely as a post-processing step but as an integral component of the overall model compression and serving architecture. While these methods optimize precision for specific components, the following section explores how to systematically allocate these varying precisions across the entire network through mixed-precision allocation and hardware-aware search to maximize end-to-end efficiency.

4.6 Mixed-Precision Allocation and Hardware-Aware Search

4.6 Mixed-Precision Allocation and Hardware-Aware Search

Building upon the specialized quantization strategies for KV caches and attention mechanisms discussed in the preceding section, the optimization landscape expands further when considering the entire model architecture. While the previous section highlighted how distinct components like keys and values require tailored precision handling, a holistic approach to efficient serving necessitates extending these principles across all layers, channels, and tokens. Uniform quantization, though implementation-friendly, often fails to capture the heterogeneous sensitivity of different network components to quantization noise, leading to suboptimal trade-offs between accuracy and efficiency. Mixed-Precision Quantization (MPQ) addresses this limitation by allocating distinct bit-widths to specific parts of the network, thereby maximizing hardware utilization while preserving critical information. However, the search space for optimal bit-width configurations grows exponentially with network depth, rendering exhaustive search infeasible. Consequently, recent research has pivoted toward sophisticated algorithms for dynamic bit-width assignment, ranging from differentiable search mechanisms and hardware-aware heuristic optimizations to one-shot learning paradigms.

A primary challenge in MPQ lies in efficiently navigating the discrete and combinatorial nature of the bit-width search space. Traditional approaches relying on Reinforcement Learning (RL) or iterative Neural Architecture Search (NAS) are often computationally prohibitive, requiring hundreds of GPU-hours to converge. To mitigate this cost, differentiable search methods have emerged as a dominant strategy. These techniques relax discrete bit-width choices into continuous probability distributions, enabling gradient-based optimization alongside weight training. For instance, the Stochastic Differentiable Quantization (SDQ) method employs Differentiable Bitwidth Parameters (DBPs) to act as probability factors between adjacent bit-width choices, facilitating a globally optimized search space with smoother gradient approximation [168]. Similarly, FracBits introduces a novel learning-based algorithm where the bit-width of each layer exists in a fractional status between two consecutive integers during optimization, gradually converging to an integer configuration that meets specific computational constraints [169]. These methods significantly reduce search overhead compared to RL-based alternatives while maintaining high accuracy.

Beyond differentiable relaxation, alternative approaches leverage explicit sensitivity metrics to guide bit allocation without extensive retraining. The concept of using learnable parameters, such as scale factors within the quantizer, as importance indicators has proven effective. By treating these scales as proxies for layer sensitivity, researchers can formulate the MPQ search as a one-time Integer Linear Programming (ILP) problem, drastically reducing search time from hours to seconds [170].

Furthermore, methods like LRP-QViT utilize explainability techniques, specifically Layer-wise Relevance Propagation, to assign mixed-precision allocations based on the contribution of each layer to the final classification task, demonstrating particular efficacy in Vision Transformers where attention mechanisms exhibit varying sensitivity [171]. Other works, such as BitPruning, introduce regularizers that directly penalize large bit-length representations during training, effectively learning thrifty representations that minimize inference bit-length while maintaining accuracy targets [172].

A critical evolution in this domain is the integration of hardware-specific constraints directly into the search process, shifting the focus from theoretical operation counts to actual latency and energy measurements. Hardware-aware search acknowledges that theoretical reductions in binary operations do not always translate to wall-clock speedups due to memory bandwidth bottlenecks and instruction set limitations. The “Free Bits” framework exemplifies this by proposing a hybrid search methodology that combines a hardware-agnostic differentiable search with a subsequent hardware-aware heuristic optimization. This approach tailors mixed-precision configurations to specific microcontroller platforms, achieving significant latency reductions even on systems lacking native sub-byte arithmetic support [173]. Similarly, the Fast-Bit Allocation for Mixed-Precision Quantization (FBM) method optimizes bit-width allocation by simulating or measuring performance on the target device, ensuring that the resulting model maximizes hardware efficiency rather than just minimizing bit-count [174]. At the extreme edge, specialized processor designs like the Mixed Precision Inference Core (MPIC) utilize status-based SIMD instructions to dynamically handle varying precisions without opcode bloat, highlighting the necessity of co-design between allocation algorithms and architectural support [175].

To further alleviate the computational burden of the search-retrain paradigm, one-shot weight-coupling learning has been proposed. This paradigm optimizes all potential bit-width configurations simultaneously within a set of shared weights, effectively coupling the training process. However, this introduces the challenge of bit-width interference, where optimizing for one precision negatively impacts others. To counteract this, advanced schedulers dynamically freeze turbulent bit-widths during training, while information distortion mitigation techniques align the behavior of poorly performing bit-widths with well-performing ones [176]. Additionally, adaptive strategies allow for runtime flexibility, where the network can adjust its precision based on input difficulty or resource availability. The Arbitrary Bit-width Network (ABN) constructs a super-network capable of changing bit-widths at runtime via a Markov Decision Process, enabling data-dependent dynamic inference that saves substantial computation for easier samples [177]. Similarly, Data Quality-aware Mixed-precision Quantization (DQMQ) employs hybrid reinforcement learning to adapt bit-widths dynamically to varying input data qualities, enhancing robustness in real-world scenarios where data consistency cannot be guaranteed [178].

Finally, the granularity of allocation has expanded from layer-wise to channel-wise and even token-wise distributions to capture finer-grained redundancies, mirroring the fine-grained optimizations seen in KV cache strategies. Channel-wise mixed-precision assignment allows higher precision to be reserved for the most informative feature channels, yielding superior Pareto-optimal models in terms of accuracy versus energy consumption on constrained edge nodes [179]. In the context of dynamic inference, methods like Dynamic Precision Reduction (DPRed) group weights and activations to encode them with group-specific precisions, dynamically selected by hardware for activations to minimize off-chip traffic and execution time [180]. These advancements collectively illustrate a shift towards highly adaptive, hardware-conscious, and fine-grained mixed-precision strategies. While these methods systematically allocate varying precisions to maximize end-to-end

efficiency, the following section explores the critical implications of such compression on model robustness, reliability, and the comprehensive benchmarks required to evaluate trustworthiness in real-world deployments.

4.7 Robustness, Reliability, and Comprehensive Evaluation Benchmarks

4.7 Robustness, Reliability, and Comprehensive Evaluation Benchmarks

While the preceding section detailed how mixed-precision allocation and hardware-aware search optimize the efficiency of Large Language Models (LLMs), the deployment of these quantized systems in real-world scenarios necessitates a rigorous examination of their robustness, reliability, and trustworthiness that extends far beyond standard accuracy or latency metrics. Although quantization significantly reduces memory footprints and accelerates inference, the introduction of quantization noise can subtly alter model behavior, potentially exacerbating vulnerabilities to adversarial attacks, distribution shifts, and hallucinations. Consequently, evaluating quantized LLMs requires a multidimensional framework that assesses worst-case performance, noise propagation in generative tasks, and adherence to standardized benchmarks for safety and fairness, ensuring that efficiency gains do not come at the cost of model integrity.

A critical area of concern is the impact of quantization on adversarial robustness, where the relationship between bit-width reduction and model resilience is often complex and counter-intuitive. Some empirical studies suggest that quantization can act as an implicit regularizer, enhancing robustness against specific adversarial perturbations. For instance, research on transformer-based text classifiers indicates that quantization can significantly improve adversarial accuracy compared to full-precision counterparts, sometimes outperforming explicit adversarial training methods without incurring additional computational overhead during training [181]. Similarly, evaluations in computer vision domains have revealed that lower-bit quantized models often exhibit higher resilience to adversarial attacks, although they may become more susceptible to natural corruptions and systematic noises such as impulse noise or nearest-neighbor interpolation [182] [183]. However, this robustness is not universal; it heavily depends on the quantization pipeline. Inconsistencies in existing literature often stem from whether robust optimization is integrated into the Post-Training Quantization (PTQ) or Quantization-Aware Training (QAT) stages [184]. Furthermore, malicious actors could potentially exploit quantization artifacts to inject backdoors or trigger specific failure modes that only manifest in the low-precision regime, highlighting the need for security-aware quantization frameworks [185].

Beyond adversarial settings, the reliability of quantized models under distribution shifts and their retention of parametric knowledge are paramount. Quantization noise can propagate through deep layers, distorting the probability distributions used for token generation and leading to increased hallucination rates or a loss of factual consistency. Studies focusing on emergent abilities, such as in-context learning and chain-of-thought reasoning, reveal a sharp threshold: while 4-bit quantization generally preserves these capabilities, reducing precision to 2-bit can cause severe degradation, fundamentally altering the model’s reasoning capacity [186]. This degradation is not always captured by perplexity metrics alone, necessitating evaluation frameworks that specifically measure knowledge retention and alignment. Comprehensive analyses across diverse model families indicate that while larger quantized models can outperform smaller full-precision ones, the trade-off between compression and parametric knowledge loss must be carefully managed to prevent the erosion of long-tail factual recall [187] [188].

To address these challenges, the community is moving towards standardized, holistic evaluation

benchmarks that integrate trustworthiness dimensions with efficiency metrics. Recent initiatives have proposed structured frameworks evaluating quantized LLMs across three critical dimensions: knowledge and capacity, alignment, and efficiency, utilizing diverse benchmarks to uncover failure modes invisible to standard language modeling tasks [188]. Notably, the interplay between compression and trustworthiness is intricate; for example, moderate quantization (e.g., 4-bit) has been found to retain or even improve certain ethical and fairness metrics compared to pruning, whereas extreme compression (e.g., 3-bit) tends to significantly degrade trustworthiness across multiple dimensions [189]. Furthermore, evaluating the compressibility of quantized models involves understanding the trade-offs between further data compression for I/O efficiency and the potential performance penalties on memory-constrained edge devices [190].

Effective evaluation also requires robust calibration strategies to ensure that the confidence scores of quantized models remain reliable indicators of truthfulness. Techniques such as multi-domain temperature scaling and local intrinsic dimension analysis of internal activations are being explored to quantify uncertainty and detect hallucinations in compressed models [191] [192]. Additionally, benchmarking efforts like MQBench emphasize the gap between algorithmic performance and hardware deployability, arguing that reproducibility and hardware-aware constraints must be central to any comprehensive evaluation suite [193]. Ultimately, ensuring the reliability of quantized LLMs demands a shift from viewing quantization solely as a compression tool to treating it as a modification of the model’s loss landscape that requires specialized validation protocols for safety, fairness, and robustness in dynamic environments.

5 Synergistic Compression: Integrating Pruning, Distillation, and Architectural Adaptations

5.1 Synergistic Foundations and Multiplicative Compression Effects

5.1 Synergistic Foundations and Multiplicative Compression Effects

While the deployment of Large Language Models (LLMs) on resource-constrained hardware often necessitates isolated interventions, a paradigm shift is underway toward viewing model compression techniques as components of a unified, synergistic optimization framework. Historically, quantization, knowledge distillation, and pruning have been studied independently; however, recent empirical evidence suggests that their strategic combination yields super-multiplicative reductions in model size and computational cost, frequently surpassing the theoretical limits of any single method applied in isolation. This subsection establishes the theoretical underpinnings of these synergistic interactions, analyzing how the structural redundancies exposed by one technique can be exploited by another to achieve extreme compression ratios while maintaining model fidelity. This foundation is critical for understanding how subsequent serving optimizations, such as KV cache management and speculative decoding, can operate more effectively on models that have been holistically compressed.

The fundamental premise of synergistic compression lies in the observation that different strategies target distinct forms of redundancy within neural networks. Quantization addresses numerical redundancy by reducing the precision of weights and activations, effectively compressing the information density per parameter. Pruning targets structural redundancy by eliminating unnecessary connections or entire channels, thereby reducing the total parameter count. Knowledge distillation leverages functional redundancy, transferring the rich representations learned by a large “teacher” model into a compact “student” architecture. When applied sequentially or jointly, these methods

do not merely accumulate individual benefits; rather, they exhibit complementary behaviors where the application of one technique facilitates the efficacy of the others. For instance, research has demonstrated that while quantization and distillation consistently provide greater benefits than pruning when used alone, their combination rarely yields diminishing returns. Instead, practitioners have observed complementary and super-multiplicative reductions to model size, suggesting that the interaction between these methods creates a more compressible loss landscape [194].

A critical theoretical insight driving this synergy is the concept of the “Chain of Compression,” which posits that applying compression techniques in a specific combinational sequence allows the model to capitalize on the unique features of each approach. By first employing techniques that smooth the loss landscape or identify critical substructures, subsequent compression steps can operate with greater aggressiveness. For example, structured pruning can be viewed as a form of channel-wise quantization where the bit-width is zero, allowing for a unified view where pruning and quantization are solved as a single joint optimization problem. This unification simplifies the compression pipeline and avoids the divergence issues often seen in step-wise applications, leading to automated solutions that simultaneously determine layer sparsity and bit-width [195]. Furthermore, treating pruning as a special case of quantization enables the use of single-path models to encode all candidate configurations, transforming the complex search for optimal compression strategies into a more tractable subset selection problem [196].

The interaction between these techniques also extends to the optimization dynamics during training. Traditional approaches often suffer from accuracy degradation when aggressively combining low-bit quantization with high-sparsity pruning due to limited numerical representation and the loss of critical gradient pathways. However, advanced frameworks have shown that integrating these methods through differentiable joint optimization can significantly reduce Bit-Operations (BOPs) while maintaining top-tier accuracy. By incorporating variational information bottleneck-based structured pruning and mixed-bit precision quantization into a single differentiable loss function, it is possible to automatically trade off between pruning and quantization for optimal hardware efficiency [197]. Moreover, the order of operations matters; studies on convolutional neural networks have revealed that a “Chain of Compression” can reduce computation costs by 100 to 1000 times with negligible accuracy loss, validating the hypothesis that these methods exhibit complementary behavior when sequenced correctly [198].

Mitigating accuracy loss in such highly compressed regimes requires sophisticated strategies that go beyond simple fine-tuning. One effective approach involves optimizing the model to reside in flatter minima of the loss landscape, which has been shown to correlate with higher compressibility. Sharpness-Aware Minimization (SAM) has been successfully combined with iterative magnitude pruning and post-training quantization to find parameterizations that are inherently more amenable to compression, yielding superior “winning tickets” that transfer effectively across tasks [199]. Additionally, the integration of knowledge distillation into quantization-aware training (QAT) serves as a powerful regularizer. By forcing the quantized student model to mimic the attention maps and output distributions of a full-precision teacher, the student can recover information lost due to low-bit discretization. This is particularly crucial for sub-2-bit quantization, where standard MSE loss on attention scores is insufficient, and specialized distillation losses are required to restore self-attention mechanisms [200].

The synergistic effects are not limited to general performance metrics but also extend to the preservation of parametric knowledge and fairness. While compression can sometimes amplify biases or erase rare knowledge, joint optimization frameworks that consider fairness constraints during the compression process can mitigate these disparate impacts. For instance, characterizing bias in com-

pressed models reveals that pruning disproportionately affects underrepresented features, but this can be addressed by auditing “Compression Identified Exemplars” and adjusting the compression strategy accordingly [201]. Furthermore, recent advances in extreme compression for pre-trained transformers have shown that skipping pre-training distillation and focusing on aggressive layer reduction combined with extreme quantization can reduce model sizes by 50 times while achieving state-of-the-art results, provided that the training strategy accounts for the interplay between hyperparameters [202].

In conclusion, the theoretical foundation of synergistic compression rests on the principle that quantization, pruning, and distillation are not mutually exclusive but are deeply interconnected levers for model optimization. By recognizing that pruning can be formulated as zero-bit quantization and that distillation can stabilize the training of ultra-low-bit models, researchers can design holistic compression pipelines. These pipelines leverage the multiplicative effects of combined techniques to push the boundaries of what is possible in efficient LLM serving, enabling the deployment of massive models on edge devices without sacrificing the reliability and robustness required for real-world applications. The move towards joint, differentiable, and hardware-aware optimization represents the next frontier in realizing the full potential of these synergistic interactions, setting the stage for advanced adaptation strategies like Mixture-of-Experts architectures that further refine parameter efficiency [203].

5.2 Parameter-Efficient Fine-Tuning Enhanced by Mixture-of-Experts Architectures

5.2 Parameter-Efficient Fine-Tuning Enhanced by Mixture-of-Experts Architectures

Building upon the theoretical foundations of synergistic compression established in Section 5.1, where joint optimization of quantization, pruning, and distillation was shown to yield super-multiplicative efficiency gains, this subsection explores how these principles extend to the adaptation phase of Large Language Models (LLMs). While Section 5.1 focused on compressing the static backbone of models, the rapid escalation in LLM scale has rendered full parameter fine-tuning increasingly prohibitive due to excessive memory consumption and computational costs. Although Parameter-Efficient Fine-Tuning (PEFT) methods like Low-Rank Adaptation (LoRA) have emerged as a standard solution by freezing pre-trained weights and updating only low-rank decomposition matrices, these approaches often face limitations in model capacity and flexibility when adapting to complex, multi-domain downstream tasks. To overcome the inherent constraints of static, global adapter parameters, recent research has pivoted towards integrating Mixture-of-Experts (MoE) mechanisms into PEFT frameworks. This synergistic approach dynamically allocates lightweight experts across layers, enabling superior adaptation capabilities with minimal trainable parameter overhead while mitigating the random routing phenomena often observed in traditional sparse architectures. This evolution from static compression to dynamic adaptation sets a critical precedent for the extreme compression strategies required for trillion-parameter MoE models discussed in Section 5.3.

A primary advancement in this domain is the recognition that not all transformer layers contribute equally to task adaptation, necessitating a non-uniform distribution of expert resources—a concept that mirrors the layer-wise sensitivity analysis used in the “Chain of Compression” described previously. The work titled [204] introduces MoLA (MoE-LoRA with Layer-wise Expert Allocation), a novel architecture that assigns a varying number of LoRA experts to different model layers. This

framework challenges the conventional homogeneous design where every layer receives an identical number of adapters. Empirical evidence suggests that allocating a greater density of experts to higher layers significantly enhances model effectiveness, allowing the system to capture more abstract and task-specific features where they are most critical. By employing this strategic, layer-wise allocation, MoLA achieves performance parity or superiority compared to baselines using the same total number of parameters, demonstrating that intelligent resource distribution is paramount for efficient adaptation.

Building upon the concept of treating adapters as experts, other frameworks have focused on improving the quality of expert specialization through advanced training objectives, thereby addressing the functional redundancy issues highlighted in synergistic compression theory. The paper [205] proposes MoELoRA, which explicitly addresses the issue of redundant or poorly differentiated experts. By incorporating contrastive learning into the fine-tuning process, this method encourages distinct experts to learn diverse feature representations, thereby preventing the collapse of expert utility that can occur with standard routing mechanisms. This approach has shown remarkable efficacy, particularly in math reasoning tasks, where it outperforms standard LoRA by a significant margin, proving that enhancing the semantic diversity of experts is as crucial as increasing their quantity.

Beyond fixed architectures, dynamic and soft routing mechanisms have been explored to further optimize the trade-off between computational cost and adaptation performance, offering a differentiable alternative to the hard sparsity constraints often found in pruning. The research titled [206] extends these concepts to multimodal domains, proposing Soft-MoA (Soft Mixture of Adapters). Unlike hard routing strategies that select a single expert per token, Soft-MoA leverages a soft assignment mechanism where input tokens are processed by weighted combinations of all available adapter experts. This differentiable approach ensures that all experts contribute to the computation, effectively resolving expert imbalance issues and allowing for smoother gradient flow during training. Similarly, the study [207] advocates for fully differentiable sparse Transformers, demonstrating that implicit soft assignments can maintain the benefits of conditional computation while avoiding the training instability and token dropping associated with discrete gating functions.

The versatility of MoE-enhanced PEFT is further demonstrated in multi-task and cross-domain scenarios, where the modular nature of experts facilitates the transfer of knowledge akin to the distillation processes discussed in Section 5.1. The framework described in [208], known as PEMT, utilizes a gated unit to measure correlations between source and target tasks, dynamically weighting adapters trained on different source domains. This allows the model to exploit transferable knowledge efficiently without retraining from scratch. Additionally, the work [209] integrates task information directly into the MoE routing mechanism at various granularities, enabling shared dynamic adapters to generalize effectively to unseen tasks. These approaches highlight how MoE architectures can serve as a modular bridge for knowledge transfer, adapting pre-trained models to diverse requirements with high fidelity.

Innovation in MoE-PEFT also extends to the structural composition of the experts themselves, leveraging matrix factorization techniques to reduce parameter counts similar to low-rank adaptations. The paper [210] proposes sharing information among experts using Matrix Product Operators (MPO), reconstructing parameter matrices to share a central tensor containing core information while maintaining specificity through auxiliary tensors. This method drastically reduces the total parameter count required for superior performance compared to standard Switch Transformers. Furthermore, addressing the challenge of scaling experts under data-limited conditions, [211] introduces variance-based constraints and cluster-level dropout to prevent overfitting and ensure diverse

knowledge acquisition among experts.

Crucially, the intersection of MoE-PEFT and the compression techniques outlined in Section 5.1 offers a pathway to even greater efficiency, foreshadowing the extreme compression strategies needed for trillion-parameter models in Section 5.3. The study [212] observes that many experts contribute negligibly to specific downstream tasks and proposes a progressive pruning strategy to reduce sparse MoE models into single-expert dense models without significant performance degradation. Complementary to this, [213] demonstrates that the residuals of PEFT modules can be aggressively compressed via sparsification and ternary quantization, facilitating efficient communication and storage of expert models. Collectively, these advancements illustrate that integrating MoE mechanisms into PEFT is not merely an additive improvement but a fundamental architectural shift. By combining dynamic routing with structural compression, these methods enable scalable and highly efficient adaptation of foundation models, bridging the gap between the holistic compression frameworks of Section 5.1 and the memory-constrained serving of massive sparse models explored in the subsequent section.

5.3 Extreme Compression Strategies for Trillion-Parameter MoE Models

5.3 Extreme Compression Strategies for Trillion-Parameter MoE Models

While Section 5.2 explored how Mixture-of-Experts (MoE) architectures can be leveraged to enhance Parameter-Efficient Fine-Tuning (PEFT) through dynamic adapter allocation, the deployment of the foundational trillion-parameter MoE models themselves presents a distinct and formidable challenge: the memory wall. The advent of MoE has fundamentally altered the scaling landscape, offering pathways to capacities exceeding one trillion parameters while maintaining manageable inference costs via sparse activation. However, this architectural advantage introduces a severe storage bottleneck; the sheer volume of parameters required to house vast arrays of experts often exceeds the memory capacity of even high-end enterprise GPUs. For instance, models like SwitchTransformer-c2048 possess parameter counts reaching 1.6 trillion, necessitating approximately 3.2 TB of accelerator memory for execution in standard precision. This demand renders direct deployment impractical for most real-world applications, creating an urgent need for extreme compression strategies that reduce the memory footprint by orders of magnitude without compromising generative capabilities or routing efficiency. To address this, recent research has converged on specialized pipelines that synergistically combine sub-1-bit quantization, dynamic expert pruning, and intelligent memory swapping mechanisms, bridging the gap between the massive scale discussed in PEFT contexts and the practical constraints addressed in unified optimization frameworks.

A primary avenue for tackling the memory wall in trillion-parameter MoE models is the development of ultra-low-bit quantization algorithms specifically co-designed with the sparse nature of expert weights. Traditional quantization methods often struggle to maintain accuracy when pushing below 4 bits, particularly given the diverse weight distributions found across thousands of distinct experts. Breakthroughs in this domain have demonstrated that it is possible to compress these massive models to less than 1 bit per parameter while preserving fidelity. A paradigmatic example of this approach is [214], which introduces a scalable framework capable of compressing the 1.6 trillion parameter SwitchTransformer-c2048 model to under 160 GB. This represents a compression ratio of roughly 20x, effectively reducing the precision to 0.8 bits per parameter. The success of [214] lies not merely in the algorithmic compression but in its holistic co-design; it utilizes a custom data format optimized for bespoke GPU decoding kernels. This integration ensures that the decompression

overhead is minimal, allowing for end-to-end compressed inference with runtime overheads of less than 5% compared to ideal uncompressed execution. Consequently, [214] enables the execution of trillion-parameter models on affordable commodity hardware configurations, such as a single server equipped with four NVIDIA A6000 or eight NVIDIA 3090 GPUs, a feat previously unattainable.

Complementary to static quantization, dynamic memory management strategies have emerged as critical components for serving large MoE models under tight memory constraints. These approaches recognize that in sparse MoE architectures, only a small fraction of experts are activated for any given token, suggesting that keeping the entire model resident in high-bandwidth memory is inefficient. [215] proposes a framework that leverages this sparsity by maintaining a dynamic set of “Virtual Experts” in the main memory while storing the full library of experts in slower, higher-capacity storage. The core innovation of [215] is its profiling-guided planner and importance-aware scheduler, which predictively identify, update, and swap experts based on activation patterns. This allows the system to fully utilize available memory budgets and bandwidth, achieving latency profiles similar to pruned compact models while retaining the superior accuracy of the full-scale MoE. By seamlessly managing the mapping between virtual and actual experts, [215] effectively decouples the model’s logical size from the physical memory limits of the deployment device.

Further refining the intersection of quantization and sparsity, specialized techniques have been developed to apply aggressive compression selectively to expert layers, which have been observed to be more robust to quantization noise than dense feedforward networks. [216] demonstrates that applying ultra-low-bit quantization, down to 2 bits, exclusively to expert weights can reduce the total model size by nearly 80% relative to half-precision baselines. The findings in [216] highlight a complementary effect where the inherent regularization of the MoE routing mechanism mitigates the accuracy loss typically associated with such aggressive quantization. When combined with optimized GPU runtimes, this strategy yields significant speedups, proving that extreme compression need not come at the cost of performance.

In the context of edge and mobile deployment, where memory is even more constrained, strategies involving offloading and hierarchical storage become paramount. [217] introduces an inference engine that partitions the model across the storage hierarchy, keeping non-expert weights in device memory while fetching expert weights from external storage only upon activation. This approach, supported by expert-wise bitwidth adaptation and predictive preloading, minimizes I/O swapping overheads. Similarly, [218] addresses the limitations of GPU memory by orchestrating computation between the CPU and GPU, utilizing the CPU’s processing power to minimize data movement and enable the execution of models like Mixtral-8x7B on consumer-grade hardware. These systems illustrate that extreme compression for trillion-parameter models is not a singular algorithmic fix but a synergistic orchestration of sub-1-bit quantization [214], dynamic swapping [215], and heterogeneous resource management. Together, these advancements are dismantling the barriers to deploying massive sparse models, transitioning them from theoretical curiosities confined to supercomputing clusters to practical tools accessible on commodity infrastructure. This focus on extreme, model-level compression sets the stage for the subsequent discussion in Section 5.4, which explores unified optimization frameworks that jointly refine pruning, quantization, and distillation to achieve further efficiency gains across diverse architectural components.

5.4 Unified Optimization Frameworks for Joint Pruning, Quantization, and Distillation

5.4 Unified Optimization Frameworks for Joint Pruning, Quantization, and Distillation

While Section 5.3 detailed extreme compression strategies tailored specifically for the unique memory walls of trillion-parameter Mixture-of-Experts (MoE) models, the broader landscape of efficient LLM serving requires a more generalized approach applicable to diverse architectural scales. Traditional deployment pipelines often treat pruning, quantization, and knowledge distillation as sequential, disjointed steps. This isolated application frequently leads to suboptimal solutions due to the compounding of errors and a failure to exploit the synergistic interactions between different compression modalities. To overcome these limitations, recent research has shifted toward unified optimization frameworks that formulate compression as a joint problem. These holistic methodologies navigate the trade-offs between accuracy, latency, and memory footprint within a single, coherent search space, bridging the gap between the model-specific extreme compression discussed previously and the cross-model modularity explored in the subsequent section.

A primary paradigm in unified compression is the adoption of differentiable search approaches that treat pruning masks, quantization bit-widths, and architectural hyperparameters as continuous, learnable variables. By relaxing discrete decision spaces into continuous domains, these methods leverage gradient-based optimization to jointly discover optimal configurations. For instance, differentiable architecture search (DAS) frameworks have been extended to handle mixed-precision and sparsity constraints simultaneously. The work on [219] exemplifies this by introducing bit-level sparsity quantization, which treats each bit of a quantized weight as an independent trainable variable. By employing a differentiable bit-sparsity regularizer, [219] enables the dynamic exploration of the full mixed-precision space within a single gradient-based optimization process, effectively inducing all-zero bits to realize dynamic precision reduction without the need for cumbersome neural architecture search loops. Similarly, [220] proposes a differentiable channel sparsity search (DCSS) that automatically identifies optimal sparsity combinations across layers. By utilizing weight sharing within filters to eliminate shape mismatching, [220] balances computational costs and accuracy metrics more effectively than manual pruning ratios, demonstrating the power of joint search in convolutional networks.

Beyond standard differentiable relaxation, recent advancements incorporate sharpness-aware minimization (SAM) and robust training objectives to enhance the intrinsic compressibility of models. The hypothesis driving these approaches is that flatter loss landscapes correspond to solutions that are more robust to the perturbations introduced by pruning and quantization. While specific SAM applications to joint compression are an evolving field, the principles are evident in methods that regularize the search space to prevent performance collapse. For example, [221] highlights the critical role of explicit sparse regularization in stabilizing differentiable search. By addressing information bypass leakage, this approach ensures that the learned architecture parameters remain robust, a principle that is directly transferable to joint pruning and quantization frameworks where stability against discrete approximation errors is paramount. Furthermore, [222] introduces a structured pruning framework based on an enhanced sparsification paradigm that maintains the magnitude of dropped weights and enhances the expressivity of retained weights through self-distillation. This method illustrates how integrating distillation-like mechanisms directly into the sparsification training loop can mitigate the capacity damage often seen in aggressive joint compression scenarios.

Another significant direction in unified optimization is the development of one-shot weight-coupling

learning paradigms, which aim to eliminate the costly retraining phases traditionally associated with mixed-precision quantization and pruning. Conventional methods often suffer from bit-width interference, where optimizing for one precision level negatively impacts others within a shared super-network. To address this, [176] devises a one-shot training-searching paradigm where all potential bit-width configurations are coupled and optimized simultaneously within a set of shared weights. Crucially, this work identifies and mitigates bit-width interference through a dynamic bit-width scheduler and an information distortion mitigation technique, allowing for the extraction of high-performance subnets without hundreds of GPU-hours of retraining. This concept of coupling extends to architectural search as well, where [223] proposes distilling knowledge from “prioritized paths” within a hyper-network. By allowing subnetworks to teach each other collaboratively, [223] boosts the convergence of individual models, effectively unifying the benefits of weight sharing and knowledge distillation in a single stage.

The integration of lossy and lossless compression mechanisms also represents a frontier in unified frameworks, moving beyond treating encoding and parameter reduction as separate concerns. Rather than treating them independently, [224] proposes a unified parametric weight transformation that allows different lossy compression methods to be performed jointly in a post-training manner. This approach utilizes a differentiable counter to guide the optimization toward a state that is highly amenable to subsequent lossless compression, achieving stable high compression ratios with negligible accuracy loss. Additionally, the combination of structured sparsity and low-precision quantization is explored in hardware-aware contexts. [225] demonstrates that applying binarization and structured sparsity as joint constraints during training can yield DNN models with drastically reduced memory footprints and energy consumption, tailored specifically for area-constrained embedded platforms.

Finally, the challenge of searching for optimal combinations in vast discrete spaces is addressed by combinatorial optimization techniques integrated into learning frameworks. [226] characterizes N:M sparsity learning as a combinatorial problem, solving it via a divide-and-conquer strategy where combination subsets are assigned learnable scores. This allows for the efficient optimization of fine-grained sparsity during normal training, avoiding the need for expensive pre-training phases. Similarly, [227] explores direct combinatorial optimization for binary weights, framing risk minimization as a non-monotone submodular maximization problem. These methods underscore a broader trend: the most effective unified frameworks are those that can seamlessly blend continuous relaxation for search efficiency with rigorous combinatorial or regularization techniques to ensure the final discrete model retains high fidelity. By synthesizing pruning, quantization, and distillation into a single optimization trajectory, these unified frameworks provide a robust foundation for model efficiency. This sets the stage for Section 5.5, which expands the scope from optimizing a single model instance to treating these compressed components as modular, transferable assets capable of being recombined and distilled across heterogeneous architectures.

5.5 Cross-Model Modularity and Transferable Expert Systems

5.5 Cross-Model Modularity and Transferable Expert Systems

While Section 5.4 established unified frameworks for jointly optimizing pruning, quantization, and distillation within a single model instance, the scaling of Large Language Models (LLMs) has necessitated a broader perspective: treating model capabilities as modular, transferable assets across different architectures. The rapid proliferation of LLMs has catalyzed a paradigm shift towards modular designs, where specialized knowledge is encapsulated within Parameter-Efficient Fine-

Tuning (PEFT) modules or sparse Mixture-of-Experts (MoE) layers. Although these approaches drastically reduce training costs and enable scalable expansion, they introduce a critical deployment bottleneck: the fragmentation of knowledge across incompatible model instances. This subsection explores the theoretical foundations and practical methodologies for cross-model modularity, addressing the dual challenges of transferring compressed modules between heterogeneous models and distilling sparse, high-capacity expert systems into compact, dense student models. This focus on modularity serves as a crucial bridge, connecting the internal compression strategies of Section 5.4 with the hardware-specific execution constraints discussed in the subsequent Section 5.6.

A fundamental barrier to cross-model modularity has been the prevailing assumption that modular components, such as adapters or LoRA weights, are inextricably tied to the specific pre-trained language model (PLM) hosting them. Traditionally, PEFT modularity was confined to single-model setups, limiting the reusability of expert knowledge across the ecosystem. Recent advancements, however, have begun to dismantle this siloed approach. Research indicates that it is feasible to transfer pre-trained, task-specific PEFT modules between models within the same family, and even across incompatible architectures, without altering inference complexity. By leveraging knowledge distillation as a bridging mechanism, the functional behavior of a module trained on a robust, large PLM can be mapped onto a smaller target model. This “transferable modularity” facilitates the accumulation of specialized skills—such as named entity recognition or natural language inference—into a shared repository of modules that can be dynamically composed, thereby enhancing zero-shot and few-shot generalization capabilities across diverse linguistic domains [228].

The challenge of transferring expertise becomes even more pronounced when dealing with Sparsely activated Mixture-of-Experts (SMoE) models. While SMoE architectures offer a pathway to scale parameter counts to the trillions without a proportional increase in computational cost, their deployment is hampered by high memory usage due to expert duplication and significant communication overheads in distributed settings. Furthermore, the sparse nature of these models makes them inherently difficult to compress using standard quantization or pruning techniques designed for dense networks, a limitation that echoes the joint optimization challenges discussed previously. To address these issues, novel frameworks have been proposed to consolidate expert information. For instance, strategies that leverage routing statistics to guide the merging of multiple experts into fewer, more knowledgeable units have shown promise. By aligning neuron permutations and grouping dominant experts based on activation frequencies, it is possible to merge redundant experts while preserving critical knowledge. This merged representation often exhibits lower dimensionality, naturally facilitating subsequent compression steps such as low-rank decomposition or structural sparsification, ultimately yielding models with drastically reduced memory footprints and FLOPs while maintaining performance [229].

Beyond mere merging, the distillation of sparse MoE models into dense student networks represents a critical frontier in synergistic compression. The “curse of capacity gap” often plagues this process, where a small dense student struggles to mimic the complex, conditional computation of a large sparse teacher. Innovative solutions involve introducing minimal sparse structures into the student during the distillation phase to bridge this gap. Approaches like the “Mixture of Minimal Experts” (MiniMoE) impose extra parameters on the student in the form of tiny, sparse experts that introduce negligible inference overhead but significantly enhance the student’s expressiveness. This allows the dense student to capture the nuanced, task-specific knowledge held by the sparse teacher, effectively lifting the capacity gap and achieving state-of-the-art performance at small FLOPs [230]. Similarly, frameworks that treat the distillation process as a “knowledge integration” task have demonstrated success in gathering key knowledge from different pre-trained experts via methods like Top-K

gathering or Singular Value Decomposition, followed by refining a dense student model to offset noise. This “One Student Knows All Experts Know” paradigm enables a single dense model to preserve a substantial percentage of the benefits derived from a massive MoE system while achieving significant inference speedups due to its hardware-friendly architecture [231].

Furthermore, the compression of PEFT residuals themselves has emerged as a vital technique for facilitating cross-model transfer. In scenarios where multiple expert models must be served or transmitted over high-latency networks, the sheer size of these modules can be prohibitive. Methods employing sparsification and ternary quantization on PEFT residuals—often referred to as task vectors—can achieve compression ratios ranging from 8x to 50x without requiring additional retraining. Remarkably, these compressed experts not only facilitate efficient communication but also maintain, and in some cases enhance, their compositional generalization capabilities when merged. This suggests that compression can act as a regularizer, filtering out noise in the expert parameters and resulting in more robust transferable modules [213].

The synergy between modularity, sparsity, and distillation is further exemplified by approaches that progressively drop non-professional experts for specific downstream tasks. By observing that most experts contribute minimally to fine-tuning on a target task, researchers have developed methods to prune these redundant experts, effectively reducing a massive MoE model into a single-expert dense model that retains nearly all the performance benefits of the original sparse architecture. This task-specific expert pruning eliminates the need for complex expert parallelism and communication protocols, making large-scale models deployable on cloud or mobile environments [212]. Additionally, the evolution of training frameworks, such as those utilizing dense-to-sparse gating mechanisms, allows for the gradual specialization of experts, which can subsequently be distilled more effectively into compact forms. These evolutionary training strategies ensure that the knowledge captured by experts is robust and well-defined, simplifying the subsequent transfer and compression processes [232].

In conclusion, the future of efficient LLM serving lies in the ability to treat model components as transferable, composable, and compressible assets. By decoupling expertise from specific model scales and architectures, cross-model modularity enables a more sustainable and flexible AI ecosystem. The integration of advanced distillation techniques, expert merging strategies, and residual compression allows for the creation of compact, dense students that inherit the diverse capabilities of massive, sparse teachers. However, realizing the full potential of these modular and compressed models requires more than just algorithmic innovation; it demands a rigorous alignment with the underlying physical infrastructure. As we transition to the next section, we examine how hardware-aware co-design is essential to overcome the irregular data patterns and memory bottlenecks inherent in these sparse and quantized modular systems, ensuring that theoretical efficiency gains translate into tangible wall-clock latency reductions.

5.6 Hardware-Aware Co-Design for Sparse and Quantized Inference

5.6 Hardware-Aware Co-Design for Sparse and Quantized Inference

While Section 5.5 established the theoretical and algorithmic foundations for treating model components as transferable, composable, and compressible assets—ranging from merged MoE experts to quantized PEFT residuals—the realization of these synergistic benefits is fundamentally gated by the capabilities of the underlying hardware infrastructure. The transition from abstract compression metrics to tangible wall-clock latency reductions requires a rigorous co-design approach that aligns software abstractions with physical hardware constraints. The irregular data patterns,

dynamic sparsity, and non-standard bit-widths generated by the modular and compressed systems discussed previously stand in stark contrast to the dense, regular compute paradigms of modern accelerators. Consequently, bridging this gap necessitates the development of custom decoding kernels, specialized memory hierarchies, and intelligent orchestration mechanisms tailored specifically to the unique footprint of sparse and quantized inference.

A primary challenge in deploying the quantized MoE models and compressed modules introduced in the preceding section lies in the efficient execution of low-bit operations on hardware traditionally optimized for floating-point arithmetic. Standard GPU tensor cores often lack native support for the ultra-low-bit integer or custom fixed-point formats required by extreme compression schemes, such as sub-1-bit representations. Naive implementations of these formats incur significant overhead due to bit-manipulation instructions and data unpacking, which can easily negate the bandwidth savings achieved by algorithmic compression. To address this, recent research emphasizes the creation of bespoke decoding kernels co-designed with specific compression formats. For instance, the [214] framework exemplifies this co-design philosophy by introducing custom GPU decoding kernels tailored for its sub-1-bit parameter format. This approach allows for the efficient end-to-end compressed inference of trillion-parameter models, achieving minor runtime overheads relative to uncompressed execution by tightly integrating the decompression logic with the matrix multiplication units. Similarly, the [216] study highlights that combining low-bit quantization with MoE architectures requires optimized GPU runtime implementations to realize speed-ups, demonstrating that the algorithmic robustness of merged experts must be paired with hardware-aware execution strategies to mitigate memory bandwidth bottlenecks.

Beyond compute kernels, the irregular activation patterns inherent in sparse MoE models and the fragmented nature of transferable modules pose severe challenges for memory management and data movement. In standard dense models, memory access is predictable and contiguous, allowing for high bandwidth utilization. However, the dynamic expert routing and modular composition strategies discussed in Section 5.5 lead to scattered memory accesses, causing cache thrashing and underutilization of memory bandwidth. Effective co-design must therefore incorporate heterogeneous memory management strategies that intelligently offload inactive experts while keeping hot paths resident in high-bandwidth memory. The [233] paper addresses this by proposing novel offloading strategies that leverage the innate properties of MoE LLMs to accelerate parameter movement between CPU and GPU memory, enabling the execution of large models on consumer hardware. Furthermore, [234] introduces “Expert Buffering,” a caching mechanism that retains only active experts in GPU memory while buffering the rest in CPU memory, significantly reducing static memory allocation. This is complemented by dynamic gating and load balancing techniques that optimize the communication-computation overlap, ensuring that the latency of fetching sparse experts or modular components does not become the critical path bottleneck.

The complexity of mapping these compressed and sparse workloads to heterogeneous hardware is further compounded by the need for adaptive data layouts and advanced compiler support. Traditional compilers often fail to optimize for the irregular tensor shapes and varying bit-widths characteristic of the synergistic compression techniques explored previously. Advanced compiler infrastructures are required to automate the generation of efficient low-precision kernels and manage data movement across heterogeneous memory pools. The [235] proposes a composable programming model that decouples hardware customizations from algorithm specifications, enabling holistic optimizations across function boundaries for spatial accelerators. This is crucial for handling the hierarchical nature of compressed models where different layers or modular components may require different precision or sparsity handling. Additionally, [236] demonstrates the utility of an

intermediate representation that describes both computational kernels and hardware capabilities, facilitating automated instruction mapping and scheduling that can adapt to novel instruction sets and memory architectures required by quantized sparse models.

Moreover, the interplay between compression and hardware coherence in heterogeneous systems cannot be overlooked. As systems increasingly integrate diverse accelerators (GPUs, FPGAs, NPUs) to handle different aspects of sparse inference and modular execution, the coherence protocol governing data sharing becomes a performance determinant. [237] illustrates how reinforcement learning can dynamically select optimal coherence modes for accelerators at runtime, adapting to workload characteristics to minimize off-chip memory accesses. This adaptive orchestration is vital for MoE systems and modular setups where the active set of experts or modules changes per token, requiring rapid and efficient data synchronization across the memory hierarchy. Similarly, [238] argues for fine-grained specialization of coherence requests to improve cache reuse and reduce network traffic, a strategy directly applicable to the fragmented memory access patterns of quantized sparse inference.

Finally, the physical limitations of memory bandwidth in edge and mobile deployments necessitate co-design approaches that integrate compression directly into the memory subsystem. Techniques such as near-data processing and in-memory acceleration offer promising avenues to alleviate the memory wall, particularly for the compact, dense student models and compressed residuals derived in Section 5.5. [239] presents a software-hardware co-design for SRAM Computing-in-Memory (CIM) accelerators, where model compression algorithms like pruning and quantization are specifically tailored to the constraints of CIM macros. This ensures that the compressed model structure aligns with the physical limitations of word lines and bit lines, maximizing energy efficiency. In summary, the future of efficient LLM serving relies not merely on better compression algorithms, but on a deeply integrated co-design ecosystem where custom kernels, adaptive memory managers, and intelligent compilers work in unison to exploit the synergistic potential of sparsity, quantization, and modularity, transforming theoretical efficiency gains into practical deployment realities.

6 System-Level Architectures, Hardware Co-Design, and Distributed Serving

6.1 Disaggregated Prefill-Decode Architectures and Resource Allocation

6.1 Disaggregated Prefill-Decode Architectures and Resource Allocation

While homogeneous clusters have traditionally served as the backbone for LLM inference, a more fundamental bottleneck arises from the intrinsic disparity between the two primary phases of generation: prefill and decoding. The prefill phase, which processes the entire input prompt to generate the first output token, is inherently compute-bound and highly parallelizable, allowing it to saturate GPU compute units effectively. Conversely, the decoding phase, which generates subsequent tokens autoregressively, is strictly memory-bound due to the repetitive fetching of model weights and the growing Key-Value (KV) cache, resulting in low compute utilization. Traditional serving systems colocate these phases on the same hardware resources, batching requests to maximize throughput. However, this colocation introduces severe performance interference, as the resource-intensive prefill operations of new requests can stall the latency-sensitive decoding iterations of ongoing requests. This interleaving creates a complex trade-off where optimizing for throughput often violates strict latency Service Level Objectives (SLOs), particularly for Time to First Token (TTFT) and Time Per Output Token (TPOT).

To resolve these inefficiencies, a paradigm shift towards disaggregated architectures has emerged, physically or logically separating prefill and decoding computations onto distinct hardware units. This disaggregation eliminates direct contention between phases, allowing each to be optimized independently according to its specific resource demands. A pioneering work in this domain, [DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving], explicitly argues that colocating prefill and decoding not only leads to strong interferences but also couples resource allocation and parallelism plans in a suboptimal manner. By assigning prefill and decoding tasks to different GPUs, [DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving] demonstrates that it is possible to eliminate these interference patterns entirely. This separation enables the system to co-optimize resource allocation and parallelism strategies tailored specifically for the unique requirements of each phase, rather than relying on a one-size-fits-all approach that compromises both.

The benefits of this architectural shift are quantifiable and significant. In scenarios with stringent latency requirements, traditional systems are forced to either prioritize one latency metric over the other or over-provision compute resources, leading to poor utilization. In contrast, disaggregated systems can dynamically allocate resources based on real-time demand. [DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving] reports that their approach can serve 4.48 times more requests or meet 10.2 times tighter SLOs compared to state-of-the-art colocated systems, while maintaining latency constraints for over 90% of requests. This improvement is achieved by minimizing the communication overhead caused by disaggregation through intelligent placement strategies that consider the serving cluster’s bandwidth topology.

Furthermore, the concept of disaggregation extends beyond simple phase separation to include advanced scheduling mechanisms that manage the flow of requests between specialized pools. Related research, such as [12], complements this architectural shift by introducing scheduling techniques like chunked-prefills. While [12] focuses on creating stall-free schedules within a shared environment to interleave prefill and decode without pausing ongoing decodes, the underlying motivation aligns with the goals of disaggregation: preventing long-latency prefill operations from blocking the critical path of token generation. When combined with physical disaggregation, such scheduling insights allow for even finer-grained control over resource utilization, ensuring that the high-throughput capabilities of the prefill cluster do not starve the decoding cluster of necessary bandwidth or compute cycles.

The challenges of resource allocation in disaggregated systems also mirror broader issues in shared computing environments, requiring dynamic adaptation to fluctuating workloads. For instance, [240] highlights the necessity of partitioning cores and dynamically adjusting allocation policies to handle bursty loads and guarantee tail latency SLOs, a principle directly applicable to managing the decode cluster in an LLM serving architecture. Similarly, [17] discusses the limitations of hardware-based resource isolation and proposes software-layer scheduling to time-share interference-free partitions, reinforcing the idea that software-defined resource orchestration is critical when hardware boundaries are drawn between prefill and decode units.

Moreover, the communication overhead introduced by physically separating compute units necessitates robust data movement strategies. The efficiency of transferring the KV cache or intermediate states from the prefill cluster to the decode cluster becomes a bottleneck if not managed correctly. Research into disaggregated systems, such as [DaeMon Architectural Support for Efficient Data Movement in Disaggregated Systems], provides valuable insights into alleviating data movement overheads through mechanisms like bandwidth partitioning, link compression, and adaptive granularity selection. Although [DaeMon Architectural Support for Efficient Data Movement in

Disaggregated Systems] focuses on general memory disaggregation, its findings on transparent data migration and handling runtime variability in network latencies are crucial for designing the interconnects between prefill and decode pools in LLM clusters. Additionally, emerging hardware technologies like co-packaged optics, as discussed in [241], offer the potential to further reduce the latency and energy costs of these interconnects, making the disaggregated prefill-decode architecture even more viable for future large-scale deployments.

In conclusion, the transition to disaggregated prefill-decode architectures represents a critical evolution in LLM serving infrastructure. By decoupling the compute-bound prefill phase from the memory-bound decoding phase, systems can achieve unprecedented levels of efficiency and predictability. Works like [DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving] have established the theoretical and practical foundations for this approach, demonstrating that tailored resource allocation and the elimination of phase interference are key to meeting the rigorous demands of modern LLM applications. As the field progresses, the integration of advanced scheduling algorithms, efficient data movement protocols, and next-generation interconnect technologies will further refine these architectures. This logical separation of tasks sets the stage for the next layer of optimization: leveraging heterogeneous computing frameworks that unify diverse hardware accelerators—such as CPUs, GPUs, and FPGAs—to execute these specialized workloads with maximum synergy.

6.2 Heterogeneous Computing Frameworks for CPU-GPU-FPGA Collaboration

6.2 Heterogeneous Computing Frameworks for CPU-GPU-FPGA Collaboration

Building upon the logical separation of prefill and decode phases established in Section 6.1, the next frontier in optimizing LLM serving lies in leveraging heterogeneous computing frameworks that unify diverse hardware accelerators. While disaggregated architectures effectively eliminate phase interference by assigning tasks to specialized GPU pools, further efficiency gains can be realized by integrating Central Processing Units (CPUs), Graphics Processing Units (GPUs), and Field-Programmable Gate Arrays (FPGAs) into a cohesive inference pipeline. This heterogeneous approach addresses the limitations of homogeneous GPU clusters, where memory capacity constraints and I/O bottlenecks often hinder the processing of massive model weights or irregular workloads. In this paradigm, CPUs provide flexibility and expansive memory pools, GPUs deliver unparalleled parallel throughput for matrix operations, and FPGAs offer energy-efficient, customizable acceleration. Recent research has focused on developing frameworks that bridge these disparities through automatic workload offloading, asynchronous I/O overlapping, and dynamic scheduling algorithms, thereby creating a synergistic environment that complements the disaggregated prefill-decode model.

A primary challenge in realizing this synergy is the efficient distribution of tasks across devices with vastly different performance characteristics. Traditional offloading strategies often treat accelerators as static co-processors, leading to underutilization when workloads are irregular or when data transfer latencies dominate computation time. To address this, novel frameworks have emerged that decouple load balancing from work processing, allowing for fine-grained resource allocation. For instance, researchers have proposed GPU fine-grained load-balancing abstractions that support both static and dynamic schedules via a programmable interface, enabling the system to adapt to irregular parallel algorithms without tightly coupled, application-specific logic [242]. This decoupling

is critical for LLM serving, where the sequence length and batch size can vary significantly between requests, creating irregular computational patterns that static schedulers fail to optimize. Such flexibility is essential in a disaggregated setting, where the decode cluster must handle variable-length generation streams without stalling.

In the context of resource-constrained environments and edge deployments, the collaboration between CPUs and GPUs becomes even more vital, offering a cost-effective alternative to massive GPU clusters. The “HeteGen” framework exemplifies this by introducing a principled approach to heterogeneous parallel computing specifically designed for LLM inference on low-resource hardware. HeteGen mitigates the I/O bottlenecks that typically plague CPU-GPU offloading by employing asynchronous overlap techniques, where data transfer and computation occur concurrently. By strategically partitioning the model layers and overlapping the movement of weights with the execution of previous layers, HeteGen achieves substantial inference speedups, surpassing state-of-the-art homogeneous methods by over 300% in certain configurations [243]. This approach highlights the necessity of viewing the CPU not merely as a host but as an active participant in the inference pipeline, capable of handling specific sub-tasks or managing memory prefetching to keep the GPU saturated, a strategy that aligns well with the memory-bound nature of the decoding phase discussed previously.

Furthermore, the integration of FPGAs into the heterogeneous mix introduces additional complexity due to their reconfigurable nature and distinct memory hierarchies, yet offers unique opportunities for customizing data paths for specific model architectures. Effective frameworks must dynamically distribute tasks among CPUs, GPUs, and FPGAs to fully exploit all available computing devices while minimizing load imbalance. A software-defined framework has been developed to enable such parallel utilization, demonstrating that allowing CPU cores to assist FPGA execution can yield significant performance benefits even under strict energy constraints [115]. This collaborative execution model is particularly effective in embedded System-on-Chip (SoC) architectures, where unified abstractions allow the runtime to balance workloads across ARM processors, NEON SIMD engines, and FPGA accelerators through mechanisms like work-stealing. Such systems have demonstrated throughput improvements of over 7x compared to software-only solutions, validating the potential of tightly integrated heterogeneous pipelines [244].

Dynamic scheduling algorithms serve as the backbone of these heterogeneous frameworks, as they must continuously adapt to runtime conditions such as network latency, device thermal throttling, and fluctuating request rates. In distributed edge environments, where tasks may be partitioned across multiple devices with varying capabilities, the scheduling problem becomes NP-hard. Solutions like the “IP-SSA” (Independent Partitioning and Same Sub-task Aggregating) algorithm optimize offloading policies by grouping identical sub-tasks into batches, thereby maximizing GPU utilization while minimizing user energy consumption [245]. Moreover, when future task arrivals are unpredictable, learning-based agents utilizing Deep Deterministic Policy Gradients (DDPG) can be trained to make optimal grouping and scheduling decisions in real-time, outperforming static heuristic approaches by adapting to the stochastic nature of edge workloads [245]. These adaptive scheduling mechanisms are crucial for maintaining the high goodput targets set by disaggregated architectures when operating across diverse hardware.

The complexity of scheduling is further compounded in systems with dynamic performance asymmetry, where interference from co-located tasks causes fluctuating execution speeds. Advanced schedulers address this by learning the performance characteristics of the underlying platform and leveraging “moldability”—the ability of tasks to switch between single-threaded and multithreaded modes—to mitigate interference [246]. Additionally, simulation frameworks like HeSP have shown

that treating recursive task partitioning and scheduling as simultaneous degrees of freedom can yield significant performance gains on heterogeneous CPU-GPU and big.LITTLE platforms, suggesting that future serving systems must jointly optimize how tasks are split and where they are placed [247].

Finally, the emergence of serverless computing paradigms has introduced new opportunities for heterogeneous collaboration through elastic resource allocation, extending the concept of disaggregation to a global scale. Frameworks like “Singularity” demonstrate the viability of preemptive and elastic scheduling across a global fleet of diverse accelerators, including GPUs and FPGAs. By enabling jobs to be transparently migrated and resized across different hardware types without code modification, such systems achieve high utilization and reliability, effectively abstracting the underlying hardware heterogeneity from the developer [248]. Similarly, compiler-assisted scheduling frameworks that probe kernel resource requirements at runtime can place tasks on the most appropriate device based on current load and memory availability, improving throughput by up to 2.7x for neural network workloads [118]. These advancements collectively illustrate that the future of efficient LLM serving lies not in isolated accelerators, but in sophisticated, software-defined orchestration layers that harmonize the unique strengths of CPUs, GPUs, and FPGAs into a cohesive, adaptive inference engine. However, to fully capitalize on this heterogeneous hardware potential, especially when dealing with the memory constraints of large models, it is imperative to employ advanced quantization techniques supported by robust compiler infrastructures, a topic we explore in the following section.

6.3 Compiler Infrastructures and Automated Kernel Optimization for Low-Bit Operations

6.3 Compiler Infrastructures and Automated Kernel Optimization for Low-Bit Operations

While heterogeneous computing frameworks (discussed in Section 6.2) provide the architectural foundation for orchestrating diverse hardware resources, realizing the full potential of these systems—particularly for quantized workloads—requires a sophisticated compiler infrastructure. The deployment of Large Language Models (LLMs) across CPUs, GPUs, and FPGAs necessitates a robust software layer capable of bridging the gap between high-level model specifications and the intricate low-level constraints of each device. As quantization emerges as a primary strategy for reducing memory footprint and computational cost, the role of deep learning compilers has evolved from merely scheduling operations to actively generating and optimizing low-precision kernels. Traditional frameworks often rely on vendor-specific libraries optimized for standard floating-point arithmetic, leaving significant performance on the table for quantized workloads. To address this, modern compiler infrastructures employ end-to-end optimization strategies that automate the generation of efficient code for sub-8-bit precisions, leveraging techniques such as operator fusion, automated search spaces, and learning-based cost models to harmonize with the heterogeneous landscape.

A fundamental challenge in executing quantized models within these heterogeneous environments is the lack of native support for arbitrary low-precision operators in standard linear algebra libraries. While frameworks like TensorFlow and PyTorch support quantization simulation, they often fall back to inefficient emulation or dequantization paths during actual inference on non-specialized hardware, undermining the efficiency gains sought by heterogeneous offloading. To overcome this,

compilers like TVM have introduced augmented dialects, such as the Quantized Neural Network (QNN) dialect, which extends the internal representation with quantization context. This allows the compiler to generate efficient code for pre-quantized INT8 models across various targets, including CPUs and GPUs, achieving significant speedups over floating-point baselines by eliminating redundant dequantization steps [249]. Similarly, workflows have been developed to automatically generate high-performance low-precision operators for arbitrary bit-widths, utilizing memory tiling and vectorization to exploit specific CPU architectures, demonstrating speedups of up to 16x over optimized 16-bit baselines on mobile processors [33].

To further alleviate the memory bottlenecks inherent in quantized inference, operator fusion stands as a critical optimization technique within these compiler infrastructures. Fusion reduces the overhead of global memory access and kernel launch latency by combining multiple element-wise operations, such as quantization scaling, bias addition, and activation functions, into a single kernel. This is particularly vital in heterogeneous systems where data movement between CPU and GPU memory pools can dominate execution time. However, traditional fusion heuristics are often restrictive, failing to handle complex data dependencies or the heterogeneous parallelism found in modern transformer architectures. Advanced frameworks like FusionStitching address these limitations by formulating fusion plan optimization as an integer linear programming problem, enabling the stitching of memory-intensive element-wise and reduction operators into large GPU kernels. This approach has demonstrated average performance improvements of 1.4x over state-of-the-art methods by effectively leveraging on-chip resources like registers and scratchpads [250]. Furthermore, compilers like DNNFusion expand fusion opportunities by classifying operators based on mathematical properties, allowing for graph rewriting that identifies fusion patterns missed by pattern-based approaches, resulting in speedups of up to 9.3x on mobile devices [251].

The complexity of optimizing tensor programs for low-bit operations across diverse hardware has driven the adoption of learning-based cost models and automated tuning frameworks. Analytical cost models, which rely on static hardware characteristics, often struggle to predict performance accurately for complex, high-level intermediate representations (IR) like MLIR or for novel hardware accelerators such as FPGAs. Consequently, researchers have turned to machine learning to guide the search for optimal kernel configurations. For instance, MetaTune utilizes meta-learning to predict the performance of optimized codes with pre-trained graph neural networks, significantly reducing the overhead of auto-tuning while improving inference times by 8-13% compared to traditional transfer learning methods [252]. Similarly, ML-driven cost models have been developed to predict hardware characteristics such as register usage and instruction counts directly from high-level MLIR representations, treating the IR as text input to leverage advancements in natural language processing for more accurate hardware modeling [253].

Automated search strategies are essential for exploring the vast space of loop transformations and scheduling parameters required for efficient low-bit execution in dynamic heterogeneous environments. Frameworks like LoopTune employ deep reinforcement learning to optimize tensor traversal orders and loop nests, generating code that rivals hand-tuned libraries while reducing tuning time from hours to seconds [254]. Other approaches, such as SIP, apply stochastic instruction perturbation to discover optimal GPU native instruction schedules, automatically finding configurations that improve CUDA kernel throughput for LLM workloads [255]. These learning-based autotuners are particularly effective when combined with flexible intermediate representations that allow for joint graph-level and operator-level optimization, as seen in systems like ALT, which co-optimizes data layouts and loop structures to achieve superior end-to-end performance [256].

Furthermore, the integration of quantization-aware compilation extends beyond inference to train-

ing and fine-tuning scenarios, ensuring consistency across the model lifecycle. Compilers are increasingly supporting low-precision training kernels, such as those for FP8 arithmetic, which require specialized handling of gradient accumulation and optimizer states. The FP8-LM framework exemplifies this trend by providing an automatic mixed-precision framework that seamlessly integrates low-bit gradients and optimizer states, achieving substantial reductions in memory usage and training time [257]. Additionally, specialized runtimes like DeepliteRT automate the conversion of fake-quantized models into compact ultra-low-bit representations, deploying highly optimized convolution operators on ARM-based edge devices to achieve speedups of over 4x compared to existing methods [258].

In conclusion, the efficacy of low-bit LLM serving is inextricably linked to the sophistication of the underlying compiler infrastructure, which acts as the essential glue between algorithmic quantization and heterogeneous hardware execution. By automating the generation of low-precision kernels through advanced operator fusion, learnable cost modeling, and automated search strategies, modern compilers enable the practical deployment of quantized models across a diverse landscape of CPUs, GPUs, and FPGAs. These systems not only mitigate the “framework tax” associated with inefficient memory management but also unlock the full potential of hardware accelerators, ensuring that theoretical compression ratios translate into tangible reductions in latency and energy consumption. As we look toward future architectures, these compiler-driven optimizations lay the groundwork for even more radical shifts, such as the memory-centric computing paradigms discussed in the following section, where the boundaries between storage and computation continue to blur.

6.4 Memory-Centric Acceleration and Data Movement Minimization

6.4 Memory-Centric Acceleration and Data Movement Minimization

While the compiler infrastructures discussed in Section 6.3 provide the software mechanisms to generate efficient low-bit kernels and optimize operator fusion, the physical limitations of data movement between memory and compute units remain a fundamental bottleneck. As Large Language Models (LLMs) scale, the performance constraint has decisively shifted from compute-bound operations to memory-bound constraints, a phenomenon known as the “memory wall.” In autoregressive generation, the linear growth of the Key-Value (KV) cache and massive parameter counts necessitate frequent data shuffling between off-chip memory and processing units, consuming significant energy and latency. To address this, recent research has pivoted toward memory-centric acceleration strategies that fundamentally rearchitect the relationship between storage and computation, moving beyond mere software optimization to hardware-level paradigm shifts. The primary approach in this domain is Processing-in-Memory (PIM), which seeks to eliminate or drastically reduce data movement by integrating computational capabilities directly within or near memory arrays.

Processing-in-Memory architectures operate on the principle that moving computation to the data is far more efficient than moving data to the computation. This approach is particularly critical for data-intensive applications where off-chip data movement can consume more than 60% of the total system energy [259]. Modern PIM designs generally fall into two categories: processing-near-memory (PnM), which places logic layers adjacent to DRAM stacks, and processing-using-memory (PuM), which exploits the physical properties of memory cells to perform logic operations. For instance, digital PIM architectures leverage massive parallel bitwise logic operations within memory elements to execute fundamental arithmetic operations, demonstrating significant throughput and

energy improvements over traditional GPU baselines [260]. By utilizing the high internal bandwidth of 3D-stacked DRAM, PIM systems can effectively free up valuable memory channel bandwidth while reducing system energy consumption, making them ideal candidates for accelerating memory-bounded tasks such as LLM inference [86].

Specific adaptations of PIM for transformer-based models have yielded promising results, directly addressing the memory-bound nature of attention mechanisms. The PIM-GPT architecture, for example, achieves end-to-end acceleration of autoregressive transformers by executing multiply-accumulate (MAC) operations directly within DRAM chips, thereby eliminating the need to move matrix data off-chip for the majority of the inference process [261]. Similarly, digital PIM platforms have been optimized to accelerate bulk bit-wise operations, which are foundational to many quantized neural network computations discussed in the previous section, achieving throughput gains of up to 71x compared to CPU baselines [262]. However, the efficacy of PIM is highly dependent on the specific workload characteristics; while general-purpose PIM architectures like those from UPMEM offer viable alternatives for memory-bound machine learning training, they do not always scale linearly with node count, necessitating careful algorithm-hardware co-design [263].

Complementing PIM, another critical strategy for minimizing data movement is the integration of on-the-fly weight decompression engines. As model sizes exceed available memory capacity, compression techniques such as quantization and pruning become essential. However, the decompression overhead can introduce new computational bottlenecks if not handled efficiently, potentially negating the bandwidth savings achieved by the compilers in Section 6.3. Research indicates that naive decompression can undermine the benefits of compression, particularly in domain-specific architectures designed for sparse workloads [264]. To mitigate this, advanced hardware designs now incorporate dedicated decompression modules that generate weights on-chip in real-time. For example, the unzipFPGA framework introduces a weights generator module that enables on-the-fly generation of decompressed weights, effectively counteracting bandwidth limitations in memory-bound layers and achieving significant performance density gains over state-of-the-art accelerators [265]. This approach allows systems to store models in highly compressed formats while maintaining high compute utilization, effectively decoupling memory footprint from computational throughput.

Furthermore, algorithmic innovations in communication-avoiding matrix multiplication play a pivotal role in reducing data movement overheads, serving as a bridge between single-node memory optimizations and the distributed serving challenges discussed in the following section. Traditional linear algebra kernels often suffer from suboptimal data reuse and excessive communication between memory hierarchy levels or across networked nodes. Communication-optimal tiling strategies have been developed to derive lower bounds on data movement and construct algorithms that attain these bounds, even for nested loop programs with arbitrary bounds [266]. In the context of sparse matrix operations, which are prevalent in pruned LLMs, 2.5D algorithms combined with one-sided MPI communication have demonstrated substantial performance boosts by trading increased local memory usage for reduced global communication [267]. Additionally, flexible communication-avoiding models implemented on reconfigurable hardware, such as FPGAs, allow for the optimization of matrix multiplication to simultaneously target maximum performance and minimum off-chip data movement, scaling effectively with both compute and memory resources [268].

The synergy between these hardware and algorithmic approaches is essential for overcoming the memory wall. While PIM provides the architectural foundation for near-data processing, on-the-fly decompression ensures that compressed models can be served without latency penalties, and communication-avoiding algorithms minimize the residual data movement that remains inevitable in complex distributed serving scenarios. Future advancements in this space will likely focus on

refining the programming models for PIM to ease adoption, as the current gap between high-level abstractions and microarchitectural realities remains a significant barrier [269]. Moreover, addressing reliability concerns inherent in emerging memory technologies, such as variation in ReRAM cells, through algorithm-hardware co-design frameworks like HybridAC, will be crucial for ensuring the robustness of memory-centric accelerators in production environments [270]. Ultimately, the transition to memory-centric computing represents a paradigm shift necessary to sustain the scaling of LLM serving; however, as individual node optimizations reach their limits, the focus must expand to cluster-level coordination. This leads naturally to the discussion in the next section on distributed serving systems, where minimizing inter-node communication and managing heterogeneous resources become paramount for scaling to trillions of parameters.

6.5 Distributed Serving Systems and Scalable Inference Pipelines

6.5 Distributed Serving Systems and Scalable Inference Pipelines

While memory-centric acceleration and data movement minimization (discussed in Section 6.4) address the bottlenecks within individual nodes, scaling Large Language Models (LLMs) to trillions of parameters necessitates a paradigm shift toward sophisticated distributed serving architectures. As model sizes exceed the memory capacity of even the most advanced single accelerators, serving systems must leverage complex combinations of data, tensor, and pipeline parallelism. However, simply distributing a model introduces significant overheads driven by inter-node communication latency, resource heterogeneity, and the “straggler” problem, where slower nodes delay the entire inference pipeline. Addressing these challenges requires a holistic approach that integrates algorithmic optimizations with system-level scheduling, extending the efficiency gains achieved by local memory optimizations to the cluster level.

A primary bottleneck in distributed serving is the communication overhead inherent in synchronizing states across devices. In traditional data-parallel setups, the frequency of gradient or activation exchange can dominate execution time, particularly when network bandwidth is limited. Research has shown that without careful optimization, a significant portion of GPU hours in large-scale clusters is spent on communication with no overlapping computation, severely throttling throughput [271]. To mitigate this, modern serving frameworks employ advanced parallelization strategies that compose tensor, pipeline, and data parallelism to maximize hardware utilization. For instance, interleaved pipeline parallelism schedules have been proposed to improve throughput by over 10% while maintaining a comparable memory footprint to existing approaches, effectively scaling training and inference to thousands of GPUs [272]. Furthermore, the integration of communication compression techniques, such as those targeting pipeline parallel traffic and embedding synchronization, allows systems to reduce inter-node traffic volume without sacrificing model quality, thereby enabling scalability even on clusters with constrained bandwidth [273]. These techniques complement the on-chip compression strategies discussed previously by ensuring that the benefits of reduced data footprints are preserved across the network fabric.

Beyond communication volume, the efficiency of distributed serving is heavily influenced by the physical topology of the cluster and the allocation of tasks to hardware resources. Heterogeneity in compute capabilities, memory bandwidth, and network interconnects can lead to severe load imbalances. Static partitioning strategies often fail to account for these variations, resulting in suboptimal performance. Automated frameworks that search for optimal model parallel strategies while explicitly modeling hardware heterogeneity have demonstrated the ability to outperform expert-tuned configurations by significant margins, achieving up to 1.77x higher throughput on

heterogeneous clusters [274]. Additionally, the geometry of network partitions plays a critical role; studies on supercomputer architectures indicate that optimizing partition geometries to maximize internal bisection bandwidth can eliminate link contention and yield up to a twofold speedup for contention-bound workloads [275]. This underscores the necessity for serving systems to be topology-aware, dynamically mapping model segments to physical nodes to minimize cross-partition communication, a logical extension of the data locality principles central to memory-centric design.

The “straggler” problem remains a pervasive challenge in scalable inference pipelines, where variability in node performance or transient network delays can stall synchronous operations. Traditional synchronous barriers force faster nodes to wait for the slowest, drastically reducing overall cluster efficiency. To address this, recent advances propose adaptive mechanisms that predict and mitigate stragglers proactively. Techniques utilizing machine learning models, such as Encoder LSTM networks, can analyze historical resource consumption to predict straggling tasks and dynamically adapt scheduling, reducing Service Level Agreement (SLA) violations and execution time [276]. In the context of federated and distributed learning, asynchronous tiers and weighted aggregation heuristics have been successfully employed to minimize the straggler effect while maintaining convergence speed and accuracy [277]. These concepts are increasingly being adapted for inference serving, where speculative execution and dynamic cutoff times allow systems to discard or approximate contributions from lagging nodes to maintain high throughput [278]. Such adaptive behaviors foreshadow the more comprehensive runtime auto-tuning and dynamic adaptation strategies detailed in the subsequent section.

Disaggregated architectures represent another frontier in scalable serving, aiming to decouple conflicting resource requirements of different inference phases. By separating the prefill (prompt processing) and decoding (token generation) phases onto distinct hardware units, systems can eliminate interference and tailor resource allocation to the specific needs of each phase. This approach, exemplified by frameworks like DistServe, has been shown to serve significantly more requests within strict latency constraints compared to colocated strategies, effectively decoupling Time to First Token (TTFT) and Time Per Output Token (TPOT) optimizations [14]. Similarly, streaming KV-cache mechanisms enable fault-tolerant and efficient prompt-token disaggregation, reducing pipeline bubbles and optimizing GPU memory management in distributed environments [279]. This architectural separation not only enhances scalability but also creates distinct optimization domains that benefit from the specialized hardware-software co-design approaches discussed next.

Finally, the scalability of serving systems is also tested by the unique characteristics of specific workloads, such as deep learning recommendation models which involve massive embedding tables. Distributed inference strategies for these models must carefully manage static embedding table distributions and handle the sparsity of input requests to minimize latency overhead. Research indicates that with optimized embedding table mapping strategies, distributed inference can achieve near-linear scaling with marginal latency increases, proving viable for data-center scale deployments [280]. Moreover, the emergence of model parallelism for statistical multiplexing allows multiple models to share devices efficiently, exploiting bursty workload patterns to reduce serving latency even when individual models could fit on single devices [281].

In conclusion, building scalable distributed serving pipelines for LLMs requires a multi-faceted strategy that addresses communication bottlenecks, hardware heterogeneity, and straggler mitigation through advanced parallelism, topology-aware scheduling, and architectural disaggregation. While this section has focused on the structural and algorithmic foundations of distributed scaling, the dynamic and stochastic nature of real-world production environments demands even greater agility. The following section explores how hardware-software co-design and learning-based auto-tuning

can further refine these distributed systems, enabling them to adapt in real-time to fluctuating workloads and evolving hardware landscapes.

6.6 Hardware-Software Co-Design for Dynamic Adaptation and Auto-Tuning

6.6 Hardware-Software Co-Design for Dynamic Adaptation and Auto-Tuning

Building upon the structural foundations of distributed serving and architectural disaggregation discussed in Section 6.5, the next frontier in efficiency lies in bridging the gap between static system designs and the volatile realities of production environments. While distributed architectures address scalability through parallelism and topology-aware scheduling, they often rely on fixed configurations determined offline. The deployment of Large Language Models (LLMs) in heterogeneous and dynamic environments necessitates a paradigm shift from these static compilation strategies to holistic hardware-software co-design that leverages machine learning for runtime adaptation. Traditional optimization approaches, dependent on pre-deployment profiling, frequently fail to accommodate the stochastic nature of LLM workloads, fluctuating resource availability, and the intricate interdependencies between software algorithms and underlying hardware microarchitectures. To overcome these limitations, modern co-design methodologies employ adaptive control loops and learning-based auto-tuners that continuously monitor system states, reconfiguring execution parameters in real-time. This evolution transforms serving infrastructure into a self-aware entity capable of navigating vast configuration spaces to simultaneously optimize latency, throughput, and energy efficiency.

A foundational aspect of this co-design is the intelligent reduction of the adaptation space. The configuration space for modern serving stacks—encompassing compiler flags, kernel launch parameters, memory allocation strategies, and precision settings—is often combinatorially explosive, rendering exhaustive search infeasible during production. Consequently, researchers have developed techniques utilizing machine learning to prune this space, identifying Pareto-optimal configurations without evaluating every possibility. For instance, approaches that reduce large adaptation spaces in self-adaptive systems demonstrate that configurable machine learning pipelines can filter suboptimal options, reducing the time required to make adaptation decisions by over 90% while maintaining performance goals [282]. By integrating such reduction strategies into LLM serving runtimes, systems can rapidly converge on optimal batch sizes or quantization levels as request patterns shift, directly complementing the memory-centric and communication optimizations detailed in previous sections.

Furthermore, the integration of causal inference into performance modeling enhances the robustness of auto-tuning across diverse hardware environments, a critical requirement given the heterogeneity addressed in Section 6.5. Standard predictive models often struggle when deployed in unseen contexts due to distribution shifts in workload or hardware characteristics. Causal transfer learning approaches, such as those identifying invariant causal predictors, allow optimization processes to operate in a reduced search space that generalizes better across environmental changes. This is vital for LLM serving platforms scaling across clusters with mixed GPU generations or CPU-GPU nodes. Methods like CAMEO illustrate how identifying causal relationships enables faster optimization of system performance by reusing knowledge from source environments where intervention is cheaper, a principle directly applicable to calibrating LLM inference engines before full-scale deployment [283]. Similarly, frameworks that capture intricate interactions between configuration options via causal inference, such as Unicorn, provide reliable performance predictions for new environments, ensuring that auto-tuning decisions remain valid even as the serving infrastructure evolves [284].

Runtime reconfiguration also benefits significantly from reinforcement learning (RL) and control-theoretic hybrids, offering a dynamic solution to the “straggler” problems and load imbalances previously discussed. In scenarios where LLM serving must adhere to strict Service Level Objectives (SLOs) under varying loads, RL agents can learn optimal policies for resource allocation and task scheduling. Research into self-adaptive systems highlights the efficacy of combining Monitor-Analyze-Plan-Execute (MAPE) loops with reinforcement learning to guide the configuration of interacting modules, maximizing overall system utility despite combinatorial complexities [285]. Moreover, hybrid approaches that merge control theory with AI offer formal guarantees on stability while leveraging the flexibility of learning-based tuning. Such architectures utilize hierarchical managers to separate high-level requirements from low-level knob adjustments, ensuring that dynamic adaptation does not compromise system dependability—a crucial consideration for safety-critical LLM applications [286].

The concept of “lifelong” adaptation is particularly relevant for long-running LLM services where the adaptation space itself may drift over time due to model updates or changing user behavior. Systems equipped with lifelong machine learning layers can track these shifts, identify new tasks or workload characteristics, and update their internal models accordingly. This prevents the degradation of performance that occurs when static models encounter novel operating conditions, effectively allowing the serving infrastructure to evolve alongside the applications it supports [287]. Additionally, the use of ensemble learning techniques allows for more accurate modeling of performance and power characteristics by combining linear, nonlinear, and tree-based methods. This robustness is essential when optimizing parallel deep learning workloads, where hardware performance counters can be correlated with application properties to drive fine-grained improvements in energy and speed [288].

Hardware-specific co-design further extends these principles to the realm of reconfigurable computing and specialized accelerators, providing the physical substrate for the dynamic strategies described above. For systems utilizing FPGAs or dynamic reconfigurable regions, predicting the necessary resources and optimal floorplans is a complex challenge. Machine learning methodologies that estimate resource requirements based on historical data enable dynamic runtime reconfiguration, ensuring that hardware fabric is utilized efficiently without the overhead of static allocation [289]. Moreover, polymorphic architectures that support dynamic allocation of computing resources based on program needs exemplify the potential of self-aware systems. These architectures apply machine learning and control theory to autonomously manage resources, approximating computations to meet power and time budgets without explicit programmer intervention [290].

Finally, the success of these co-design strategies relies on collaborative infrastructure that facilitates the sharing of optimization knowledge across the community. Platforms that crowdsource experimentation and utilize unified repositories allow researchers and engineers to build upon collective findings, gradually improving optimization heuristics for compilers and runtime systems. This collaborative approach, exemplified by initiatives like Collective Mind, transforms the optimization process from an isolated, ad-hoc activity into a systematic, reproducible science, enabling the continuous improvement of auto-tuning strategies for complex systems [291]. By synthesizing these advanced co-design methodologies, future LLM serving systems can achieve unprecedented levels of efficiency, dynamically mapping algorithms to hardware resources with a degree of agility that static approaches cannot match, thereby closing the loop between the distributed architectures of Section 6.5 and the evolving demands of next-generation AI workloads.

7 Specialized Deployment Scenarios: Edge, Mobile, and Multimodal Systems

7.1 On-Device Inference and Hardware-Aware Model Adaptation

7.1 On-Device Inference and Hardware-Aware Model Adaptation

The deployment of Large Language Models (LLMs) directly on resource-constrained edge and mobile devices represents a paradigm shift in artificial intelligence, driven by the critical needs for data privacy, latency reduction, and operational autonomy. However, the astronomical model sizes of modern LLMs clash fundamentally with the severe memory and bandwidth limitations inherent to mobile hardware. To bridge this gap before considering offloading strategies, the research community has converged on a triad of optimization strategies strictly within the device boundary: ultra-low-bit quantization, heterogeneous compute scheduling, and adaptive architectural designs. These techniques collectively enable the execution of sophisticated language models on devices ranging from smartphones to microcontrollers, transforming theoretical capabilities into practical, on-device applications and laying the groundwork for the hybrid edge-cloud systems discussed in the subsequent section.

Ultra-low-bit quantization stands as the primary mechanism for reducing the memory footprint of LLMs to fit within the tight constraints of edge devices. Traditional full-precision models are often impractical for mobile GPUs or CPUs, necessitating aggressive compression strategies that maintain acceptable accuracy. Recent advancements have demonstrated that weight-only quantization can be achieved at extremely low bit-widths without catastrophic performance degradation. For instance, the [22] framework illustrates that by protecting only 1% of salient weights based on activation observations, models can be compressed significantly while preserving generalization capabilities across different domains. This approach has successfully enabled the deployment of massive models, such as the 70B parameter Llama-2, on mobile GPUs, achieving speedups of over 3x compared to standard FP16 implementations. Similarly, [292] introduces a sparse-quantized representation that isolates outlier weights into higher precision while compressing the majority of parameters to 3-4 bits. This method allows for near-lossless compression, enabling a 33B parameter model to run on a single consumer GPU with 24GB of memory, effectively democratizing access to powerful LLMs on commodity hardware.

Beyond weight compression, the quantization of activations remains a critical challenge due to the presence of outliers that distort the quantization range. Innovative solutions like [139] address this by employing adaptive channel reassembly, which redistributes the magnitude of outliers to balance activation distributions. This technique, combined with efficient low-rank tuning, allows for accurate 4-bit quantization of large models within hours on a single GPU. Furthermore, the choice between integer and floating-point formats is gaining attention, with studies suggesting that a mixture of formats tailored to specific layer distributions can outperform uniform quantization schemes. The [161] study advocates for Mixture of Formats Quantization (MoFQ), which dynamically selects the optimal format per layer, achieving state-of-the-art results in both weight-only and weight-activation scenarios without incurring additional hardware overhead.

Hardware-aware model adaptation extends beyond compression to include intelligent scheduling across heterogeneous compute units found in modern System-on-Chips (SoCs). Mobile devices typically integrate CPUs, GPUs, and Neural Processing Units (NPUs), each with distinct strengths regarding latency, throughput, and energy efficiency. Effective on-device inference requires orchestrating workloads to leverage these diverse resources simultaneously. The [243] framework proposes a principled approach to heterogeneous parallel computing, utilizing both CPUs and GPUs to mit-

igate I/O bottlenecks. By overlapping computation and asynchronous data transfers, HeteGen achieves substantial improvements in inference speed, surpassing state-of-the-art offloading methods by over 300%. Additionally, specialized engines like [293] implement operator optimizations and dynamic shape support tailored for mobile processors from vendors like Qualcomm and MediaTek. These engines utilize sub-tensor techniques to eliminate redundant KV cache copying and employ FP4 quantization methods to reduce dequantization overhead, resulting in decoding speeds that are 2 to 3 times faster than existing mobile inference solutions.

Adaptive “big-little” neural network strategies offer another layer of optimization, dynamically adjusting model complexity based on real-time resource availability and task difficulty. This concept, borrowed from processor architecture, involves switching between a lightweight “little” model for simple queries and a more capable “big” model for complex reasoning, thereby optimizing energy consumption and battery life. Research into [294] demonstrates that such strategies can achieve up to 80% energy reduction while maintaining prediction accuracy in human activity recognition tasks. This adaptability is crucial for edge devices where power budgets fluctuate. Moreover, systems like [295] take this further by constructing resource-efficient subnetworks that can adapt to variable constraints in microseconds. By exploiting structured sparsity and permutation invariance, REDS allows models to skip computational blocks dynamically, ensuring robust performance even when device resources are severely throttled.

The integration of these techniques is further supported by advanced compilation and runtime environments designed specifically for the edge. Frameworks like [258], although initially focused on vision, provide a blueprint for end-to-end compilation of ultra-low-bit models on ARM-based targets, automating the conversion of fake-quantized models into efficient integer-only representations. Similarly, [296] combines activation-guided quantization with token pruning to balance task performance and real-world inference speed on SIMD-based edge accelerators. These systems highlight the necessity of co-designing algorithms with hardware capabilities, ensuring that theoretical compression ratios translate into tangible latency reductions and energy savings. While these methods maximize local capability, they also define the boundary conditions for when local execution becomes insufficient, naturally motivating the transition to dynamic offloading strategies.

In conclusion, the landscape of on-device LLM inference is rapidly evolving through the synergistic application of ultra-low-bit quantization, heterogeneous scheduling, and adaptive architectural strategies. By leveraging insights from [22], [243], and [294], researchers are overcoming the traditional barriers of memory and compute. As mobile hardware continues to advance and software stacks become more sophisticated, the vision of running trillion-parameter class models locally on personal devices moves closer to reality. However, when even these optimized on-device strategies reach their limits under extreme constraints or complex tasks, the system must seamlessly extend its reach, setting the stage for the dynamic computation offloading and task partitioning strategies explored in the following section.

7.2 Dynamic Computation Offloading and Task Partitioning Strategies

7.2 Dynamic Computation Offloading and Task Partitioning Strategies

While Section 7.1 established the frontier of on-device inference through aggressive quantization and hardware-aware adaptation, there remains a class of workloads where even optimized local execution hits the physical limits of memory, compute, or battery life. Conversely, relying solely on centralized cloud inference introduces unacceptable latency and privacy risks for real-time applications. To bridge this gap, the paradigm of dynamic computation offloading and task partitioning

has emerged as a critical intermediate solution. Unlike the static compression techniques discussed previously, this approach fundamentally involves splitting inference workloads across a continuum—executing initial layers on the end device, intermediate segments on nearby edge servers, and complex reasoning tasks on remote cloud data centers. This strategy optimizes the trade-off between latency, energy consumption, and accuracy by adaptively selecting split points based on fluctuating network conditions and hardware states, effectively extending the device’s capabilities without the rigid topology of pure cloud reliance that Section 7.3 will later seek to decentralize further.

A foundational concept in this domain is Split Computing, where a Deep Neural Network (DNN) or LLM is divided into a “head” executed on the end device and a “tail” processed on an edge server or cloud. The primary challenge lies in determining the optimal split location dynamically, as static configurations often fail under variable network bandwidth and server loads. Research has demonstrated that dynamic split computing, which selects the optimal partition point based on the real-time state of the communication channel, can significantly outperform static approaches. By leveraging natural bottlenecks inherent in modern architectures, these systems avoid the need for retraining while adapting to environmental changes [297]. The complexity of this optimization is further compounded by the heterogeneity of edge clusters, where devices possess varying memory capacities and compute powers. Advanced algorithms have been developed to partition DNNs across distributed edge clusters to minimize bottleneck latency, achieving throughput improvements that scale effectively with the number of nodes [298].

To address the stochastic nature of wireless environments, context-adaptive frameworks have been proposed to manage the deployment lifecycle dynamically, ensuring smooth transitions that complement the adaptive strategies seen in on-device learning. For instance, the [299] framework introduces a novel methodology where DNNs are pre-partitioned into executable “atoms.” These atoms can be dynamically combined and offloaded using a graph-based decision algorithm that responds to runtime context, achieving substantial latency reductions and memory savings compared to rigid baselines. Similarly, the need for seamless adaptation during operational shifts has led to innovations like [300], which utilizes “Dynamic Switching” to initialize new pipelines with updated partitions without incurring the significant service downtime associated with traditional pause-and-resume mechanisms. This capability is vital for maintaining high availability in mobile scenarios where network speeds fluctuate rapidly, serving as a precursor to the more complex fault-tolerant orchestration required in fully distributed peer-to-peer networks.

The integration of early exiting mechanisms with split computing offers an additional layer of adaptability, creating a hybrid approach known as SplitEE or similar variants. In these systems, the decision to offload is not only based on the split layer but also on the confidence of intermediate exits. If an early exit on the device provides sufficient accuracy, the inference terminates locally; otherwise, the intermediate features are transmitted to the cloud for further processing. This strategy effectively reduces communication overhead for “easy” samples while reserving cloud resources for complex inputs [301]. However, the reliability of such offloading decisions hinges on accurate confidence estimation. Studies have highlighted that miscalibrated DNNs can lead to erroneous offloading choices, suggesting that calibration-aided partitioning is essential for maintaining inference integrity [302]. Furthermore, specialized approaches like [303] extend this logic to handle environmental distortions, employing unsupervised online algorithms to adjust splitting policies in the absence of ground truth.

Optimization in dynamic offloading often extends beyond simple latency minimization to include energy efficiency and privacy preservation, addressing constraints that are paramount when moving data off-device. Frameworks such as [304] co-optimize Dynamic Voltage and Frequency Scaling

(DVFS) with offloading parameters using deep reinforcement learning, simultaneously adjusting hardware frequencies and feature map offloading strategies to reduce energy consumption without sacrificing accuracy. Privacy concerns, particularly regarding the reconstruction of raw data from intermediate feature maps, have also driven the development of secure partitioning schemes. Systems like [305] incorporate differential privacy mechanisms directly into the partitioning process, adding refined noise to offloaded features to mitigate reconstruction attacks while adapting the partition point to dynamic bandwidth constraints.

Moreover, the orchestration of these strategies in multi-user or multi-task environments introduces further complexity, acting as a stepping stone toward the massive-scale coordination discussed in collaborative edge intelligence. Algorithms must account for batch processing capabilities on edge servers and the coupled nature of communication and computation resources. Research into multi-user co-inference has proposed optimal grouping algorithms and deep deterministic policy gradient agents to manage concurrent task arrivals and varying latency constraints [245]. Additionally, the emergence of decentralized approaches allows individual devices to make proactive offloading decisions without global knowledge, as seen in [306], which addresses heterogeneity and data leakage risks in federated settings. These decentralized mechanisms begin to blur the line between client-server offloading and the peer-to-peer collaboration that defines the next evolutionary step.

In summary, dynamic computation offloading and task partitioning represent a multifaceted solution to the efficiency trilemma in edge AI, effectively bridging the gap between the localized optimizations of Section 7.1 and the distributed collaboration of Section 7.3. By continuously monitoring system states and employing adaptive algorithms, these strategies ensure that LLM serving remains robust, efficient, and responsive across diverse and volatile deployment scenarios. While these methods successfully leverage the client-server continuum, they often rely on centralized coordination or fixed topologies that may struggle under extreme network volatility or when cloud access is limited. This limitation naturally motivates the transition to Collaborative Edge Intelligence, where the aggregation of resources across multiple heterogeneous devices via Device-to-Device links offers a more resilient and scalable architecture for future AI services.

7.3 Collaborative Edge Intelligence and Distributed Resource Orchestration

7.3 Collaborative Edge Intelligence and Distributed Resource Orchestration

While dynamic computation offloading and task partitioning (discussed in Section 7.2) effectively bridge the gap between single devices and centralized clouds, they often rely on a rigid client-server topology that may struggle under extreme resource constraints or network volatility. To further overcome the limitations of standalone edge units and reduce dependence on distant cloud infrastructure, the paradigm of Collaborative Edge Intelligence has emerged. This approach leverages the aggregate resources of multiple heterogeneous edge devices connected via Device-to-Device (D2D) links, transforming a collection of constrained nodes into a unified, high-capacity inference cluster. The core challenge shifts from simple split-point selection to the sophisticated orchestration of distributed resources, aiming to minimize inter-device communication overhead while maximizing collective throughput and reliability.

A fundamental strategy in this domain involves the horizontal distribution of model layers across a device cluster. By partitioning a Deep Neural Network (DNN) into sub-models and assigning them to different nodes, systems can effectively pool the memory and processing capabilities of the

entire network. For instance, the DEFER framework demonstrates how spreading DNN partitions across multiple compute nodes in a series pattern can significantly increase inference throughput while reducing per-device energy consumption. In experimental evaluations involving ResNet50, such distributed approaches achieved a 53% higher throughput compared to single-device inference, highlighting the efficacy of workload sharing [307]. Similarly, the SEIFER framework utilizes Kubernetes-based orchestration to automate the partitioning and placement of DNNs across edge networks, proving that scalable deployment can improve the inference pipeline throughput by up to 200% when utilizing sufficient numbers of resource-constrained nodes [308]. These systems illustrate that effective resource orchestration requires not only algorithmic partitioning but also robust deployment mechanisms capable of handling node heterogeneity and fault tolerance.

However, naive model splitting in a distributed setting often incurs substantial communication costs due to the transmission of large intermediate feature maps between devices. To address this, recent research has focused on optimizing the trade-off between computation and communication through advanced task orchestration strategies. The DistrEdge method employs deep reinforcement learning to dynamically adjust distribution strategies based on real-time device computing characteristics and network conditions, achieving speedups of 1.1 to 3 times over state-of-the-art methods in heterogeneous environments [309]. Furthermore, the PICO framework addresses the NP-hard problem of mapping Convolutional Neural Networks (CNNs) with complex directed acyclic graph structures onto diverse mobile devices. By utilizing a generic graph partition algorithm and a many-to-many mapping strategy, PICO successfully balances computation and communication loads, delivering throughput improvements ranging from 1.8 to 6.8 times on clusters of Raspberry Pi devices [310].

Beyond static partitioning, dynamic collaboration mechanisms such as coded cooperative computation have gained traction for enhancing reliability and reducing latency in time-varying edge environments. The Coded Cooperative Computation Protocol (C3P) introduces erasure coding to mix data in sub-tasks, allowing the system to adapt to heterogeneous and fluctuating resources. This approach ensures high resource utilization efficiency, exceeding 99%, and significantly reduces task completion delays compared to baseline offloading strategies [311]. Additionally, decentralized collaboration frameworks leverage D2D connectivity to mitigate the “straggler effect” caused by resource heterogeneity. By designing D2D-aided coded federated learning methods, systems can optimize compression rates to minimize communication rounds, which is particularly beneficial for real-time applications where continuous data generation leads to model drift [312].

To further alleviate the communication bottleneck inherent in multi-device collaboration, task-oriented communication represents a critical frontier. Rather than transmitting raw data or full feature maps, devices can extract and transmit only task-relevant information. The concept of task-oriented communication utilizes the Information Bottleneck principle to remove data redundancy, ensuring that only essential features are shared among devices for downstream inference [313]. This is complemented by semantic communication strategies, such as attention-aware patch selection in Vision Transformers, where edge devices transmit only critical image patches identified by attention scores, reducing communication overhead by up to 68% with minimal accuracy loss [314].

Orchestration in collaborative edge intelligence must also account for the stochastic nature of wireless channels and the potential for device failures, which are more pronounced in peer-to-peer topologies than in centralized offloading. Frameworks like Edge-PRUNE provide a flexible infrastructure for fault-tolerant collaborative inference, formally defining computation models that allow systems to maintain operation despite device failures, albeit with some execution time overhead [315]. Moreover, joint optimization of routing and computation offloading is essential in multi-hop D2D networks. Recent work formulates network-wide cost minimization problems that consider

congestion and queueing delays, devising fully distributed algorithms that converge to optimal solutions even in congested scenarios [316].

Finally, the integration of game-theoretic approaches and mean field models offers a scalable solution for managing massive populations of collaborating devices. By modeling task offloading as a Stackelberg game and analyzing D2D collaboration state evolution on dynamic graphs, researchers have developed online schemes that optimize service utility and system costs simultaneously. These methods have demonstrated the ability to mitigate user workloads by nearly 74% while maintaining high energy efficiency [317]. As edge devices become increasingly heterogeneous and network conditions more volatile, the future of collaborative edge intelligence will rely heavily on these adaptive, distributed orchestration mechanisms. These strategies not only extend the capabilities of Section 7.2 by decentralizing computation but also lay the groundwork for the energy-efficient resource allocation and power management techniques detailed in the subsequent Section 7.4, ensuring that distributed AI services remain both robust and sustainable.

7.4 Energy-Efficient Resource Allocation and Power Management

7.4 Energy-Efficient Resource Allocation and Power Management

Building upon the distributed orchestration mechanisms established in Section 7.3, which leverage collaborative edge intelligence to pool resources across heterogeneous devices, a critical constraint emerges: the sustainability of such distributed operations under strict energy budgets. While collaborative frameworks effectively mitigate latency and reliability issues through device-to-device (D2D) links and horizontal model partitioning, they inherently increase the complexity of power management due to the added communication overhead and the stochastic nature of wireless channels. Consequently, the paradigm shifts from mere resource pooling to energy-efficient resource allocation and power management. This subsection investigates joint optimization frameworks that simultaneously address communication and computation resources, detailing advanced methodologies for minimizing energy consumption through dynamic voltage scaling, task offloading strategies, and reinforcement learning-based scheduling. These strategies are essential not only for extending the operational lifetime of battery-constrained edge nodes but also for ensuring that the distributed intelligence described previously remains viable in real-world deployments.

The fundamental objective in energy-efficient Mobile Edge Computing (MEC) systems is to minimize the total system cost, typically comprising service delay and operational energy expenditure. Traditional approaches often treat computation and communication as separate domains; however, modern frameworks recognize the strong coupling between transmission power, bandwidth allocation, and CPU/GPU cycle frequencies, a coupling that is particularly pronounced in the multi-hop D2D topologies discussed in Section 7.3. For instance, joint optimization of task offloading scheduling and transmit power allocation has been shown to significantly reduce execution latency and device energy consumption. By leveraging flow shop scheduling theory to determine the optimal order of offloading and employing convex optimization for power allocation, systems can achieve near-optimal execution delays while realizing substantial energy savings [318]. Furthermore, in scenarios involving multiple users sharing edge resources, the allocation problem often manifests as a Mixed-Integer Non-Linear Program (MINLP). To address the NP-hard nature of these problems, researchers have developed decomposition techniques that separate offloading decisions from resource allocation, utilizing convex and quasi-convex optimization to derive sub-optimal solutions that closely approximate global optima [319].

A particularly promising avenue for sustaining the long-term operation of collaborative edge clusters involves the integration of renewable energy sources. While sustainable, these sources introduce high intermittency and unpredictability, complicating the stable orchestration required by frameworks like DEFER or SEIFER. Addressing this volatility requires dynamic workload offloading policies that adapt to real-time energy harvesting conditions. Reinforcement learning (RL) has emerged as a powerful tool for such online learning scenarios. Algorithms that decompose offline value iteration with online RL have demonstrated the ability to learn optimal policies for dynamic workload offloading and edge server provisioning on the fly. These approaches significantly outperform standard Q-learning algorithms in terms of learning rate and run-time performance, effectively minimizing long-term system costs in renewable-powered environments [320]. Similarly, other studies have proposed efficient reinforcement learning-based resource management algorithms that handle the unpredictability of renewables by learning optimal policies for offloading to centralized clouds, thereby ensuring high quality of service despite fluctuating power supplies [321].

Beyond simple offloading, advanced frameworks explore multi-objective optimization to balance conflicting goals such as energy consumption, latency, and fairness, which are critical when managing a diverse population of collaborating devices. In practical systems, the relative importance of these objectives is often unknown or varies dynamically. Multi-objective deep reinforcement learning (MORL) schemes, particularly those utilizing Proximal Policy Optimization (PPO), have been designed to handle unknown preference weights. By constructing sophisticated state encodings and reward functions, these frameworks can enhance the hypervolume of the Pareto front, offering a diverse set of optimal trade-offs between delay and energy consumption [322]. Additionally, fairness concerns necessitate energy-based proportional fairness frameworks that consider the battery status and service requirements of individual users. Distributed algorithms like dynamic branch-and-bound Benders decomposition have been proposed to solve these complex optimization problems, ensuring that resource allocation maximizes fairness indices such as Jain’s index while maintaining overall system efficiency [323].

At the hardware level, Dynamic Voltage and Frequency Scaling (DVFS) remains a cornerstone technique for reducing energy consumption. When combined with heterogeneous computing architectures featuring both CPUs and GPUs, joint computation-and-communication resource management (C²RM) frameworks can optimize speed scaling and workload partitioning. Theoretical analyses have revealed the existence of equilibriums in energy rates across different control variables, which can be exploited to design algorithms that compute optimal policies faster than standard optimization tools [324]. Moreover, in time-sensitive networks, iterative algorithms that decompose problems into offloading selection, transmission power optimization, and subcarrier allocation have proven effective. These methods often employ equivalent parametric convex programming to handle non-linear objective functions, demonstrating superior performance over existing schemes in minimizing energy consumption under strict latency constraints [325].

The stochastic nature of mobile environments, characterized by time-varying wireless channels and random task arrivals, further complicates resource allocation, echoing the challenges of network volatility mentioned in the context of collaborative intelligence. Deep Reinforcement Learning (DRL) frameworks, such as those based on Deep Deterministic Policy Gradient (DDPG), enable decentralized dynamic computation offloading. By allowing each mobile user to learn efficient policies independently based on local observations, these systems can adaptively allocate power for local execution and task offloading, outperforming conventional discrete power control strategies [326]. Similarly, end-to-end DRL approaches have been utilized to select optimal edge servers and allocate computational resources in dynamic IoT environments, maximizing the number of

completed tasks before deadlines while minimizing energy usage [327].

Finally, the concept of “green” computing extends to the orchestration of multiple orchestrators and learners in mobile edge learning scenarios, reinforcing the need for scalable solutions in massive device populations. Lightweight heuristic algorithms have been developed to facilitate trade-offs between energy consumption, learning accuracy, and solution complexity in large-scale decentralized systems [328]. Furthermore, collaborative frameworks that integrate computing, pushing, and caching via Soft Actor-Critic learning have shown the ability to implicitly predict user requests, thereby reducing unnecessary transmission loads and computing costs [329]. Collectively, these advancements underscore the necessity of holistic, adaptive, and learning-driven approaches to achieve sustainable energy management. These energy-efficient strategies provide the foundational stability required for the next layer of complexity: handling user mobility. As detailed in the subsequent Section 7.5, maintaining these optimized energy and resource balances becomes even more challenging when end-users are in motion, necessitating dynamic service placement and migration protocols that preserve both power efficiency and service continuity.

7.5 Mobility-Aware Service Placement and Migration

7.5 Mobility-Aware Service Placement and Migration

Building upon the energy-efficient resource allocation strategies discussed in the previous section, the deployment of Large Language Models (LLMs) in mobile edge computing (MEC) environments faces an additional, dynamic dimension: the inherent mobility of end-users. While static resource optimization addresses power and latency trade-offs for fixed nodes, mobile scenarios require inference services to dynamically adapt to a user’s changing geographical location to maintain strict latency Service Level Objectives (SLOs). The fundamental challenge here is a three-way trade-off between service continuity, operational cost, and energy consumption. Frequently migrating an LLM service instance to follow a moving user minimizes transmission latency but incurs significant overhead due to state transfer, model loading times, and network bandwidth consumption, which directly impacts the energy budgets analyzed previously. Conversely, keeping the service static at a distant edge node preserves computational resources but results in unacceptable latency spikes that degrade the interactive experience required for generative AI applications. Addressing this dichotomy necessitates sophisticated mobility-aware service placement and migration protocols that leverage predictive analytics and dynamic resource orchestration, serving as a critical bridge between static energy management and the specialized, real-time architectures required for multimodal applications discussed next.

A primary strategy for mitigating the adverse effects of user mobility, while adhering to energy constraints, is the adoption of predictive service placement. By anticipating user trajectories, systems can proactively migrate or replicate LLM instances to edge nodes along the user’s future path before the handover occurs. This approach transforms the reactive problem of handover management into a proactive optimization task, reducing the need for emergency, energy-intensive migrations. Research in broader mobile networking has demonstrated that leveraging prediction significantly enhances performance. For instance, frameworks that utilize predicted near-future information to guide service placement have been shown to effectively balance the conflict between improving latency performance and reducing long-term migration costs [330]. These predictive models often rely on historical trajectory data and machine learning algorithms, such as Long Short-Term Memory (LSTM) networks, to forecast user movement trends with high accuracy. In the context of Vehicular Fog Computing, learning-based handover optimization using neural networks has achieved over

99% accuracy in predicting the correct fog node, thereby minimizing service interruption during transitions [331]. Furthermore, sequence-to-sequence modeling has been successfully applied to estimate handover cells and dwell times in dense 5G networks, providing the necessary temporal windows for seamless service migration [332].

Beyond simple prediction, the architecture of mobility management itself must evolve from centralized, rigid structures to flexible, user-centric paradigms to support the fluid nature of edge LLM serving. Traditional mobility management often relies on fixed anchor points, which can create bottlenecks and sub-optimal routing paths, especially in ultra-dense networks characterized by frequent cell changes. To address this, recent literature advocates for distributed and anchor-less solutions. For example, anchor-less mobility management schemes in Information-Centric Networking (ICN) have been proposed to manage micro-mobility without the latency penalties associated with tunneling traffic through a central core [333]. Similarly, path-based forwarding solutions that decouple path calculation from data transfer eliminate the need for anchoring, allowing for more flexible service provisioning and reduced network costs compared to standardized anchored solutions like Proxy Mobile IPv6 [334]. Applying these concepts to LLM serving implies that the “state” of the LLM (particularly the KV cache) should be able to flow with the user across the edge continuum without being tethered to a specific physical server’s IP address, potentially utilizing context transfer mechanisms to preserve session continuity [335].

Dynamic service migration protocols must also account for the heterogeneity of edge resources and the stochastic nature of network conditions, requiring decisions that are both intelligent and adaptable. Static rules for migration are often insufficient in highly dynamic environments where user speed and network load fluctuate rapidly. Consequently, intelligent frameworks employing reinforcement learning and online optimization have emerged to navigate these complexities. A notable approach involves modeling the service migration problem as a Partially Observable Markov Decision Process (POMDP), where agents make online migration decisions based on incomplete system information, effectively handling the uncertainty of user mobility and resource availability [336]. Additionally, probabilistic service migration approaches have been developed to optimize delay and migration costs simultaneously, demonstrating significant improvements in service delay and reductions in migration overhead during peak traffic periods [337]. These methods are crucial for LLMs, where the cost of migrating large model weights and KV caches is substantial, requiring precise decision-making to ensure that the benefits of migration outweigh the costs.

The integration of LLMs specifically adds a layer of complexity due to the stateful nature of generative inference, distinguishing it from standard microservice migration. Unlike stateless services, LLMs maintain a persistent KV cache across multiple token generation steps, making the efficient transfer of this context paramount. Recent work proposes treating the LLM as a system service on mobile devices, emphasizing fine-grained, chunk-wise KV cache compression and swapping to minimize context switching overhead under tight memory budgets [338]. Extending this to the edge-cloud continuum requires “computing follows vehicles” paradigms, where decentralized optimization frameworks allocate resources based on real-time spatio-temporal service requests [339]. Such frameworks must coordinate not only the placement of the model but also the efficient transfer of the evolving context state. Furthermore, mobility-aware caching strategies in Device-to-Device (D2D) networks suggest that caching policies should be adapted based on user speed and contact patterns to maximize data offloading, a principle that could be extended to caching frequently accessed LLM prompts or LoRA adapters at the edge [340].

Finally, the sustainability and energy efficiency of these migration strategies remain a critical constraint, echoing the themes of the preceding section on power management. Frequent migration and

computation offloading consume significant energy, which is a limiting factor for battery-powered mobile devices and energy-constrained edge nodes. Energy-aware mobility management schemes utilizing Lyapunov optimization and multi-armed bandit theories have been proposed to optimize delay under long-term energy constraints, effectively handling imperfect system state information [341]. Similarly, risk-aware resource allocation strategies using machine learning have shown potential in managing coexisting traffic types while satisfying stringent reliability requirements, a capability essential for maintaining the quality of real-time LLM interactions in mobile scenarios [342]. As 5G and beyond networks evolve, the convergence of these predictive, distributed, and energy-efficient mobility management techniques will be paramount. This robust mobility foundation is a prerequisite for the next generation of applications, enabling the specialized, low-latency architectures for multimodal and real-time edge scenarios, such as AR/VR and autonomous systems, explored in the following section.

7.6 Specialized Architectures for Multimodal and Real-Time Edge Applications

7.6 Specialized Architectures for Multimodal and Real-Time Edge Applications

Building upon the mobility-aware service placement and migration strategies discussed in the previous section, which ensure that LLM instances dynamically track users across the edge continuum, the focus now shifts to the specialized architectural designs required to support the stringent demands of multimodal and real-time applications. While Section 7.5 addressed the *where* and *when* of service deployment in mobile scenarios, this section addresses the *how*: the internal system mechanisms necessary to process heterogeneous data streams—text, audio, and video—within the tight latency and bandwidth budgets of Augmented Reality (AR), Virtual Reality (VR), and autonomous systems. These applications impose constraints that transcend simple model compression, necessitating holistic system designs that integrate efficient video segmentation, dynamic stream processing schedulers, and adaptive middleware. Furthermore, as these architectures scale to support ubiquitous intelligence, they introduce a critical paradox: the hardware volume and energy consumption required for such high-performance distributed systems can exacerbate environmental impacts, a challenge that sets the stage for the sustainable computing paradigms explored in the following section.

A critical component of real-time multimodal edge applications is efficient video segmentation, which serves as the perceptual backbone for understanding dynamic environments. In scenarios such as autonomous driving, robotics, and smart surveillance, the ability to process video streams with low latency is paramount. Traditional segmentation models often suffer from high computational complexity, making them unsuitable for resource-constrained edge devices. To mitigate this, recent architectural innovations leverage temporal locality and feature reuse. For instance, frameworks that exploit the redundancy between consecutive video frames can significantly reduce inference costs by updating only a subset of feature maps or reusing computations from previous time steps. This approach is exemplified by systems that utilize partial channel updates or spatial-temporal masking to accelerate future video prediction while maintaining accuracy [343]. Furthermore, lightweight architectures designed specifically for edge deployment, such as those utilizing depthwise separable convolutions or dual-multiscale residual blocks, have demonstrated the ability to perform high-accuracy segmentation on IoT devices with minimal parameter counts [344]. These designs are crucial for enabling real-time all-purpose segmentation, where a single model must handle interactive, panoptic, and video segmentation tasks simultaneously without compromising frame rates [345].

Beyond the model architecture itself, the orchestration of data flow through sophisticated stream processing schedulers is essential for maintaining real-time performance in multimodal systems. These schedulers must dynamically allocate computational resources based on the urgency of the task, the availability of heterogeneous hardware (CPUs, GPUs, NPUs), and the specific latency requirements of the application. In the context of AR and VR, where motion-to-photon latency must be kept imperceptibly low to prevent user discomfort, specialized scheduling algorithms are employed to manage the rendering and inferencing pipelines. Research indicates that optimizing service placement in edge-to-cloud architectures using multi-objective genetic algorithms can significantly reduce response times while maximizing system reliability for latency-sensitive AR/VR services [346]. Moreover, proactive scheduling mechanisms that leverage predictive models to anticipate user orientation or field-of-view changes allow for pre-fetching and caching of relevant content, thereby reducing mean latency by substantial margins in wireless VR environments [347]. Such schedulers often operate in conjunction with adaptive batching strategies that group Regions of Interest (RoIs) from multiple streams to optimize serverless function invocations, ensuring that Service Level Objectives (SLOs) are met even under fluctuating workloads [348].

Middleware solutions play an equally pivotal role in bridging the gap between high-level multimodal applications and the underlying heterogeneous edge infrastructure. These middleware layers are responsible for managing the complexity of splitting inference tasks across devices, handling modality synchronization, and optimizing communication protocols. For immersive experiences requiring the integration of visual, auditory, and linguistic channels, middleware must facilitate the efficient fusion of these disparate data streams. Innovative systems have been developed to harness LLMs for assimilating information from egocentric video, speech, and context, thereby enhancing state estimation in AR environments [349]. Additionally, to address the bandwidth bottlenecks inherent in streaming high-resolution video for analytics, middleware frameworks often incorporate intelligent filtering and compression techniques. Systems like DeepStream employ Regions of Interest detection algorithms at the edge to eliminate spatial redundancy before transmission, while others utilize adaptive hybrid codecs to balance accuracy and throughput across multiple concurrent video streams [350][351].

The convergence of these architectural elements is particularly evident in the development of systems tailored for assistive technologies and complex interactive environments. For example, wearable assistive technologies for individuals with blindness rely on robust video streaming platforms that can adapt to time-varying network conditions while offloading computationally intensive microservices to the edge [352]. Similarly, the deployment of event-based vision systems, which offer high temporal resolution and dynamic range, requires middleware capable of mapping irregular event streams to heterogeneous processing elements efficiently [353]. These specialized architectures often employ cross-modal learning schemes, such as hybrid encoders combining Spiking Neural Networks (SNNs) and Artificial Neural Networks (ANNs), to minimize latency while retaining rich spatio-temporal embeddings [354].

In conclusion, the realization of efficient multimodal LLM serving and real-time edge applications demands a departure from monolithic model deployments toward specialized, co-designed architectures. By integrating advanced video segmentation techniques that exploit temporal redundancy, deploying dynamic stream processing schedulers that adapt to hardware heterogeneity, and utilizing intelligent middleware for modality fusion and bandwidth optimization, researchers are overcoming the stringent constraints of the edge. These architectural advancements not only enable low-latency interactions for AR/VR and autonomous systems but also pave the way for ubiquitous, intelligent multimodal assistants. However, the rapid proliferation of these high-performance, heterogeneous

systems introduces significant environmental challenges regarding energy consumption and hardware lifecycle. Addressing these ecological implications requires a shift toward green computing paradigms, which will be the central focus of the subsequent section.

7.7 Sustainable Edge AI and Green Computing Paradigms

7.7 Sustainable Edge AI and Green Computing Paradigms

While Section 7.6 detailed the specialized architectures required to meet the stringent latency and bandwidth constraints of multimodal edge applications, the rapid proliferation of these systems introduces a critical paradox: the very deployment strategies that enhance real-time performance risk exacerbating the environmental crisis through increased hardware volume and energy consumption. As the industry scales generative AI onto resource-constrained devices, evaluating the holistic environmental impact becomes paramount. This evaluation must extend beyond the operational energy metrics discussed in prior optimization techniques to encompass the full lifecycle of the computing infrastructure, including embodied carbon from manufacturing, disposal complexities, and the interplay between hardware selection and algorithmic efficiency. The prevailing narrative that edge computing is inherently “greener” due to reduced data transmission requires rigorous scrutiny, as the sheer volume of heterogeneous devices required for distributed LLM inference can lead to significant ecological burdens if not managed through sustainable paradigms.

A foundational aspect of sustainable edge AI is the accurate quantification of the carbon footprint associated with edge devices, a factor often overlooked in architecture-centric designs. Traditional assessments frequently neglect embodied carbon—the emissions generated during the extraction of raw materials, manufacturing, and transportation of hardware components. Research indicates that for many IoT and edge devices, the production carbon footprint can vary by a factor of more than 150x depending on the device complexity, and in some scenarios, the embodied emissions may dominate the total lifecycle impact [355]. Furthermore, the manufacturing of storage systems and semiconductors contributes substantially to global CO₂ emissions, with the production of a single gigabyte of flash memory emitting approximately 0.16 kg of CO₂ [356]. Consequently, sustainable edge deployment strategies must prioritize embodied energy-aware hardware selection. This involves choosing hardware architectures that offer the best balance between computational capability and manufacturing impact, potentially favoring reconfigurable platforms like FPGAs which, despite higher initial embodied costs, can amortize their carbon footprint over longer lifespans and diverse applications through reuse [357].

To address these challenges, the concept of “Green Edge AI” has emerged, advocating for a shift from the static, performance-only optimization typical of specialized architectures to dynamic, carbon-aware resource management. This paradigm recognizes that energy efficiency alone is insufficient; the carbon intensity of the electricity source powering the edge node must also be considered. Frameworks such as GreenScale have demonstrated that optimizing application placement and scheduling based on time-and-location-specific carbon intensity can reduce carbon emissions by up to 29.1% compared to traditional performance-centric approaches [358]. Similarly, systems like EASE integrate renewable energy forecasting with job scheduling to minimize the carbon footprint of vehicular edge networks, achieving near carbon neutrality while maintaining Quality of Service (QoS) [359]. These approaches highlight the necessity of context-aware orchestration, where inference tasks are dynamically migrated between edge nodes, cloud servers, and user devices based on real-time grid carbon intensity and local renewable energy availability, complementing the stream processing schedulers discussed previously.

Lifecycle-aware model compression techniques represent another critical pillar of sustainable edge AI, bridging the gap between the efficiency methods covered earlier in this survey and broader environmental goals. While quantization and pruning are traditionally employed to meet memory and latency constraints on edge devices, their role in sustainability is profound. By reducing the model size and computational complexity, these techniques directly lower operational energy consumption. However, a truly sustainable approach must also consider the trade-offs between compression ratios and the need for frequent model retraining or hardware replacement. Energy-aware model compression methods, such as EDCompress, utilize reinforcement learning to identify optimal compression strategies tailored to specific hardware dataflows, achieving up to 37x improvements in energy efficiency with negligible accuracy loss [360]. Moreover, the integration of compressed models with carbon-aware inference services, as seen in frameworks like Clover, allows for dynamic adjustment of model quality based on current carbon constraints, effectively balancing accuracy with environmental impact [361].

The sustainability of edge LLMs also depends on an architectural shift towards modular and reusable hardware designs to counter the “defuturing” effect of rapidly obsolete hardware, which is driven by the relentless pursuit of higher performance and contributes significantly to electronic waste and embodied emissions [362]. To mitigate this, future edge infrastructures should embrace chiplet-based architectures and refurbished hardware solutions, such as REFRESH FPGAs, which leverage retired dies integrated via 2.5D packaging to extend hardware utility and amortize embodied costs [363]. Additionally, the adoption of photonic integrated circuits offers a promising avenue for ultra-low-power inference, potentially reducing both operational and embodied carbon footprints compared to traditional CMOS-based accelerators [364].

Finally, achieving a sustainable edge AI ecosystem requires a holistic governance framework that incorporates social and environmental certifications, extending the system-level orchestration principles to policy and ethics. Initiatives like SECure propose an ESG-inspired framework that combines compute-efficient machine learning with data sovereignty and federated learning to create eco-socially responsible AI systems [365]. Public policy recommendations further emphasize the need for international cooperation to establish common standards for carbon neutrality in AI, urging a transition from isolated efficiency gains to systemic, lifecycle-oriented sustainability strategies [366]. By integrating embodied energy-aware hardware selection, lifecycle-aware compression, carbon-aware scheduling, and circular economy principles, the field can mitigate the environmental risks of edge LLM deployment. Only through such comprehensive green computing paradigms can the promise of ubiquitous, intelligent edge systems—enabled by the architectural innovations described in Section 7.6—be realized without compromising planetary boundaries.

8 Reliability, Security, and Evaluation Benchmarks

8.1 The Impact of Compression on Parametric Knowledge and Hallucination

8.1 The Impact of Compression on Parametric Knowledge and Hallucination

The deployment of Large Language Models (LLMs) in resource-constrained environments has necessitated the widespread adoption of model compression techniques, primarily pruning and quantization. While these methods successfully reduce memory footprints and accelerate inference, they introduce complex trade-offs regarding the model’s retained parametric knowledge and its propensity to generate hallucinations. A critical challenge in this domain is that standard evaluation

metrics, such as perplexity or top-line accuracy, often fail to capture subtle degradations in factual reliability and reasoning capabilities. Recent research indicates that compression does not merely scale down model performance uniformly; rather, it can fundamentally alter the internal representation of knowledge, leading to the emergence of specific failure modes that are distinct from those observed in dense, full-precision models.

One of the most significant findings in the study of compressed LLMs is the divergence between general performance metrics and knowledge-intensive tasks. Although training-free compression methods can achieve high sparsity ratios or low bit-widths with negligible changes in perplexity, these models often suffer severe degradation when evaluated on benchmarks requiring precise factual retrieval and reasoning. The study [367] introduces the Knowledge-Intensive Compressed LLM Benchmark (LLM-KICK) to highlight this discrepancy, revealing that pruning methods, in particular, can fail at trivial sparsity ratios (e.g., 25-30%) on knowledge-intensive tasks despite maintaining low perplexity. This suggests that perplexity is an insufficient proxy for truthfulness, as it measures the likelihood of token sequences rather than the integrity of the underlying factual knowledge encoded in the weights. Consequently, compressed models may appear fluent while systematically hallucinating facts that were present in the original dense model.

The mechanism by which compression induces hallucination is closely tied to the disruption of parametric knowledge. There is an ongoing debate regarding whether compression causes models to “forget” knowledge entirely or merely “displace” it, making it inaccessible without specific prompting strategies. Research presented in [368] investigates this by comparing retraining approaches against inference-time dynamic prompting. The findings strongly support the “knowledge displaced” conjecture, suggesting that the information remains latent within the compressed weights but requires specific input-side augmentations to be retrieved effectively. This implies that hallucinations in compressed models may often stem from a failure in retrieval mechanisms rather than a total erasure of facts, offering a pathway for mitigation through advanced prompting techniques rather than solely through architectural changes.

Furthermore, the relationship between model compression and hallucination is not uniform across all types of errors. Specific studies have begun to dissect the internal dynamics that lead to non-factual generations. For instance, [369] identifies that hallucinations often arise from insufficient subject attribute knowledge in lower layers or a failure to select correct attributes in upper layers. When pruning or quantization is applied, these delicate mechanistic balances are disturbed. The removal of small-magnitude weights, traditionally considered redundant “junk DNA,” has been shown to irreversibly impair performance on difficult downstream tasks, as noted in [370]. This monotonic relationship suggests that aggressive pruning strips away the nuanced parameters necessary for resolving complex queries, thereby increasing the likelihood of the model resorting to associative memory and generating hallucinations to fill knowledge gaps.

To address these challenges, the community is moving beyond standard accuracy metrics toward specialized detectors and metrics designed to identify hallucinations in compressed settings. New approaches leverage the internal states of the model to detect inconsistencies before they manifest in the output. The work [371] proposes that correct generations exhibit sharper context activations in hidden states compared to incorrect ones, offering an entropy-based metric to constrain decoding and mitigate hallucinations. Similarly, [372] exploits the eigenvalues of response covariance matrices in the dense embedding space to measure semantic consistency, providing a robust method for detecting overconfident hallucinations that might persist even after quantization. These internal signal-based methods are crucial because external output analysis often fails to distinguish between a confident hallucination and a factual statement in compressed models.

Additionally, the impact of compression extends to the model’s reliance on parametric versus contextual knowledge. In scenarios involving knowledge conflicts, compressed models may exhibit an exacerbated over-reliance on memorized parametric knowledge, leading to hallucinations when faced with contradictory context. The framework presented in [373] highlights how models often prioritize internal memories over provided context, a behavior that can be amplified when compression reduces the model’s capacity to process complex contextual nuances. Moreover, the assessment of compression effects must account for the specific nature of the hallucination, whether it is an internal fabrication or an external misalignment. The study [374] emphasizes the need to associate hallucination rates with specific risk factors, such as deficits in commonsense memorization or relational reasoning, which are precisely the areas most vulnerable to the noise introduced by quantization and pruning.

Finally, the evaluation of compressed models requires a holistic approach that considers not just the rate of hallucination but also the specific conditions under which they occur. Research such as [375] demonstrates that hallucinations regarding known facts exhibit distinct dynamic curves in output token probabilities across model layers. By utilizing these dynamic patterns as features, classifiers can detect hallucinatory predictions with high accuracy. This underscores the necessity of developing evaluation benchmarks that probe the depth of parametric knowledge retention rather than superficial fluency. As the field progresses, the integration of these specialized metrics and the understanding of the “displaced” nature of knowledge will be paramount in designing compression strategies that maintain the trustworthiness and reliability of LLMs in real-world applications, setting the stage for examining how these efficiency-driven modifications interact with adversarial security threats.

8.2 Adversarial Robustness and Security Vulnerabilities in Efficient Models

8.2 Adversarial Robustness and Security Vulnerabilities in Efficient Models

Building upon the understanding of how compression alters parametric knowledge and induces hallucinations, the discourse on efficient LLM serving must next address the critical dimension of adversarial security. The deployment of Large Language Models (LLMs) on resource-constrained devices necessitates aggressive model compression techniques, primarily pruning and quantization. However, the intersection of model efficiency and security presents a complex landscape where the implications of compression on adversarial robustness are neither linear nor universally predictable. While compression offers significant reductions in memory footprint and latency, it introduces distinct vulnerabilities and, paradoxically, in some scenarios, enhances resilience against specific threat models. Understanding this duality is critical for deploying efficient models in safety-critical applications where adversarial attacks pose existential risks, serving as a necessary precursor to examining the privacy implications of memorization discussed subsequently.

The relationship between model compression and adversarial robustness has been a subject of intense debate, with empirical evidence suggesting contradictory outcomes depending on the compression method, the nature of the attack, and the optimization pipeline employed. Early investigations often assumed that reducing model capacity through pruning or quantization would inherently degrade robustness by removing the redundancy that full-precision models rely on for defense. However, recent studies challenge this assumption, indicating that compression does not necessarily lead to a loss in robustness and can, under certain conditions, preserve or even improve

resistance to adversarial perturbations [376]. For instance, research has shown that pruned models can maintain robustness levels comparable to their uncompressed counterparts when evaluated against standard attack vectors, suggesting that the “sweet spots” in the compression-robustness trade-off space are attainable [377].

Quantization, the process of reducing the precision of weights and activations, exhibits a particularly nuanced interaction with adversarial robustness. The discretization of weights can introduce a phenomenon known as gradient masking, where the non-differentiable nature of quantized operations obscures the true gradient direction, potentially providing a false sense of security against gradient-based attacks [378]. While this obfuscation might hinder white-box attacks, it does not necessarily confer genuine robustness and can be bypassed by more sophisticated attack strategies or transfer-based approaches. Conversely, some studies have observed that quantization can act as a regularizer, smoothing the loss landscape and inadvertently improving robustness against simple gradient-based attacks, although this effect is highly dependent on the attack strength and the specific quantization scheme employed [379]. In the domain of Natural Language Processing, quantization has been found to significantly improve the adversarial accuracy of Transformer-based text classifiers, outperforming traditional adversarial training methods without incurring additional computational overhead during training [181].

To address the inherent vulnerabilities introduced or exposed by compression, researchers have developed frameworks for the joint optimization of efficiency and safety. A prominent approach involves Adversarially Trained Model Compression (ATMC), which formulates the compression process as a unified constrained optimization problem. By integrating pruning, factorization, and quantization directly into the constraints of an adversarial training objective, ATMC achieves a superior trade-off among model size, accuracy, and robustness compared to sequential application of these techniques [380]. Similarly, the concept of “Defensive Quantization” has been proposed to explicitly control the Lipschitz constant of the network during the quantization process. This method ensures that the magnitude of adversarial noise remains non-expansive during inference, effectively mitigating the error amplification effects often observed in vanilla quantization schemes [381].

Furthermore, the interplay between pruning and adversarial training has led to the development of specialized algorithms such as “Blind Adversarial Pruning” (BAP). BAP introduces a cutoff-scale strategy to adaptively estimate non-uniform budgets for adversarial examples during the gradual pruning process, thereby stabilizing the robustness of the pruned model across varying compression ratios and attack strengths [382]. Another innovative direction is the use of second-order information to identify and preserve parameters critical for adversarial robustness while pruning others, as seen in the “Masking Adversarial Damage” (MAD) framework, which effectively compresses adversarially trained networks without sacrificing their defensive capabilities [383].

Despite these advancements, significant security risks remain, particularly concerning the publication of efficient on-device models. The availability of compressed “sibling” models can inadvertently aid adversaries in launching transfer attacks against larger, proprietary commercial models. Research indicates that exploiting published on-device variants as transfer priors can increase the adversarial vulnerability of original large-scale models by up to 100 times, highlighting a severe security implication of model dissemination [384]. This finding underscores the necessity of developing defense mechanisms, such as similarity-unpairing, which fine-tunes on-device models to reduce their structural similarity to the teacher model, thereby hindering transferability.

Moreover, the robustness of compressed models is not uniform across all data distributions. Com-

pression techniques have been shown to disproportionately impact performance on underrepresented features and long-tail data distributions, a phenomenon termed “Compression Identified Exemplars” (CIE). This disparate impact can amplify existing algorithmic biases, making compressed models more susceptible to targeted attacks on minority subgroups [201]. Consequently, evaluating the security of efficient models requires moving beyond average accuracy metrics to include rigorous assessments of fairness and subgroup robustness. This intersection of security and bias provides a natural bridge to the following discussion on privacy, where the risks of data leakage and memorization similarly depend on the granular handling of user data and statistical dependencies.

In conclusion, the path to secure and efficient model deployment requires a holistic approach that transcends isolated compression or defense strategies. The evidence suggests that while compression techniques like pruning and quantization introduce unique vulnerabilities, they also offer opportunities for enhancing robustness when integrated with adversarial training through unified optimization frameworks [385]. Future efforts must focus on developing adaptive compression pipelines that dynamically balance efficiency gains with rigorous security guarantees, ensuring that the drive for lightweight AI does not compromise the trustworthiness of deployed systems, thereby setting the stage for a deeper examination of privacy risks and data leakage in compressed environments.

8.3 Privacy Risks, Memorization, and Data Leakage in Compressed LLMs

8.3 Privacy Risks, Memorization, and Data Leakage in Compressed LLMs

Following the examination of adversarial robustness, where compression techniques exhibited a dual nature of introducing vulnerabilities while occasionally enhancing resilience, the discourse on efficient LLM serving must also address the critical dimension of privacy. The deployment of Large Language Models (LLMs) in resource-constrained environments necessitates aggressive compression techniques such as quantization, pruning, and distillation. While these methods significantly reduce memory footprints and inference latency, they introduce a complex landscape of privacy threats that differ fundamentally from those in full-precision models. The primary concern revolves around the phenomenon of memorization, where models inadvertently retain and leak sensitive training data. Research has established that neural networks possess a high capacity for memorizing unique features and specific training samples, a risk that persists even when standard regularization techniques are applied to prevent overfitting [386]. In the context of compressed LLMs, the interaction between compression artifacts and memorization creates unique vulnerabilities. For instance, while compression is often viewed as a form of regularization that might theoretically reduce overfitting, empirical evidence suggests that heuristic mitigations, excluding perhaps strict L2 regularization, are largely ineffective at preventing the memorization of sensitive user text in predictive models [387].

A critical aspect of privacy in efficient serving is the distinction between training data leakage and inference-time contextual privacy, a nuance that parallels the security challenges discussed previously. Compressed models deployed on edge devices or via API services often process sensitive user inputs in real-time. Recent studies highlight that LLMs frequently fail to maintain contextual integrity, revealing private information in outputs even when prompted to remain confidential [388]. This risk is exacerbated in compressed settings where the model’s ability to discern nuanced privacy boundaries may be degraded due to the loss of precision in parameter representation. Furthermore, the “Onion Effect” of memorization suggests that removing the most vulnerable outlier data points merely exposes a new layer of previously safe points to privacy attacks, implying that simple data sanitization or partial compression strategies are insufficient without rigorous privacy guarantees

[389].

To address these vulnerabilities, Differential Privacy (DP) has emerged as the gold standard for providing formal guarantees against such leaks, yet its integration with compression strategies presents a difficult trade-off similar to the efficiency-robustness tension. The application of DP during training often leads to significant utility degradation, particularly in language generation tasks where the model’s fluency and coherence are paramount [390]. When combining DP with compression, the situation becomes more nuanced. Some research indicates that gradient compression can interact positively with DP mechanisms; specifically, aggressive sparsification or rank reduction of gradients can sometimes mitigate the negative impact of the Gaussian noise added for privacy, thereby improving test accuracy in differentially private training scenarios [391]. However, this synergy is not universal. Other studies warn that certain compression or regularization mechanisms might provide a false sense of security, as they may appear effective against existing Membership Inference Attacks (MIAs) while still leaving the model vulnerable to more sophisticated, adaptive adversaries [392].

The evaluation of privacy risks in compressed models is further complicated by the assumptions made about data distribution, a factor that foreshadows the fairness concerns detailed in the subsequent section. Traditional privacy analyses often assume that training samples are independent and identically distributed (IID). However, in real-world scenarios involving user data, statistical dependencies are common. Investigations have shown that when data dependencies exist, standard DP protections may not provide meaningful privacy, as the privacy parameter effectively scales with the training set size, rendering the model susceptible to MIAs that exploit these correlations [393]. This is particularly relevant for compressed LLMs serving multi-turn conversations or personalized content, where input sequences are highly correlated. Moreover, the reliance on reference data for empirical privacy defenses introduces another layer of risk; if the reference data itself is not properly protected, the trade-off between model utility and privacy for both training and reference datasets becomes skewed, potentially leading to ineffective defense mechanisms [394].

Furthermore, the granularity of privacy protection is crucial in efficient serving, especially when considering the heterogeneous impact on different user groups. Classical DP protects individual records, but in many applications, protecting a user’s entire contribution (user-level DP) or specific elements within a user’s data (element-level DP) is more appropriate [395][396]. Compressed models operating under these stricter constraints face even steeper accuracy-privacy trade-offs. For example, in federated learning settings where communication compression is used alongside DP, the interplay between gradient clipping, compression ratios, and network connectivity creates a delicate balance where convergence rates and privacy guarantees must be jointly optimized [397]. Additionally, side-channel attacks pose a distinct threat to compressed models running on heterogeneous hardware. Timing side-channels, arising from non-constant time operations in optimized kernels, can leak class information and compromise user privacy even if the model itself is trained with DP [398].

Ultimately, ensuring privacy in compressed LLMs requires a holistic approach that goes beyond simple accuracy metrics, mirroring the comprehensive strategies needed for security and fairness. It demands a rigorous re-evaluation of how memorization is measured, how differential privacy budgets are allocated in low-bit regimes, and how contextual privacy is maintained during inference. As evidenced by the failure of many heuristic defenses, only approaches with provable guarantees, such as rigorous DP training, have consistently demonstrated the ability to prevent memorization, despite the associated computational and performance costs [387]. Future research must therefore focus on developing compression-aware privacy mechanisms that can navigate the trilemma of

efficiency, utility, and security without relying on unrealistic assumptions about data independence or adversary capabilities, thereby laying the groundwork for understanding how these efficiency measures differentially impact subgroup robustness and social bias.

8.4 Fairness, Social Bias, and Subgroup Robustness Under Compression

8.4 Fairness, Social Bias, and Subgroup Robustness Under Compression

Building upon the discussion of privacy risks and memorization in Section 8.3, where compression was shown to interact complexly with data leakage and differential privacy, the discourse on efficient LLM serving must now address the critical dimension of algorithmic fairness. The deployment of Large Language Models (LLMs) in high-stakes societal applications necessitates a rigorous examination of how compression techniques—such as quantization, pruning, and knowledge distillation—impact social bias and subgroup robustness. A central tension exists in determining whether compression acts as a regularizing force that mitigates overfitting to biased training data or if it exacerbates existing disparities by disproportionately degrading performance on minority subgroups and long-tail data distributions. This inquiry is essential not only for ethical deployment but also for establishing the valid evaluation frameworks discussed in the subsequent Section 8.5, as aggregate accuracy metrics often mask severe failures in equitable performance.

A fundamental observation in the literature is that compression does not affect all data samples uniformly. Research indicates that while top-line accuracy metrics may remain stable after compression, the performance on specific subsets of data can diverge significantly. This phenomenon is often characterized by the concept of “Compression Identified Exemplars” (CIEs) or “Pruning Identified Exemplars” (PIEs), which represent a small subset of data points—typically atypical, noisy, or underrepresented examples—that are systematically more impacted by the introduction of sparsity or reduced precision [201]. These findings suggest that compression disproportionately impacts model performance on the underrepresented long-tail of the data distribution, thereby amplifying existing algorithmic biases against minority groups [399]. In the context of facial recognition, for instance, quantization has been shown to “decompress race biases,” where the compression process leads to a significant increase in error rates for specific ethnic subgroups, effectively forgetting the nuanced features required for fair recognition across diverse populations [400].

The relationship between compression and bias is further complicated by the nature of the compression technique employed, revealing a dual nature similar to the privacy-utility trade-offs noted earlier. Quantization, which reduces the precision of model weights and activations, can act as a regularizer, potentially reducing social bias by preventing the model from memorizing spurious correlations present in the training data. For example, research on language models has found that quantization showed a regularizer effect, with optimal trade-offs in bias reduction occurring at specific compression levels relative to pretraining time [401]. Similarly, in generative language models, knowledge distillation and pruning have been linked to consistent patterns of toxicity and bias reduction, supporting the hypothesis that limiting model capacity can serve as a mechanism for developing fairer models [402]. Furthermore, moderate bit-range quantization has been found to unexpectedly improve certain trustworthiness dimensions, including ethics and fairness, whereas extreme quantization to very low bit levels tends to significantly reduce trustworthiness [189].

However, this regularization effect is not universal and often depends heavily on the data distribution and the specific subgroup being evaluated. In scenarios involving long-tailed data distributions, where minority classes have significantly fewer samples, compression can severely degrade the model’s ability to generalize to these groups. Studies on subgroup robustness reveal that worst-

group performance does not depend solely on model size but is critically influenced by the compression method used [403]. The interaction between compression and data imbalance is particularly detrimental; when datasets are both long-tailed and noisily labeled, standard compression methods may fail to provide universal improvements, sometimes benefiting majority subpopulations at the direct expense of minority groups [404]. This disparity is echoed in biometrics applications, where lossy image compression algorithms like JPEG have been demonstrated to have a more pronounced negative impact on facial recognition performance for specific racial phenotype categories, such as those with darker skin tones [405].

Moreover, the evaluation of bias in compressed models is often hindered by methodological flaws, a challenge that underscores the need for the rigorous benchmarking frameworks detailed in Section 8.5. The use of template-based bias probes, which explicitly state group membership, can lead to misleading conclusions due to unstated norms in the pre-training data that imply group membership without explicit statements [406]. This suggests that the apparent fairness or bias of a compressed model might be an artifact of the evaluation protocol rather than an intrinsic property of the model itself. Additionally, in contrastive learning settings, compression and representation learning can lead to “representation harm,” where the representations of minority groups collapse into those of majority groups, further entrenching allocation harms in downstream tasks [407].

The challenge is further compounded in distributed learning environments where gradient compression is used to mitigate communication bottlenecks, creating a parallel to the privacy challenges in federated settings discussed previously. Heavy-tailed gradient distributions, common in real-world data, can cause standard quantization schemes to fail, disproportionately affecting the convergence and fairness of the model for underrepresented groups unless specific truncation and non-uniform quantization strategies are employed [408]. Furthermore, the interplay between differential privacy and compression reveals that gradient clipping and noise addition can disproportionately affect underrepresented classes, creating a disparate impact on group utility that requires adaptive clipping strategies to mitigate [409].

In conclusion, the impact of compression on fairness and subgroup robustness is nuanced and context-dependent. While compression can theoretically act as a regularizer to reduce bias, empirical evidence overwhelmingly suggests that without careful design and evaluation, it tends to exacerbate disparities for minority subgroups and long-tail distributions. The identification of Compression Identified Exemplars provides a valuable tool for auditing these models, yet the field must move beyond aggregate accuracy metrics to ensure that efficient serving does not come at the cost of equitable performance. These findings highlight the urgent need for the specialized, integrity-focused benchmarking frameworks described in the following section, which are essential for accurately diagnosing these biases and validating that compressed models meet rigorous safety and fairness standards before deployment.

8.5 Benchmarking Frameworks for Trustworthiness, Safety, and Evaluation Integrity

8.5 Benchmarking Frameworks for Trustworthiness, Safety, and Evaluation Integrity

Building upon the nuanced findings in Section 8.4 regarding how compression disproportionately affects fairness and subgroup robustness, the reliable deployment of efficient Large Language Models (LLMs) demands rigorous evaluation frameworks that can accurately assess trustworthiness under these constrained conditions. As model sizes increase and deployment scenarios diversify, the integrity of the evaluation process itself becomes a paramount concern. Comprehensive benchmarking suites must not only measure performance across diverse safety dimensions but also address sys-

temic vulnerabilities in the evaluation pipeline, including data contamination, lack of standardized threat models, and the nuances of over-defensiveness. Without robust benchmarks that specifically account for the distortions introduced by quantization and pruning, claims regarding the safety of compressed models remain unsubstantiated, potentially leading to the deployment of systems that appear efficient but are inherently fragile or unsafe for minority subgroups.

A foundational challenge in establishing trustworthy benchmarks is the proliferation of fragmented datasets and inconsistent evaluation goals, a problem exacerbated when assessing compressed architectures. The landscape of LLM safety is characterized by a vast array of open datasets, yet many suffer from narrow scopes or lack coverage in non-English languages and specific risk categories. A systematic review of 102 open datasets reveals significant gaps, including a scarcity of resources for assessing long-term catastrophic risks and a heavy reliance on synthetic data, which may not reflect real-world adversarial behaviors [410]. To address this fragmentation, recent efforts have moved towards creating comprehensive, multi-dimensional benchmarks capable of capturing the complex interplay between efficiency and safety. For instance, the AEGIS framework introduces a broad content safety risk taxonomy comprising 13 critical and 9 sparse risk categories, supported by a curated dataset of approximately 26,000 human-LLM interactions [411]. Similarly, the ALERT benchmark employs a fine-grained risk taxonomy with over 45,000 instructions to facilitate red-teaming methodologies, enabling researchers to identify vulnerabilities through adversarial testing scenarios [412]. These suites are particularly essential for compressed models, as compression can inadvertently alter a model’s alignment with safety guardrails and amplify the biases identified in Section 8.4, necessitating evaluation across a wider spectrum of harm than standard perplexity metrics can capture.

However, the validity of any benchmark is contingent upon the integrity of the test data, a concern that is critical when evaluating models prone to “forgetting” or memorization artifacts due to compression. A critical issue undermining current evaluation practices is data contamination, where benchmark samples are implicitly or explicitly included in the model’s training corpus. This phenomenon allows models to “cheat” via memorization rather than demonstrating genuine reasoning or safety alignment, a risk that is heightened when compression techniques like quantization act as regularizers that may preserve spurious correlations while discarding nuanced features. Longitudinal analyses have provided strong evidence of contamination, showing statistically significant correlations between benchmark release dates and model performance, particularly in code and mathematical domains [413]. The opacity of training data in large-scale models exacerbates this issue, making it difficult to audit model capabilities rigorously. Recent studies indicate that contamination levels can range from 1% to 45% across popular benchmarks, with larger models often gaining disproportionate advantages from contaminated test sets [414]. To mitigate this, novel methods such as estimating contamination via output distribution perplexity have been proposed, offering a way to quantify memorization without access to the full training set [415]. Furthermore, frameworks like CDD (Contamination Detection via output Distribution) and TED (Trustworthy Evaluation via output Distribution) have been developed to detect and correct for contamination effects, ensuring that safety evaluations reflect true generalization rather than rote recall [416].

Beyond contamination, effective benchmarking requires standardized threat models that account for the specific vulnerabilities introduced by compression, bridging the gap between the fairness concerns of Section 8.4 and the mitigation strategies discussed in Section 8.6. Compression techniques like pruning and low-rank modifications can disentangle safety-critical regions from utility-relevant ones, creating sparse regions that, if removed, compromise safety without affecting general performance [417]. Consequently, benchmarks must evaluate models against heterogeneous attacks,

including membership inference attacks and adversarial perturbations tailored to low-bit architectures. The SafeCompress framework exemplifies this approach by integrating test-driven development paradigms to simulate attack mechanisms during the compression process, ensuring that safety is co-optimized with performance [418]. Additionally, the phenomenon of “over-defensiveness,” where models refuse benign inputs due to overly aggressive safety tuning—a potential side effect of the regularization effects discussed previously—must be quantified. The SODE benchmark specifically addresses this by evaluating both safety and over-defensiveness, revealing that certain defense strategies, such as self-checking, can drastically increase refusal rates on safe inputs [419].

The development of specialized benchmarks for high-risk domains further underscores the need for targeted evaluation integrity, especially as compressed models are increasingly deployed in resource-constrained, safety-critical environments. In cybersecurity, the WMDP benchmark serves as a proxy for measuring hazardous knowledge in biosecurity and chemical security, while also acting as a testbed for unlearning methods to remove such capabilities [420]. Similarly, CyberMetric and CyberSecEval 2 provide rigorous assessments of LLM knowledge in cybersecurity and their susceptibility to prompt injection and code interpreter abuse, highlighting the safety-utility tradeoff where conditioning models to reject unsafe prompts can lead to false refusals of benign requests [421][422]. For multimodal and physical safety, benchmarks like SC-Safety offer multi-round adversarial interactions to test Chinese LLMs, while others focus on crash scenario generation and surrogate safety measures for autonomous vehicles, emphasizing the need for high-fidelity, dynamic testing environments [423][424][425].

Ultimately, the future of trustworthy LLM serving relies on the convergence of these efforts into holistic evaluation ecosystems that seamlessly connect the diagnosis of bias (Section 8.4) with the implementation of robust defenses (Section 8.6). As noted in recent industry reports, theoretical frameworks are invaluable for organizing risk domains, but they must be coupled with collaborative practices to refine metrics and standards [426]. The integration of contamination-resistant datasets, standardized threat models for compressed architectures, and metrics that balance safety with utility and over-defensiveness forms the bedrock of reliable evaluation. Only through such comprehensive and integrity-focused benchmarking can the community ensure that efficient LLM serving strategies do not come at the cost of societal safety, subgroup fairness, or model reliability.

8.6 Mitigation Strategies and Safe Compression Methodologies

8.6 Mitigation Strategies and Safe Compression Methodologies

Building upon the rigorous evaluation frameworks and integrity challenges outlined in Section 8.5, the deployment of compressed Large Language Models (LLMs) in safety-critical domains necessitates robust mitigation strategies that directly address the vulnerabilities identified during benchmarking. While Section 8.5 highlighted how compression can distort safety alignments, amplify biases, and exacerbate data contamination risks, this subsection focuses on the proactive methodologies designed to restore and enhance trustworthiness within efficient architectures. The research landscape has consequently shifted from viewing compression merely as a resource-saving technique to treating it as a co-optimization problem where safety, robustness, and performance are jointly maximized. This transition is critical for countering the specific threats exposed by benchmarks like ALERT and SafeCompress, ensuring that efficiency gains do not come at the cost of inherited latent defects, catastrophic forgetting, or heightened susceptibility to adversarial attacks.

A primary strategy for hardening compressed models involves integrating adversarial robustness directly into the compression pipeline, addressing the concern raised in Section 8.5 that standard

pruning and quantization may inadvertently disentangle safety-critical regions from utility-relevant ones. Traditional approaches often treat compression and robustness as sequential steps, leading to suboptimal trade-offs where safety guardrails are eroded. However, emerging frameworks demonstrate that adversarial fine-tuning of compressed models can yield robustness levels comparable to fully adversarially trained dense models while maintaining computational efficiency. For instance, research indicates that structured weight pruning and quantization do not inherently degrade robustness; rather, when followed by targeted adversarial fine-tuning, these compressed models can achieve significant improvements in resisting adversarial perturbations [427]. This approach effectively decouples the efficiency gains of compression from the security benefits of adversarial training, allowing practitioners to deploy lightweight models without sacrificing the defense capabilities required to meet the stringent threat models discussed previously. Furthermore, specialized pruning algorithms have been developed to preserve adversarial robustness without requiring the generation of adversarial examples during the pruning phase itself. By leveraging concurrent self-distillation and information bottleneck regularization, methods such as those proposed in [428] can compress previously trained robust networks while maintaining their defensive integrity, thereby significantly improving training efficiency and mitigating the risk of over-defensiveness noted in prior evaluations.

Beyond post-compression fine-tuning, innovative training paradigms inspired by software engineering principles are redefining how safe models are constructed, directly responding to the call in Section 8.5 for test-driven development in AI safety. The concept of “test-driven development” has been adapted into the realm of deep learning through test-driven sparse training frameworks, which simulate attack mechanisms as safety tests during the training process to guide dynamic sparse training toward resilience against specific threat vectors. The [418] framework exemplifies this by introducing SafeCompress, which incorporates simulations of black-box and white-box membership inference attacks directly into the compression objective. This ensures that the resulting sparse model is not only compact but also inherently fortified against the heterogeneous attacks highlighted in recent benchmarking suites, addressing the risk that intrinsic defects in large models might be inherited and exacerbated in their compressed counterparts. Similarly, other works have explored the intersection of sparsity and robustness through principled mathematical formalisms, such as the Feynman-Kac formalism, to co-design sparse architectures that double channel sparsity while simultaneously improving both natural and robust accuracy [429], offering a theoretical grounding for the empirical safety gains observed in red-teaming scenarios.

Another critical avenue for mitigating the reliability losses associated with compression is the utilization of inference-time interventions, particularly dynamic prompting, which addresses the “forgetting” versus “displacement” debate central to evaluation integrity. As noted in Section 8.5, contamination and memorization artifacts can skew performance metrics, raising questions about whether compression causes models to truly lose knowledge or merely displace it within the parameter space. Empirical evidence strongly supports the latter conjecture, suggesting that knowledge is not erased but becomes less accessible via standard decoding paths. To address this, [368] proposes Inference-time Dynamic Prompting (IDP), a technique that increases prompt diversity during generation without incurring additional inference overhead or requiring parameter updates. This method has been shown to outperform classical re-training alternatives like LoRA in recovering post-compression performance, reducing inference latency by up to 60% while saving parameter space by 21 times. By effectively redirecting the model’s inference trajectory, dynamic prompting serves as a low-cost, high-impact mitigation strategy that restores the utility of compressed models in knowledge-intensive tasks, ensuring that efficiency does not compromise the genuine reasoning capabilities that benchmarks strive to measure.

Furthermore, the reliability of compressed systems under hardware faults and non-ideal conditions requires specialized training objectives to counteract the fragility introduced by aggressive optimization. Quantized models, especially those with high activation sparsity, often exhibit sharper minima in their loss landscapes, making them more susceptible to bit-flip and stuck-at faults—a vulnerability that standard benchmarks may overlook without specific stress testing. To counteract this, sharpness-aware quantization (SAQ) training schemes have been introduced to flatten the loss landscape, thereby enhancing fault tolerance without compromising the energy and latency benefits of sparsity [430]. Additionally, comprehensive evaluations have revealed that moderate quantization can sometimes act as a regularizer, unexpectedly improving certain trustworthiness dimensions like fairness and ethics, whereas extreme quantization or aggressive pruning tends to degrade them [189]. This underscores the necessity of tailored mitigation strategies that are sensitive to the specific compression ratio and technique employed, validating the nuanced findings regarding subgroup robustness discussed in Section 8.4 and the evaluation gaps identified in Section 8.5.

Finally, the development of unified frameworks that address multi-perturbation robustness while saving parameters represents a frontier in safe compression, aiming to create the holistic ecosystems envisioned at the conclusion of the previous subsection. Approaches like Parameter-Saving Adversarial Training (PSAT) utilize hypernetworks to train specialized sub-models against different perturbation types, aggregating them to provide comprehensive defense with significantly fewer parameters than maintaining multiple full-sized robust models [431]. Collectively, these mitigation strategies illustrate a paradigm shift where compression is no longer an isolated optimization step but an integral component of a holistic security architecture. By combining adversarial fine-tuning, test-driven sparse training, and inference-time dynamic adaptations, researchers are paving the way for efficient AI systems that are not only fast and small but also robust, reliable, and safe for widespread deployment, effectively bridging the gap between the diagnosis of bias and the implementation of rigorous, integrity-focused defenses.

9 Open Challenges and Future Research Directions

9.1 Bridging the Hardware-Software Co-Design Gap for Neuromorphic Integration

9.1 Bridging the Hardware-Software Co-Design Gap for Neuromorphic Integration

While dynamic adaptation mechanisms address the variability of operational environments, the fundamental efficiency of Large Language Model (LLM) serving is increasingly constrained by the physical limits of conventional von Neumann architectures. The trajectory of LLM scaling has precipitated an unprecedented demand for computational resources, pushing traditional systems to their economic and thermal boundaries. As model parameters grow into the trillions, the inefficiencies inherent in separating memory from processing units have become the primary bottleneck, manifesting as the notorious “memory wall.” In this context, neuromorphic hardware has emerged as a promising paradigm to address these challenges by mimicking the brain’s co-located computation and memory structure. However, a profound disconnect remains between these emerging hardware capabilities and the existing software stacks designed for digital GPUs. Bridging this hardware-software co-design gap is arguably the most critical challenge for the next decade of efficient AI serving, requiring a fundamental rethinking of how algorithms are described, compiled, and executed on non-deterministic, heterogeneous substrates.

The core of this disconnect lies in the divergence between the deterministic, high-precision as-

assumptions of current deep learning frameworks and the stochastic, low-precision, and often analog nature of neuromorphic devices. Emerging technologies such as memristors, atomic switches, and ferroelectric field-effect transistors offer tremendous potential for density and energy efficiency but introduce significant device variability and stochastic behavior that traditional software cannot tolerate without substantial modification [432]. Current LLM serving stacks, optimized for CUDA cores and tensor processing units, assume bit-exact reproducibility and uniform latency, assumptions that crumble when mapped onto neuromorphic fabrics where synaptic weights may drift or exhibit probabilistic switching. To overcome this, the community must move beyond simple porting of existing models and instead adopt a “complexity engineering” approach, where the description of learning algorithms is modified to inherently accommodate the non-idealities of the underlying hardware [432].

A pivotal step toward resolving this fragmentation is the development of unified intermediate representations (IRs) that can abstract away the idiosyncrasies of specific hardware platforms while preserving the computational intent of the model. The lack of a common reference frame has led to a plethora of isolated software tools, each tied to a specific neuromorphic simulator or chip, severely hindering reproducibility and scalability. The proposal of a Neuromorphic Intermediate Representation (NIR) marks a significant advancement in this direction, defining a set of computational primitives as idealized continuous-time hybrid systems that can be mapped across diverse technology stacks [433]. By decoupling the evolution of hardware from software, such unified IRs enable the creation of portable LLM serving pipelines that can dynamically adapt to different neuromorphic backends, whether they are based on spiking neural networks, analog photonic circuits, or mixed-signal CMOS designs. This interoperability is essential for scaling neuromorphic solutions from small-scale prototypes to the massive distributed systems required for trillion-parameter models.

Furthermore, bridging the co-design gap requires addressing the communication bottlenecks that arise in large-scale neuromorphic systems. While neuromorphic architectures excel at local computation, the high fan-out connectivity required by transformer attention mechanisms poses a daunting challenge for on-chip communication networks. Traditional network-on-chip designs are often ill-suited for the flexible, large-fanout connectivity patterns of LLMs, necessitating novel routing architectures and memory technologies that can support efficient data movement [434]. Future co-design efforts must therefore jointly optimize the algorithmic structure of LLMs—potentially through sparse attention mechanisms or hierarchical caching—to align with the physical constraints of neuromorphic interconnects. This includes leveraging 3D integration technologies to vertically stack memory and logic, thereby reducing communication latency and enabling the dense integration required for brain-scale intelligence [435].

The path forward also demands a shift in how reliability and variability are managed within the software stack. Unlike digital systems where errors are rare and catastrophic, neuromorphic systems operate in a regime where variability is intrinsic. Strategies such as population coding, which encode information across clusters of neurons rather than single devices, have shown promise in mitigating the impact of non-uniform devices [436]. Software compilers for LLMs must evolve to automatically incorporate such fault-tolerance mechanisms, dynamically adjusting mapping strategies to minimize the impact of device aging and thermal gradients [437]. Additionally, the integration of cryogenic neuromorphic hardware offers a potential avenue to suppress noise and enhance speed, but this introduces new layers of complexity in system management and thermal control that must be addressed by holistic co-design frameworks [438].

Ultimately, realizing the potential of neuromorphic integration for LLM serving requires a symbi-

otic relationship between algorithm designers and hardware architects. It is insufficient to merely accelerate existing models; we must innovate new architectures that are natively suited to the physics of emerging devices. This involves exploring hybrid stochastic synapses that leverage rather than fight device variability [439] and developing design-technology co-optimization schemes that balance latency, energy, and reliability at the circuit level [440]. Only by establishing unified intermediate representations, embracing complexity engineering, and fostering deep hardware-software collaboration can we transcend the current scalability barriers and unlock a new era of sustainable, ultra-efficient artificial intelligence. The transition from static, digital-centric serving to dynamic, neuromorphic-native inference represents not just an incremental improvement, but a radical reboot of the entire AI infrastructure stack, laying the physical foundation upon which the adaptive mechanisms discussed in the following section will operate.

9.2 Dynamic and Adaptive Inference Mechanisms for Heterogeneous Environments

9.2 Dynamic and Adaptive Inference Mechanisms for Heterogeneous Environments

While Section 9.1 established the critical need for hardware-software co-design to overcome the physical limitations of conventional architectures, the deployment of Large Language Models (LLMs) in real-world scenarios introduces a complementary challenge: the inherent volatility of operational environments. Even with advanced neuromorphic or heterogeneous substrates, traditional serving architectures often rely on static compression techniques, where models are quantized, pruned, or distilled offline to fit a specific hardware profile or latency budget. This static paradigm fails to address the dynamic nature of production conditions, such as fluctuating network bandwidth, varying battery levels on edge devices, sudden spikes in user demand, and shifting data distributions. Consequently, a critical frontier in efficient LLM serving is the transition toward dynamic and adaptive inference mechanisms that leverage machine learning to enable self-reconfiguration, dynamic precision scaling, and closed-loop sensor-AI feedback. This shift moves the field from one-time optimization to continuous, runtime adaptation, ensuring robust Quality of Service (QoS) amidst uncertainty and laying the groundwork for the lossless compression paradigms discussed in Section 9.3.

A primary challenge in heterogeneous environments is managing the vast adaptation space available to a serving system. Modern systems possess numerous configuration options, including varying bit-widths for quantization, early-exit thresholds, and model switching strategies. Exhaustively searching this space at runtime to find an optimal configuration is often computationally infeasible within strict latency constraints. To address this, recent research advocates for reducing the adaptation space using machine learning techniques that can rapidly identify Pareto-optimal configurations without exploring every possibility. For instance, approaches that utilize machine learning to find Pareto-optimal configurations and restrict the search space to make planning tractable have shown promise for highly configurable systems [441]. Furthermore, frameworks like ML2ASR+ demonstrate that configurable machine learning pipelines can effectively reduce adaptation spaces by over 90% for diverse goal types, enabling timely decision-making in self-adaptive systems [282]. Applying these principles to LLM serving implies developing lightweight controllers that can instantly select the appropriate quantization level or draft model complexity based on current system telemetry.

Dynamic precision scaling represents another vital component of adaptive inference, serving as a bridge between rigid static quantization and the flexible requirements of varying workloads. Rather

than adhering to a fixed bit-width, adaptive systems must modulate the precision of weights and activations in real-time to balance accuracy and efficiency. The concept of “slimmable” networks, originally explored in vision tasks, suggests that models can be trained to operate at multiple capacities, allowing for dynamic adjustment of computational cost and latency [442]. Extending this to LLMs involves creating architectures where the bit-width of operations can be switched on the fly. Research into adaptive bit-widths has shown that joint training techniques can produce models capable of instant adaptation to platform constraints, offering a flexible accuracy-efficiency trade-off [443]. Moreover, the sensitivity of model parameters to quantization is not uniform; metrics like FIT, which combine Fisher information with quantization models, can predict layer-wise sensitivity without retraining, facilitating smarter, dynamic mixed-precision allocation [444]. In a serving context, this allows the system to allocate higher precision to critical layers during complex reasoning tasks while aggressively compressing less sensitive components during periods of high load, a strategy that precursors the more granular lossless techniques explored later.

Handling distribution shifts and concept drift is equally crucial for maintaining reliability in heterogeneous environments, particularly as models interact with non-stationary data streams. Data distributions in production rarely remain stationary, leading to performance degradation in static models. Self-adaptive systems must incorporate mechanisms for continual test-time adaptation to mitigate catastrophic forgetting and error accumulation. Techniques such as weight-averaged predictions and stochastic restoration of source weights have been proposed to enable long-term adaptation in non-stationary domains [445]. Additionally, layer-wise auto-weighting algorithms can autonomously identify which layers require focused adaptation versus preservation, significantly reducing the computational overhead of test-time tuning [446]. For LLM serving, this suggests the need for inference engines that can detect domain shifts in user prompts and dynamically adjust their internal representations or switch to specialized adapter modules without human intervention.

The integration of sensor-AI feedback loops further enhances adaptability by closing the gap between physical context and computational strategy, effectively extending the hardware awareness discussed in Section 9.1 into the software control plane. In edge and mobile scenarios, sensors provide real-time data about the environment, user intent, and device state, which should directly influence inference behavior. Frameworks like AdaSpring exemplify this by enabling context-adaptive and runtime-evolutionary compression, where the system searches for the most suitable compression configuration based on the immediate deployment context [447]. Similarly, the concept of “homeostatic” neural networks introduces artificial regulatory features that allow models to self-regulate their learning rates and internal states in response to vulnerability and concept shifts, mimicking biological adaptability [448]. By embedding such feedback loops, LLM serving systems can proactively throttle computation when battery levels drop or increase model complexity when high-stakes decisions are detected.

Finally, the orchestration of these adaptive mechanisms requires robust system-level architectures capable of managing uncertainty. Self-adaptive approaches that leverage multiple models, such as the Machine Learning Model Balancer, facilitate dynamic model switching to ensure consistent QoS under varying uncertainties [449]. Systems like SWITCH provide exemplars for evaluating these dynamic switching strategies, highlighting the importance of real-time observability and logging in adaptive ML-enabled systems [450]. Future research must focus on unifying these disparate adaptive strategies—space reduction, precision scaling, drift handling, and sensor feedback—into cohesive, end-to-end serving frameworks. The goal is to create LLM serving infrastructures that are not merely efficient under ideal conditions but are resilient, self-healing, and continuously optimizing in the face of the inherent unpredictability of heterogeneous deployment environments. These

dynamic capabilities provide the necessary operational flexibility to support the rigorous demands of the lossless compression paradigms detailed in the subsequent section, ensuring that efficiency gains do not come at the expense of model fidelity or system stability.

9.3 Towards Lossless and Ultra-Efficient Compression Paradigms

9.3 Towards Lossless and Ultra-Efficient Compression Paradigms

While Section 9.2 established the necessity of dynamic and adaptive inference mechanisms to navigate heterogeneous environments, the underlying compression strategies supporting these adaptations have predominantly relied on lossy techniques. The prevailing trajectory in efficient Large Language Model (LLM) serving has been dominated by quantization and pruning, which accept a degree of fidelity degradation in exchange for reduced memory footprints and computational costs. Although methods such as post-training quantization (PTQ) and structured pruning have achieved remarkable success in reducing model sizes by factors of 4x to 10x, they inherently introduce approximation errors that can accumulate during autoregressive generation, potentially exacerbating hallucinations or degrading reasoning capabilities in complex tasks. The fundamental limitation of these lossy paradigms lies in their reliance on the assumption that neural networks possess significant redundancy that can be discarded without impacting downstream task performance. However, as models scale and are deployed in safety-critical or high-precision domains—where the adaptive systems described previously must guarantee robust Quality of Service—the tolerance for such stochastic error diminishes. Consequently, future research must pivot towards developing hardware-friendly lossless compression frameworks and novel recurrent architectures that guarantee exact reconstruction of model parameters or activations while simultaneously minimizing resource consumption.

A critical barrier to the adoption of lossless compression in deep learning has historically been the trade-off between compression ratio and decompression latency. Traditional lossless algorithms, such as Huffman coding or arithmetic coding, often introduce significant computational overhead that negates the benefits of reduced memory bandwidth. For instance, while early works like [451] demonstrated that combining pruning, quantization, and Huffman coding could reduce storage requirements by up to 49x without accuracy loss, the decoding process remained a bottleneck for real-time inference on standard hardware. Similarly, [452] proposed lossless storage formats leveraging source coding, yet the practical deployment was hindered by the irregular memory access patterns introduced by sparse representations. Future paradigms must therefore focus on co-designing compression algorithms with hardware accelerators to ensure that decompression can occur at line rate, effectively hiding the latency behind memory fetch operations. Innovations such as [453] represent a step in this direction by proposing lossless off-chip memory compression that exploits non-uniform value distributions to reduce data widths, demonstrating that specialized hardware units can decode variable-length prefixes in parallel to maintain high bandwidth utilization.

Furthermore, the pursuit of ultra-efficient compression necessitates a re-examination of numerical representations beyond standard integer or floating-point quantization. The emergence of techniques that decouple parameters into integer and floating-point components, as seen in [164], hints at a hybrid approach where the bulk of the information is stored in a highly compressed integer format, while a small, sparse residual is kept in higher precision to ensure lossless recovery. This aligns with the theoretical insights from [454], which argues for minimizing the entropy and number of unique parameters in a network through whole-network quantization and clustering, aiming for lossless task performance with significantly fewer unique weights. By treating the compression problem

as an optimization of information entropy rather than mere bit-width reduction, future frameworks can achieve compression ratios approaching the theoretical limits defined by rate-distortion theory, as explored in [455], without sacrificing the deterministic nature of the model’s output.

Another promising avenue for lossless efficiency lies in the architectural shift towards recurrent mechanisms and parameter sharing schemes that inherently reduce the memory footprint without discarding information. The concept of reusing weights across layers or time steps, effectively creating a recurrent structure within feed-forward transformers, can drastically lower the number of unique parameters that need to be stored. This is complementary to the findings in [456], which suggests that many layers in deep networks exhibit high similarity, implying that a recurrent or weight-sharing architecture could replicate the functionality of deep stacks with a fraction of the parameters. If combined with lossless encoding strategies, such as the fixed-to-fixed encoding schemes proposed in [457], these architectures could support fine-grained sparsity patterns while maintaining regular memory structures amenable to hardware acceleration. Additionally, the integration of probabilistic circuits, as discussed in [458], offers a tractable method for exact encoding and decoding with logarithmic complexity, potentially enabling lossless compression of latent representations in generative models.

Finally, the future of ultra-efficient serving will likely involve a synthesis of lossless memory compression and the dynamic, adaptive inference strategies outlined in Section 9.2. While current systems rely on static quantization levels, next-generation frameworks should employ runtime adaptive mechanisms that switch between lossy and lossless modes based on the complexity of the input or the criticality of the token being generated. This requires robust analytical models, similar to the ratio-quality modeling presented in [459], but extended to predict the necessity of full-precision recovery. Moreover, the development of streaming lossless compression frameworks, exemplified by [460], demonstrates that recurrent neural networks can be optimized for hardware-friendly, real-time lossless compression, a capability that is directly transferable to the sequential nature of LLM decoding. By bridging the gap between theoretical information limits and practical hardware constraints, the community can move towards a new era of LLM serving where models are deployed with zero fidelity loss, maximum memory efficiency, and deterministic reliability. This evolution provides the foundational efficiency required to address the broader sustainability challenges discussed in Section 9.4, ensuring that the drive for “Green AI” does not come at the cost of model integrity.

9.4 Sustainability Metrics and Green AI Evaluation Frameworks

9.4 Sustainability Metrics and Green AI Evaluation Frameworks

Building upon the vision of lossless and ultra-efficient compression paradigms outlined in Section 9.3, which promises to minimize resource consumption without sacrificing model fidelity, it becomes imperative to establish rigorous mechanisms for evaluating the ecological impact of these advancements. The rapid proliferation of Large Language Models (LLMs) has precipitated a critical paradigm shift in how the artificial intelligence community approaches model evaluation. Historically, the primary metrics for assessing LLM performance have been confined to accuracy, perplexity, and inference latency. However, as models scale to billions and trillions of parameters, the environmental footprint associated with their training and deployment has become untenable. The current evaluation landscape often overlooks the substantial energy consumption and carbon emissions inherent in serving these massive systems, even when employing the advanced compression strategies discussed previously. Consequently, there is an urgent need to transition from a purely

performance-centric view to a holistic framework that integrates comprehensive environmental metrics. This subsection argues for the establishment of standardized “Green AI” evaluation protocols that treat ecological impact as a first-class citizen alongside traditional capability benchmarks, providing the necessary measurement infrastructure to validate the efficiency gains proposed in prior sections.

The necessity for this shift is underscored by the growing disparity between the computational resources required for state-of-the-art models and the accessibility of such resources for the broader research community. Surveys of the NLP community have revealed severe concerns regarding the sustainability, reproducibility, and inclusiveness of research driven by compute-hungry models [461]. When evaluation leaderboards prioritize raw performance without accounting for the energy cost, they inadvertently incentivize the development of inefficient architectures that exacerbate environmental degradation, potentially negating the benefits of the lossless and adaptive techniques explored in Section 9.3. This dynamic creates a barrier to entry for researchers with limited computational budgets and fosters an ecosystem where the most “successful” models are often the most ecologically damaging. To counteract this, future evaluation frameworks must adopt a multi-objective approach that penalizes excessive energy usage, much like the “Efficiency Pentathlon” proposes by incorporating metrics for latency, throughput, memory overhead, and crucially, energy consumption into a standardized arena [462].

A robust Green AI evaluation framework must move beyond abstract estimates of FLOPs and instead measure actual power consumption and carbon intensity across the entire model lifecycle, including the inference phase which dominates the operational cost of deployed LLMs. Recent studies have demonstrated that simpler algorithms can often achieve performance levels comparable to large LLMs while consuming significantly less power, suggesting that the race for marginal accuracy gains is frequently not justified by the exponential increase in energy demand [463]. Therefore, new metrics should quantify the “carbon footprint” per token generated or per query answered, enabling practitioners to make informed trade-offs between model capability and environmental impact. The concept of “utility” in model selection must be redefined; as argued in critiques of current leaderboards, a highly inefficient model provides diminished utility to practitioners who bear the operational costs, even if it tops a performance chart [464]. This redefinition is particularly critical when assessing the trade-offs inherent in the approximate computing techniques that will be scrutinized for safety in the subsequent Section 9.5.

Furthermore, the intersection of fairness and environmental sustainability presents a complex challenge that future metrics must address. There is emerging evidence that techniques employed to reduce energy consumption, such as knowledge distillation, may inadvertently exacerbate social biases within models, while conversely, some fairness mitigation strategies can increase computational overhead [465]. A comprehensive evaluation framework must therefore assess not only the carbon efficiency of a model but also its ethical alignment, ensuring that the pursuit of “Green AI” does not come at the expense of equity. This aligns with broader calls to evaluate LLMs not just on their technical prowess but on their alignment with Sustainable Development Goals (SDGs), recognizing that the attitudes and outputs of these models have profound societal implications [466]. As we consider the reliability challenges of compressed models in Section 9.5, it becomes clear that sustainability cannot be viewed in isolation from trustworthiness; a model that is energy-efficient but biased or unsafe fails the broader criteria of responsible AI.

To operationalize these concepts, the community requires standardized testing protocols and transparent reporting mechanisms. Current practices often lack the rigor to compare the environmental impact of different models due to heterogeneous hardware setups and opaque optimization tech-

niques. Proposed methodologies for estimating “idealized runtime” aim to normalize these comparisons, allowing for a fair assessment of inference efficiency across different providers and hardware stacks [467]. Building upon this, future benchmarks should mandate the reporting of energy consumption data alongside accuracy scores, similar to how “QualEval” augments quantitative metrics with qualitative insights to drive model improvement [468]. By integrating environmental costs into the core evaluation loop, developers can be guided toward architectures that are not only powerful but also sustainable, creating a feedback loop that drives the adoption of the lossless and adaptive methods described in Section 9.3.

Moreover, the evaluation of compressed and efficient models requires specialized metrics that capture the nuance of degradation in relation to resource savings. Traditional metrics like perplexity often fail to reflect the true quality of text generation in compressed models, necessitating novel approaches like Divergent Token Metrics which can better assess the impact of pruning and quantization [469]. Extending this logic to sustainability, new metrics should quantify the “efficiency-accuracy frontier,” helping users identify the optimal operating point where marginal gains in performance do not justify disproportionate increases in energy consumption. This is particularly relevant for adaptive systems that dynamically adjust model size or precision based on task complexity, a direction supported by frameworks like SMART which aim to minimize inference costs while maintaining accuracy guarantees [470]. These metrics serve as the essential bridge between the theoretical efficiency gains of Section 9.3 and the practical safety constraints discussed in Section 9.5, ensuring that efficiency improvements are measurable and meaningful.

Ultimately, the path toward sustainable LLM serving requires a cultural and methodological transformation within the AI research community. It demands a move away from the singular pursuit of scaling laws toward a more balanced optimization landscape that values energy efficiency, carbon neutrality, and long-term ecological viability. As the field matures, the definition of a “state-of-the-art” model must evolve to include its environmental stewardship. By adopting comprehensive environmental metrics and standardized green evaluation frameworks, the community can ensure that the advancement of artificial intelligence contributes positively to global sustainability goals rather than accelerating climate change. This shift is not merely an academic exercise but a moral imperative to ensure that the benefits of LLMs are realized without compromising the planet’s future [471]. Furthermore, establishing these rigorous evaluation standards provides the foundational context necessary to address the critical paradox of reliability in approximate computing, setting the stage for the detailed discussion on safety and trustworthiness in the following section.

9.5 Reliability, Safety, and Trustworthiness in Approximate Computing

9.5 Reliability, Safety, and Trustworthiness in Approximate Computing

Building upon the imperative for “Green AI” evaluation frameworks established in the previous subsection, the deployment of Large Language Models (LLMs) in resource-constrained environments increasingly relies on approximate computing techniques, including aggressive quantization, pruning, and neuromorphic acceleration. While these methods are essential for reducing the carbon footprint and memory overhead discussed earlier, they introduce a critical paradox: the very mechanisms designed to enhance efficiency and sustainability often degrade the model’s robustness, safety, and trustworthiness. As models are compressed into lower precision regimes or mapped onto noisy hardware substrates, the probability of generating non-factual content, known as hal-

lucinations, tends to increase. This subsection addresses the overarching challenge of maintaining high reliability standards in approximate computing environments, arguing that future research must move beyond simple accuracy metrics to develop holistic frameworks for uncertainty quantification and hallucination mitigation tailored to compressed architectures. Without such safeguards, the pursuit of energy efficiency risks compromising the integrity of AI systems, rendering them unsuitable for high-stakes applications despite their ecological benefits.

A primary concern in approximate computing is the amplification of hallucination risks. Compression techniques, particularly ultra-low-bit quantization and structured pruning, can distort the internal representation space of LLMs, leading to a loss of parametric knowledge and an increased propensity for fabricating information. Research has shown that hallucinations are not merely random errors but often stem from specific mechanistic failures, such as insufficient attribute knowledge in lower layers or incorrect attention selection in upper layers [369]. When models are quantized, these vulnerabilities are exacerbated because the coarse discretization of weights and activations can blur the fine-grained distinctions required for factual consistency. Consequently, there is an urgent need to understand how compression artifacts interact with the generative processes that lead to hallucinations. For instance, studies indicate that higher predictive uncertainty often correlates with a higher chance of hallucination, suggesting that uncertainty estimation could serve as a vital proxy for detecting reliability issues in compressed models [472].

To mitigate these risks, future research must prioritize the development of advanced Uncertainty Quantification (UQ) methods that are robust to the noise introduced by approximate computing. Traditional UQ techniques often assume precise arithmetic, which fails to hold in quantized or neuromorphic settings where hardware non-idealities contribute to prediction variance. Effective UQ in this domain requires distinguishing between aleatoric uncertainty (inherent data noise) and epistemic uncertainty (model ignorance), with the latter being particularly sensitive to compression-induced information loss. Recent advances propose leveraging internal states and eigenvalue analysis of response covariance matrices to detect semantic inconsistencies, offering a pathway to identify hallucinations even when output probabilities appear confident [372]. Furthermore, token-level uncertainty metrics, such as Claim Conditioned Probability, have demonstrated efficacy in isolating specific factual errors, a capability that is essential for verifying the integrity of generated text in safety-critical applications like healthcare and law [473].

Another promising direction involves the integration of adaptive inference mechanisms that dynamically adjust computation based on real-time uncertainty estimates. In approximate computing scenarios, a static compression ratio may be insufficient for handling complex queries that require high fidelity. Systems capable of “introspective planning” can identify high-uncertainty situations and proactively seek clarification or trigger a fallback to a higher-precision mode, thereby balancing efficiency with safety [474]. This aligns with the concept of abstention, where models refrain from answering when uncertainty exceeds a certain threshold, significantly reducing the rate of hallucinations and improving overall system safety [475]. Future architectures should embed such decision-making capabilities directly into the inference pipeline, allowing compressed models to recognize their own limitations and dynamically trade off the energy savings emphasized in Green AI metrics against the need for factual correctness.

Moreover, the relationship between compression and adversarial robustness remains an underexplored frontier. Compressed models may exhibit different vulnerability profiles compared to their full-precision counterparts, potentially becoming more susceptible to adversarial attacks that exploit quantization boundaries. Ensuring trustworthiness requires rigorous benchmarking that evaluates not only standard performance metrics but also resilience against distribution shifts and

adversarial perturbations. Benchmarks like Med-HALT and HypoTermQA provide foundational frameworks for assessing hallucination tendencies in specific domains, yet there is a lack of standardized evaluation suites specifically designed for quantized and neuromorphic models [476; 477]. Developing such benchmarks is crucial for establishing safety guarantees before deploying efficient models in high-stakes environments, ensuring that the drive for sustainability does not inadvertently create fragile systems.

Finally, the path toward trustworthy approximate computing involves co-designing algorithms and hardware to preserve semantic integrity. Techniques such as counterfactual inception and self-reflection loops can be adapted to operate efficiently within low-precision constraints, offering post-hoc correction mechanisms that do not require retraining [478; 479]. Additionally, leveraging retrieval-augmented generation can compensate for the knowledge loss inherent in compression, provided that the retrieval mechanism itself is optimized for low-latency, low-power operation [480]. By synthesizing these approaches—robust uncertainty estimation, adaptive computation, specialized benchmarking, and algorithmic-hardware co-design—the research community can pave the way for efficient LLM serving systems that are not only fast and cheap but also reliable, safe, and trustworthy. This focus on reliability serves as a critical bridge to the next generation of physical computing substrates; as we transition toward wafer-scale and cryogenic neuromorphic architectures, ensuring that these novel, highly efficient hardware platforms maintain semantic fidelity will be paramount to their successful adoption.

9.6 Next-Generation Architectures: Wafer-Scale, Cryogenic, and Embodied AI

9.6 Next-Generation Architectures: Wafer-Scale, Cryogenic, and Embodied AI

Building on the imperative to preserve semantic fidelity in compressed models discussed previously, the pursuit of trustworthy approximate computing inevitably leads to a re-examination of the underlying physical substrates. As the demand for computational power escalates with the proliferation of large-scale language models and complex generative systems, traditional hardware scaling approaches are encountering fundamental physical and economic barriers. The termination of Moore’s Law and the stagnation of Dennard scaling have created a critical bottleneck where the growth of hardware computing power fails to keep pace with the exponential increase in algorithmic complexity. To overcome these limitations—and to provide the robust foundation required for the reliable, low-precision inference outlined in Section 9.5—the research community is increasingly turning toward frontier physical computing architectures. Among the most promising directions are wafer-scale integration, cryogenic neuromorphic systems, and embodied AI designs, each offering unique mechanisms to shatter the current memory wall while maintaining the energy efficiency and semantic integrity essential for next-generation intelligent systems.

Wafer-scale computing represents a paradigm shift from traditional multi-chip module assemblies to monolithic integration across entire silicon wafers. This approach directly addresses the severe communication bandwidth and latency constraints inherent in inter-chip connections, which are often exacerbated in distributed approximate computing setups. By providing interconnects with chip-level density and bandwidth across a vastly larger surface area, wafer-scale architectures offer remarkable advantages in programmability and integration density. Unlike conventional high-performance computing paradigms that rely on datacenter-scale clustering, wafer-scale systems eliminate the off-chip communication bottleneck, supporting the massive parallelism required for training and serving trillion-parameter models. However, realizing this vision requires overcoming unprecedented design challenges in hardware architecture, power distribution, and thermal

management. Recent comprehensive reviews highlight that while advanced packaging technologies allow for the integration of multiple chiplets, true wafer-scale computing necessitates a holistic co-optimization of design, system, and technology to manage the complexities of yield, cooling, and compiler toolchains [481]. The result is a single, unified computational fabric that not only mimics the connectivity scale of biological neural networks but also provides the stable, high-bandwidth environment necessary to mitigate the uncertainty propagation risks identified in compressed models.

Parallel to the push for larger integration areas is the exploration of cryogenic environments to achieve ultra-low power consumption and high-speed operation, a critical enabler for sustainable “Green AI” without compromising reliability. Neuromorphic computing, which draws inspiration from the brain’s efficient information processing, has long sought to match the energy efficiency of biological systems. While conventional neuromorphic hardware has made strides, it still falls short of the human brain’s efficiency due to design complexity and process variations that can amplify the noise inherent in approximate computing. Operating neuromorphic systems at cryogenic temperatures has emerged as a potent solution, leveraging the superior speed and power metrics of superconducting devices to minimize thermal noise and enhance signal fidelity. Cryogenic neuromorphic hardware utilizes devices engineered to function as neuromorphic primitives with ultra-low power demands, potentially circumventing the energy inefficiencies of room-temperature semiconductor platforms [438]. A key enabler in this domain is Superconducting Single Flux Quantum (SFQ) logic, where data is represented by quantized magnetic flux pulses. SFQ circuits operate at clock frequencies ranging from tens to hundreds of gigahertz, making them naturally suited for processing high-frequency pulse trains typical in spiking neural networks. Furthermore, deep neuromorphic networks based on SFQ technology have demonstrated tunability and scalability without relying on unconventional superconductive devices or semiconductor transistors, offering a viable path toward scalable cryogenic computing that maintains the precision required for safety-critical applications [482].

The synergy between cryogenics and optical technologies further expands the horizon for next-generation architectures, offering new pathways to balance efficiency with the strict reliability demands of approximate computing. Large-scale spiking neuromorphic systems aiming for brain-level complexity require co-optimized communication and computation to prevent the degradation of factual consistency. Optoelectronic neuromorphic platforms propose leveraging integrated photonics for communication alongside analog electronics for computation. Two distinct paths are being pursued: semiconductor-based platforms utilizing analog CMOS and waveguide-integrated photodiodes, and superconducting approaches employing Josephson junctions and superconducting single-photon detectors. While semiconductor systems benefit from a mature fabrication ecosystem, superconducting optoelectronic systems offer near-theoretically minimum burdens on light sources and new opportunities for high-endurance synaptic memory, despite challenges in interfacing low-voltage circuits and managing cryogenic infrastructure [483]. Additionally, the development of compatible cryogenic memory technologies remains a critical frontier. The lack of suitable memory operating at 4 Kelvin or lower has been a major roadblock for scalable quantum computers and practical SFQ circuits, potentially introducing latency that undermines real-time uncertainty quantification. Current research categorizes cryogenic memory platforms into superconducting, non-superconducting, and hybrid types, scrutinizing their challenges and future prospects to ensure the viability of these ultra-fast, energy-efficient classical computing systems [484].

Beyond the specifics of integration scale and operating temperature, the concept of embodied AI within these new substrates suggests a move toward systems that are not only computationally pow-

erful but also physically adaptive and resilient, capable of dynamic resource allocation as suggested by adaptive inference mechanisms. The integration of 3D technologies plays a pivotal role here, offering strategic advantages for cost-effective and flexible design. 3D integration enables the dense co-location of memory and processing elements, a core principle of neuromorphic computing that removes the von Neumann bottleneck and reduces the data movement errors that can contribute to hallucinations in compressed models. By stacking layers of logic and memory, 3D integrated system designs can incorporate advanced capabilities and provide the design flexibility necessary for future neuromorphic chips [435]. This vertical integration is essential for creating compact, high-density systems that can emulate the brain’s structural hierarchy, potentially leading to “artificial brain” prototypes with general self-learning capabilities that inherently integrate safety and uncertainty monitoring [485].

Furthermore, the transition to these next-generation architectures necessitates a rethinking of design methodologies to ensure that the novel physical properties of these substrates do not introduce new vectors for unreliability. Traditional design flows may be insufficient for handling the variability, stochasticity, and complexity of emerging devices like memristors and atomic switches in wafer-scale or cryogenic contexts, where small physical deviations could significantly impact the uncertainty estimates crucial for trustworthiness. A complexity engineering approach is proposed to reboot neuromorphic hardware design, analyzing past successes and identifying solutions that modify both the descriptions of learning and the design methodologies employed [432]. As these physical substrates mature, they promise to break the current memory walls, enabling large-scale intelligent systems that are not only faster and more efficient but also capable of adaptive, brain-like computation with built-in mechanisms for maintaining semantic fidelity. The convergence of wafer-scale integration, cryogenic operation, and 3D embodied designs marks the dawn of a new era in AI hardware, one where physical constraints are transformed into opportunities for unprecedented computational capability, finally aligning the goals of extreme efficiency with the rigorous demands of safety and trustworthiness.

References

- [1] AI and Memory Wall
- [2] S³ Increasing GPU Utilization during Generative Inference for Higher Throughput
- [3] Hydragen High-Throughput LLM Inference with Shared Prefixes
- [4] Efficient Memory Management for Large Language Model Serving with PagedAttention
- [5] H₂O Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models
- [6] Scissorhands Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time
- [7] KIVI A Tuning-Free Asymmetric 2bit Quantization for KV Cache
- [8] QAA Quality Adaptive Quantization for LLM KV Cache
- [9] FastDecode High-Throughput GPU-Efficient LLM Serving using Heterogeneous Pipelines
- [10] GEAR An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM
- [11] SubGen Token Generation in Sublinear Time and Memory

- [12] Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve
- [13] Fast Distributed Inference Serving for Large Language Models
- [14] DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving
- [15] Mélange Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity
- [16] POLCA Power Oversubscription in LLM Cloud Providers
- [17] SLO beyond the Hardware Isolation Limits
- [18] Towards Greener LLMs Bringing Energy-Efficiency to the Forefront of LLM Inference
- [19] LLM Inference Unveiled Survey and Roofline Model Insights
- [20] Time-Based Roofline for Deep Learning Performance Analysis
- [21] SqueezeLLM Dense-and-Sparse Quantization
- [22] AWQ Activation-aware Weight Quantization for LLM Compression and Acceleration
- [23] Ridgeline A 2D Roofline Model for Distributed Systems
- [24] The Framework Tax Disparities Between Inference Efficiency in NLP Research and Deployment
- [25] FlashDecoding++ Faster Large Language Model Inference on GPUs
- [26] FLAT An Optimized Dataflow for Mitigating Attention Bottlenecks
- [27] ZeroQuant-FP A Leap Forward in LLMs Post-Training W4A8 Quantization Using Floating-Point Formats
- [28] Nimble Lightweight and Parallel GPU Task Scheduling for Deep Learning
- [29] Does Form Follow Function An Empirical Exploration of the Impact of Deep Neural Network Architecture Design on Hardware-Specific Acceleration
- [30] DYNAMAP Dynamic Algorithm Mapping Framework for Low Latency CNN Inference
- [31] Characterizing Deep-Learning I O Workloads in TensorFlow
- [32] Memory Optimization for Deep Networks
- [33] Automating Generation of Low Precision Deep Learning Operators
- [34] A Full-Stack Search Technique for Domain Optimized Deep Learning Accelerators
- [35] TVM An Automated End-to-End Optimizing Compiler for Deep Learning
- [36] Dynamic DNNs and Runtime Management for Efficient Inference on Mobile Embedded Devices
- [37] No Token Left Behind Reliable KV Cache Compression via Importance-Aware Mixed Precision Quantization
- [38] TriForce Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding
- [39] Leveraging Speculative Sampling and KV-Cache Optimizations Together for Generative AI using OpenVINO

- [40] QuaRot Outlier-Free 4-Bit Inference in Rotated LLMs
- [41] SqueezeAttention 2D Management of KV-Cache in LLM Inference via Layer-wise Optimal Budget
- [42] GMLake Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching
- [43] Characterizing Physical Memory Fragmentation
- [44] TPP Transparent Page Placement for CXL-Enabled Tiered-Memory
- [45] MATRYOSHKA Non-Exclusive Memory Tiering via Transactional Page Migration
- [46] Supporting Superpages and Lightweight Page Migration in Hybrid Memory Systems
- [47] Page Tables Keeping them Flat and Hot (Cached)
- [48] DaeMon Architectural Support for Efficient Data Movement in Disaggregated Systems
- [49] An Intelligent Framework for Oversubscription Management in CPU-GPU Unified Memory
- [50] Keyformer KV Cache Reduction through Key Tokens Selection for Efficient Generative Inference
- [51] Sequence can Secretly Tell You What to Discard
- [52] On the Efficacy of Eviction Policy for Key-Value Constrained Generative Language Model Inference
- [53] SnapKV LLM Knows What You are Looking for Before Generation
- [54] SparQ Attention Bandwidth-Efficient LLM Inference
- [55] SpAtten Efficient Sparse Attention Architecture with Cascade Token and Head Pruning
- [56] Sparsifiner Learning Sparse Instance-Dependent Attention for Efficient Vision Transformers
- [57] Less is More Focus Attention for Efficient DETR
- [58] Adaptively Sparse Transformers
- [59] SCRAM Spatially Coherent Randomized Attention Maps
- [60] Deja Vu Contextual Sparsity for Efficient LLMs at Inference Time
- [61] WKVQuant Quantizing Weight and Key Value Cache for Large Language Models Gains More
- [62] StreamingDialogue Prolonged Dialogue Learning via Long Context Compression with Minimal Losses
- [63] LoMA Lossless Compressed Memory Attention
- [64] Dynamic Memory Compression Retrofitting LLMs for Accelerated Inference
- [65] Learning Low-Rank Representations for Model Compression
- [66] Vcc Scaling Transformers to 128K Tokens or More by Prioritizing Important Tokens
- [67] AttentionStore Cost-effective Attention Reuse across Multi-turn Conversations in Large Language Model Serving

- [68] Prompt Cache Modular Attention Reuse for Low-Latency Inference
- [69] ChunkAttention Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition
- [70] Stateful Large Language Model Serving with Pensieve
- [71] RelayAttention for Efficient Large Language Model Serving with Long System Prompts
- [72] Fast Multipole Attention A Divide-and-Conquer Attention Mechanism for Long Sequences
- [73] Communication Complexity of the Fast Multipole Method and its Algebraic Variants
- [74] Treeformer Dense Gradient Trees for Efficient Attention Computation
- [75] H-Transformer-1D Fast One-Dimensional Hierarchical Attention for Sequences
- [76] QuadTree Attention for Vision Transformers
- [77] Fovea Transformer Efficient Long-Context Modeling with Structured Fine-to-Coarse Attention
- [78] Combiner Full Attention Transformer with Sparse Computation Cost
- [79] ChunkFormer Learning Long Time Series with Multi-stage Chunked Transformer
- [80] FlashAttention-2 Faster Attention with Better Parallelism and Work Partitioning
- [81] A Case Study in CUDA Kernel Fusion Implementing FlashAttention-2 on NVIDIA Hopper Architecture using the CUTLASS Library
- [82] An Irredundant and Compressed Data Layout to Optimize Bandwidth Utilization of FPGA Accelerators
- [83] Increasing FPGA Accelerators Memory Bandwidth with a Burst-Friendly Memory Layout
- [84] METRO A Software-Hardware Co-Design of Interconnections for Spatial DNN Accelerators
- [85] Heterogeneous Data-Centric Architectures for Modern Data-Intensive Applications Case Studies in Machine Learning and Databases
- [86] Enabling the Adoption of Processing-in-Memory Challenges, Mechanisms, Future Research Directions
- [87] Cocco Hardware-Mapping Co-Exploration towards Memory Capacity-Communication Optimization
- [88] SMOF Streaming Modern CNNs on FPGAs with Smart Off-Chip Eviction
- [89] Beyond the Speculative Game A Survey of Speculative Execution in Large Language Models
- [90] Accelerating Large Language Model Decoding with Speculative Sampling
- [91] Decoding Speculative Decoding
- [92] DistillSpec Improving Speculative Decoding via Knowledge Distillation
- [93] Online Speculative Decoding
- [94] Medusa Simple LLM Inference Acceleration Framework with Multiple Decoding Heads
- [95] Hydra Sequentially-Dependent Draft Heads for Medusa Decoding
- [96] PaSS Parallel Speculative Sampling

- [97] The Synergy of Speculative Decoding and Batching in Serving Large Language Models
- [98] ProPD Dynamic Token Tree Pruning and Generation for LLM Parallel Decoding
- [99] Recursive Speculative Decoding Accelerating LLM Inference via Sampling Without Replacement
- [100] SpecTr Fast Speculative Decoding via Optimal Transport
- [101] Multi-Candidate Speculative Decoding
- [102] SDSAT Accelerating LLM Inference through Speculative Decoding with Semantic Adaptive Tokens
- [103] Ouroboros Speculative Decoding with Large Model Enhanced Drafting
- [104] Blockwise Parallel Decoding for Deep Autoregressive Models
- [105] Towards Fast Inference Exploring and Improving Blockwise Parallel Drafts
- [106] Lossless Acceleration of Large Language Model via Adaptive N-gram Parallel Decoding
- [107] Parallel Decoding via Hidden Transfer for Lossless Large Language Model Acceleration
- [108] Recurrent Drafter for Fast Speculative Decoding in Large Language Models
- [109] Tandem Transformers for Inference Efficient LLMs
- [110] Fast and Robust Early-Exiting Framework for Autoregressive Language Models with Synchronized Parallel Decoding
- [111] EE-LLM Large-Scale Training and Inference of Early-Exit Large Language Models with 3D Parallelism
- [112] Confident Adaptive Language Modeling
- [113] You Need Multiple Exiting Dynamic Early Exiting for Accelerating Unified Vision Language Model
- [114] Flover A Temporal Fusion Framework for Efficient Autoregressive Model Parallel Inference
- [115] Parallelizing Workload Execution in Embedded and High-Performance Heterogeneous Systems
- [116] Improving tasks throughput on accelerators using OpenCL command concurrency
- [117] Towards Enabling I O Awareness in Task-based Programming Models
- [118] Effective GPU Sharing Under Compiler Guidance
- [119] LLM can Achieve Self-Regulation via Hyperparameter Aware Generation
- [120] Self-Evaluation Improves Selective Generation in Large Language Models
- [121] Learning to Check Unleashing Potentials for Self-Correction in Large Language Models
- [122] Let's Sample Step by Step Adaptive-Consistency for Efficient Reasoning and Coding with LLMs
- [123] Escape Sky-high Cost Early-stopping Self-Consistency for Multi-step Reasoning
- [124] Perils of Self-Feedback Self-Bias Amplifies in Large Language Models

- [125] Self-Contrast Better Reflection Through Inconsistent Solving Perspectives
- [126] V-STaR Training Verifiers for Self-Taught Reasoners
- [127] COMQ A Backpropagation-Free Algorithm for Post-Training Quantization
- [128] QuantEase Optimization-based Quantization for Language Models
- [129] PD-Quant Post-Training Quantization based on Prediction Difference Metric
- [130] Improving Post-Training Quantization on Object Detection with Task Loss-Guided Lp Metric
- [131] UWC Unit-wise Calibration Towards Rapid Network Compression
- [132] FlexRound Learnable Rounding based on Element-wise Division for Post-Training Quantization
- [133] Solving Oscillation Problem in Post-Training Quantization Through a Theoretical Perspective
- [134] QFT Post-training quantization via fast joint finetuning of all degrees of freedom
- [135] Instance-Aware Group Quantization for Vision Transformers
- [136] CPT-V A Contrastive Approach to Post-Training Quantization of Vision Transformers
- [137] Outlier Suppression+ Accurate quantization of large language models by equivalent and optimal shifting and scaling
- [138] Outlier Suppression Pushing the Limit of Low-bit Transformer Language Models
- [139] QLLM Accurate and Efficient Low-Bitwidth Quantization for Large Language Models
- [140] RPTQ Reorder-based Post-training Quantization for Large Language Models
- [141] Outlier Dimensions Encode Task-Specific Knowledge
- [142] OWQ Outlier-Aware Weight Quantization for Efficient Fine-Tuning and Inference of Large Language Models
- [143] Rethinking Channel Dimensions to Isolate Outliers for Low-bit Weight Quantization of Large Language Models
- [144] OverQ Opportunistic Outlier Quantization for Neural Network Accelerators
- [145] OmniQuant Omnidirectionally Calibrated Quantization for Large Language Models
- [146] EdgeQAT Entropy and Distribution Guided Quantization-Aware Training for the Acceleration of Lightweight LLMs on the Edge
- [147] QLoRA Efficient Finetuning of Quantized LLMs
- [148] LoRA-Enhanced Distillation on Guided Diffusion Models
- [149] Accurate LoRA-Finetuning Quantization of LLMs via Information Retention
- [150] Oh! We Freeze Improving Quantized Knowledge Distillation via Signal Propagation Analysis for Large Language Models
- [151] Overcoming Oscillations in Quantization-Aware Training
- [152] Reducing the Side-Effects of Oscillations in Training of Quantized YOLO Networks

- [153] Mitigating the Impact of Outlier Channels for Language Model Quantization with Activation Regularization
- [154] R2 Loss Range Restriction Loss for Model Compression and Quantization
- [155] BiTAT Neural Network Binarization with Task-dependent Aggregated Transformation
- [156] EfficientDM Efficient Quantization-Aware Fine-Tuning of Low-Bit Diffusion Models
- [157] Self-Supervised Quantization-Aware Knowledge Distillation
- [158] Extreme Compression of Large Language Models via Additive Quantization
- [159] QuIP Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks
- [160] Post-Training Quantization with Low-precision Minifloats and Integers on FPGAs
- [161] Integer or Floating Point New Outlooks for Low-Bit Quantization on Large Language Models
- [162] AdaptivFloat A Floating-point based Data Type for Resilient Deep Learning Inference
- [163] A Transprecision Floating-Point Platform for Ultra-Low Power Computing
- [164] decoupleQ Towards 2-bit Post-Training Uniform Quantization via decoupling Parameters into Integer and Floating Points
- [165] Approaching Rate-Distortion Limits in Neural Compression with Lattice Transform Coding
- [166] Accurate Block Quantization in LLMs with Outliers
- [167] IntactKV Improving Large Language Model Quantization by Keeping Pivot Tokens Intact
- [168] SDQ Stochastic Differentiable Quantization with Mixed Precision
- [169] FracBits Mixed Precision Quantization via Fractional Bit-Widths
- [170] Mixed-Precision Neural Network Quantization via Learned Layer-wise Importance
- [171] LRP-QViT Mixed-Precision Vision Transformer Quantization via Layer-wise Relevance Propagation
- [172] BitPruning Learning Bitlengths for Aggressive and Accurate Quantization
- [173] Free Bits Latency Optimization of Mixed-Precision Quantized Neural Networks on the Edge
- [174] FBM Fast-Bit Allocation for Mixed-Precision Quantization
- [175] A Mixed-Precision RISC-V Processor for Extreme-Edge DNN Inference
- [176] Retraining-free Model Quantization via One-Shot Weight-Coupling Learning
- [177] Arbitrary Bit-width Network A Joint Layer-Wise Quantization and Adaptive Inference Approach
- [178] Data Quality-aware Mixed-precision Quantization via Hybrid Reinforcement Learning
- [179] Channel-wise Mixed-precision Assignment for DNN Inference on Constrained Edge Nodes
- [180] DPRed Making Typical Activation and Weight Values Matter In Deep Learning Computing
- [181] The Impact of Quantization on the Robustness of Transformer-based Text Classifiers

- [182] RobustMQ Benchmarking Robustness of Quantized Models
- [183] Benchmarking the Robustness of Quantized Models
- [184] Investigating the Impact of Quantization on Adversarial Robustness
- [185] Qu-ANTI-zation Exploiting Quantization Artifacts for Achieving Adversarial Outcomes
- [186] Do Emergent Abilities Exist in Quantized Large Language Models An Empirical Study
- [187] The Cost of Compression Investigating the Impact of Compression on Parametric Knowledge in Language Models
- [188] A Comprehensive Evaluation of Quantization Strategies for Large Language Models
- [189] Decoding Compressed Trust Scrutinizing the Trustworthiness of Efficient LLMs Under Compression
- [190] On the Compressibility of Quantized Large Language Models
- [191] Robust Calibration with Multi-domain Temperature Scaling
- [192] Characterizing Truthfulness in Large Language Model Generations with Local Intrinsic Dimension
- [193] MQBench Towards Reproducible and Deployable Model Quantization Benchmark
- [194] Combining Compressions for Multiplicative Size Scaling on Natural Language Tasks
- [195] Automated Model Compression by Jointly Applied Pruning and Quantization
- [196] Single-path Bit Sharing for Automatic Loss-aware Model Compression
- [197] Differentiable Joint Pruning and Quantization for Hardware Efficiency
- [198] Chain of Compression A Systematic Approach to Combinatorially Compress Convolutional Neural Networks
- [199] Train Flat, Then Compress Sharpness-Aware Minimization Learns More Compressible Models
- [200] Understanding and Improving Knowledge Distillation for Quantization-Aware Training of Large Transformer Encoders
- [201] Characterising Bias in Compressed Models
- [202] Extreme Compression for Pre-trained Transformers Made Simple and Efficient
- [203] Model compression as constrained optimization, with application to neural nets. Part V combining compressions
- [204] Higher Layers Need More LoRA Experts
- [205] MoELoRA Contrastive Learning Guided Mixture of Experts on Parameter-Efficient Fine-Tuning for Large Language Models
- [206] Efficient Fine-tuning of Audio Spectrogram Transformers via Soft Mixture of Adapters
- [207] From Sparse to Soft Mixtures of Experts
- [208] PEMT Multi-Task Correlation Guided Mixture-of-Experts Enables Parameter-Efficient Transfer Learning

- [209] Task-Based MoE for Multitask Multilingual Machine Translation
- [210] Parameter-Efficient Mixture-of-Experts Architecture for Pre-trained Language Models
- [211] MoEC Mixture of Expert Clusters
- [212] Task-Specific Expert Pruning for Sparse Mixture-of-Experts
- [213] ComPEFT Compression for Communicating Parameter Efficient Updates via Sparsification and Quantization
- [214] QMoE Practical Sub-1-Bit Compression of Trillion-Parameter Models
- [215] SwapMoE Efficient Memory-Constrained Serving of Large Sparse MoE Models via Dynamic Expert Pruning and Swapping
- [216] Mixture of Quantized Experts (MoQE) Complementary Effect of Low-bit Quantization and Robustness
- [217] EdgeMoE Fast On-Device Inference of MoE-based Large Language Models
- [218] Fiddler CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models
- [219] BSQ Exploring Bit-Level Sparsity for Mixed-Precision Neural Network Quantization
- [220] Differentiable Channel Sparsity Search via Weight Sharing within Filters
- [221] Robustifying DARTS by Eliminating Information Bypass Leakage via Explicit Sparse Regularization
- [222] Enhanced Sparsification via Stimulative Training
- [223] Cream of the Crop Distilling Prioritized Paths For One-Shot Neural Architecture Search
- [224] Lossy and Lossless (L^2) Post-training Model Size Compression
- [225] Minimizing Area and Energy of Deep Learning Hardware Design Using Collective Low Precision and Structured Compression
- [226] Learning Best Combination for Efficient N M Sparsity
- [227] Combinatorial optimization for low bit-width neural networks
- [228] Is Modularity Transferable A Case Study through the Lens of Knowledge Distillation
- [229] Merge, Then Compress Demystify Efficient SMoE with Hints from Its Routing Policy
- [230] Lifting the Curse of Capacity Gap in Distilling Language Models
- [231] One Student Knows All Experts Know From Sparse to Dense
- [232] EvoMoE An Evolutional Mixture-of-Experts Training Framework via Dense-To-Sparse Gate
- [233] Fast Inference of Mixture-of-Experts Language Models with Offloading
- [234] Towards MoE Deployment Mitigating Inefficiencies in Mixture-of-Expert (MoE) Inference
- [235] Allo A Programming Model for Composable Accelerator Design
- [236] ISA Mapper A Compute and Hardware Agnostic Deep Learning Compiler
- [237] Cohmeleon Learning-Based Orchestration of Accelerator Coherence in Heterogeneous SoCs

- [238] A Case for Fine-grain Coherence Specialization in Heterogeneous Systems
- [239] MARS Multi-macro Architecture SRAM CIM-Based Accelerator with Co-designed Compressed Neural Networks
- [240] QWin Enforcing Tail Latency SLO at Shared Storage Backend
- [241] Next-generation Co-Packaged Optics for Future Disaggregated AI Systems
- [242] A Programming Model for GPU Load Balancing
- [243] HeteGen Heterogeneous Parallel Inference for Large Language Models on Resource-Constrained Devices
- [244] Synergy A HW SW Framework for High Throughput CNNs on Embedded Heterogeneous SoC
- [245] Multi-user Co-inference with Batch Processing Capable Edge Server
- [246] Scheduling Task-parallel Applications in Dynamically Asymmetric Environments
- [247] HeSP a simulation framework for solving the task scheduling-partitioning problem on heterogeneous architectures
- [248] Singularity Planet-Scale, Preemptive and Elastic Scheduling of AI Workloads
- [249] Efficient Execution of Quantized Deep Learning Models A Compiler Approach
- [250] FusionStitching Boosting Execution Efficiency of Memory Intensive Computations for DL Workloads
- [251] DNNFusion Accelerating Deep Neural Networks Execution with Advanced Operator Fusion
- [252] MetaTune Meta-Learning Based Cost Model for Fast and Efficient Auto-tuning Frameworks
- [253] ML-driven Hardware Cost Model for MLIR
- [254] LoopTune Optimizing Tensor Computations with Reinforcement Learning
- [255] SIP Autotuning GPU Native Schedules via Stochastic Instruction Perturbation
- [256] ALT Boosting Deep Learning Performance by Breaking the Wall between Graph and Operator Level Optimizations
- [257] FP8-LM Training FP8 Large Language Models
- [258] DeepliteRT Computer Vision at the Edge
- [259] Methodologies, Workloads, and Tools for Processing-in-Memory Enabling the Adoption of Data-Centric Architectures
- [260] AritPIM High-Throughput In-Memory Arithmetic
- [261] PIM-GPT A Hybrid Process-in-Memory Accelerator for Autoregressive Transformers
- [262] Accelerating Bulk Bit-Wise X(N)OR Operation in Processing-in-DRAM Platform
- [263] Analysis of Distributed Optimization Algorithms on a Real Processing-In-Memory System
- [264] Copernicus Characterizing the Performance Implications of Compression Formats Used in Sparse Workloads

- [265] Mitigating Memory Wall Effects in CNN Engines with On-the-Fly Weights Generation
- [266] Communication-Optimal Tilings for Projective Nested Loops with Arbitrary Bounds
- [267] Increasing the Efficiency of Sparse Matrix-Matrix Multiplication with a 2.5D Algorithm and One-Sided MPI
- [268] Flexible Communication Avoiding Matrix Multiplication on FPGA with High-Level Synthesis
- [269] CUDA-PIM End-to-End Integration of Digital Processing-in-Memory from High-Level C++ to Microarchitectural Design
- [270] An Algorithm-Hardware Co-design Framework to Overcome Imperfections of Mixed-signal DNN Accelerators
- [271] MAD Max Beyond Single-Node Enabling Large Machine Learning Model Acceleration on Distributed Systems
- [272] Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM
- [273] Optimus-CC Efficient Large NLP Model Training with 3D Parallelism Aware Communication Compression
- [274] AMP Automatically Finding Model Parallel Strategies with Heterogeneity Awareness
- [275] Network Partitioning and Avoidable Contention
- [276] START Straggler Prediction and Mitigation for Cloud Computing Environments using Encoder LSTM Networks
- [277] FedAT A High-Performance and Communication-Efficient Federated Learning System with Asynchronous Tiers
- [278] High Throughput Synchronous Distributed Stochastic Gradient Descent
- [279] DéjàVu KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving
- [280] Understanding Capacity-Driven Scale-Out Neural Recommendation Inference
- [281] AlpaServe Statistical Multiplexing with Model Parallelism for Deep Learning Serving
- [282] Reducing Large Adaptation Spaces in Self-Adaptive Systems Using Machine Learning
- [283] CAMEO A Causal Transfer Learning Approach for Performance Optimization of Configurable Computer Systems
- [284] Unicorn Reasoning about Configurable System Performance through the lens of Causality
- [285] Metareasoning in Modular Software Systems On-the-Fly Configuration using Reinforcement Learning with Rich Contextual Representations
- [286] A Hybrid Approach Combining Control Theory and AI for Engineering Self-Adaptive Systems
- [287] Dealing with Drift of Adaptation Spaces in Learning-based Self-Adaptive Systems using Life-long Self-Adaptation
- [288] Utilizing Ensemble Learning for Performance and Power Modeling and Improvement of Parallel Cancer Deep Learning CANDLER Benchmarks

- [289] An Efficient Framework for Floor-plan Prediction of Dynamic Runtime Reconfigurable Systems
- [290] SAPA Self-Aware Polymorphic Architecture
- [291] Collective Mind cleaning up the research and experimentation mess in computer engineering using crowdsourcing, big data and machine learning
- [292] SpQR A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression
- [293] Transformer-Lite High-efficiency Deployment of Large Language Models on Mobile Phone GPUs
- [294] Big-Little Adaptive Neural Networks on Low-Power Near-Subthreshold Processors
- [295] REDS Resource-Efficient Deep Subnetworks for Dynamic Resource Constraints
- [296] Agile-Quant Activation-Guided Quantization for Faster Inference of LLMs on the Edge
- [297] Dynamic Split Computing for Efficient Deep Edge Intelligence
- [298] Partitioning and Deployment of Deep Neural Networks on Edge Clusters
- [299] AdaMEC Towards a Context-Adaptive and Dynamically-Combinable DNN Deployment Framework for Mobile Edge Computing
- [300] NEUKONFIG Reducing Edge Service Downtime When Repartitioning DNNs
- [301] SplitEE Early Exit in Deep Neural Networks with Split Computing
- [302] Calibration-Aided Edge Inference Offloading via Adaptive Model Partitioning of Deep Neural Networks
- [303] I-SplitEE Image classification in Split Computing DNNs with Early Exits
- [304] DVFO Learning-Based DVFS for Energy-Efficient Edge-Cloud Collaborative Inference
- [305] Privacy-preserving Security Inference Towards Cloud-Edge Collaborative Using Differential Privacy
- [306] Decentralized Proactive Model Offloading and Resource Allocation for Split and Federated Learning
- [307] DEFER Distributed Edge Inference for Deep Neural Networks
- [308] SEIFER Scalable Edge Inference for Deep Neural Networks
- [309] DistrEdge Speeding up Convolutional Neural Network Inference on Distributed Edge Devices
- [310] PICO Pipeline Inference Framework for Versatile CNNs on Diverse Mobile Devices
- [311] Dynamic Heterogeneity-Aware Coded Cooperative Computation at the Edge
- [312] Dynamic Network-Assisted D2D-Aided Coded Distributed Learning
- [313] Task-Oriented Communication for Multi-Device Cooperative Edge Inference
- [314] Attention-aware Semantic Communications for Collaborative Inference
- [315] Fault-Tolerant Collaborative Inference through the Edge-PRUNE Framework

- [316] Result and Congestion Aware Optimal Routing and Partial Offloading in Collaborative Edge Computing
- [317] Joint D2D Collaboration and Task Offloading for Edge Computing A Mean Field Graph Approach
- [318] Joint Task Offloading Scheduling and Transmit Power Allocation for Mobile-Edge Computing Systems
- [319] Joint Task Offloading and Resource Allocation for Multi-Server Mobile-Edge Computing Networks
- [320] Online Learning for Offloading and Autoscaling in Energy Harvesting Mobile Edge Computing
- [321] Online Learning for Offloading and Autoscaling in Renewable-Powered Mobile Edge Computing
- [322] Multi-objective Deep Reinforcement Learning for Mobile Edge Computing
- [323] Energy-based Proportional Fairness in Cooperative Edge Computing
- [324] Energy-Efficient Resource Management for Federated Edge Learning with CPU-GPU Heterogeneous Computing
- [325] Energy-Aware Offloading in Time-Sensitive Networks with Mobile Edge Computing
- [326] Decentralized Computation Offloading for Multi-User Mobile Edge Computing A Deep Reinforcement Learning Approach
- [327] Delay-aware and Energy-Efficient Computation Offloading in Mobile Edge Computing Using Deep Reinforcement Learning
- [328] Energy-Efficient Multi-Orchestrator Mobile Edge Learning
- [329] Joint Computing, Pushing, and Caching Optimization for Mobile Edge Computing Networks via Soft Actor-Critic Learning
- [330] Leveraging the Power of Prediction Predictive Service Placement for Latency-Sensitive Mobile Edge Computing
- [331] Using Machine Learning for Handover Optimization in Vehicular Fog Computing
- [332] Proactive Mobility Management of UEs using Sequence-to-Sequence Modeling
- [333] MAP-Me Managing Anchor-less Producer Mobility in Information-Centric Networks
- [334] Anchor Free IP Mobility
- [335] Seamless Handover for IMS over Mobile-IPv6 Using Context Transfer
- [336] Online Service Migration in Mobile Edge with Incomplete System Information A Deep Recurrent Actor-Critic Learning Approach
- [337] PDMA Probabilistic Service Migration Approach for Delay-aware and Mobility-aware Mobile Edge Computing
- [338] LLM as a System Service on Mobile Devices

- [339] When ‘Computing follows Vehicles’ Decentralized Mobility-Aware Resource Allocation in the Edge-to-Cloud Continuum
- [340] Mobility-Aware Caching in D2D Networks
- [341] EMM Energy-Aware Mobility Management for Mobile Edge Computing in Ultra Dense Networks
- [342] Risk-Aware Resource Allocation for URLLC Challenges and Strategies with Machine Learning
- [343] Borrowing from yourself Faster future video segmentation with partial channel update
- [344] LDMRes-Net Enabling Efficient Medical Image Segmentation on IoT and Edge Platforms
- [345] RAP-SAM Towards Real-Time All-Purpose Segment Anything
- [346] Optimizing Service Placement in Edge-to-Cloud AR VR Systems using a Multi-Objective Genetic Algorithm
- [347] Proactive Scheduling and Caching for Wireless VR Viewport Streaming
- [348] Tangram High-resolution Video Analytics on Serverless Platform with SLO-aware Batching
- [349] MISAR A Multimodal Instructional System with Augmented Reality
- [350] DeepStream Bandwidth Efficient Multi-Camera Video Streaming for Deep Learning Analytics
- [351] BiSwift Bandwidth Orchestrator for Multi-Stream Video Analytics on Edge
- [352] 5G Edge Vision Wearable Assistive Technology for People with Blindness and Low Vision
- [353] Ev-Edge Efficient Execution of Event-based Vision Algorithms on Commodity Edge Platforms
- [354] HALSIE Hybrid Approach to Learning Segmentation by Simultaneously Exploiting Image and Event Modalities
- [355] Assessing the embodied carbon footprint of IoT edge devices with a bottom-up life-cycle approach
- [356] The Dirty Secret of SSDs Embodied Carbon
- [357] GreenFPGA Evaluating FPGAs as Environmentally Sustainable Computing Solutions
- [358] GreenScale Carbon-Aware Systems for Edge Computing
- [359] EASE Energy-Aware Job Scheduling for Vehicular Edge Networks With Renewable Energy Resources
- [360] EDCompress Energy-Aware Model Compression for Dataflows
- [361] Clover Toward Sustainable AI with Carbon-Aware Machine Learning Inference Service
- [362] 5 6G Networks of the Future or Defuturing Networks
- [363] REFRESH FPGAs Sustainable FPGA Chiplet Architectures
- [364] Photonics for Sustainable Computing
- [365] SECure A Social and Environmental Certificate for AI Systems
- [366] Recommendations for public action towards sustainable generative AI systems

- [367] Compressing LLMs The Truth is Rarely Pure and Never Simple
- [368] Do Compressed LLMs Forget Knowledge An Experimental Study with Practical Implications
- [369] Mechanisms of non-factual hallucinations in language models
- [370] Pruning Small Pre-Trained Weights Irreversibly and Monotonically Impairs Difficult Downstream Tasks in LLMs
- [371] In-Context Sharpness as Alerts An Inner Representation Perspective for Hallucination Mitigation
- [372] INSIDE LLMs’ Internal States Retain the Power of Hallucination Detection
- [373] Entity-Based Knowledge Conflicts in Question Answering
- [374] Quantifying and Attributing the Hallucination of Large Language Models via Association Analysis
- [375] On Large Language Models’ Hallucination with Regard to Known Facts
- [376] Benchmarking Adversarial Robustness of Compressed Deep Learning Models
- [377] Pruning in the Face of Adversaries
- [378] Robustness and Transferability of Universal Attacks on Compressed Models
- [379] On the Adversarial Robustness of Quantized Neural Networks
- [380] Model Compression with Adversarial Robustness A Unified Optimization Framework
- [381] Defensive Quantization When Efficiency Meets Robustness
- [382] Blind Adversarial Pruning Balance Accuracy, Efficiency and Robustness
- [383] Masking Adversarial Damage Finding Adversarial Saliency for Robust and Sparse Network
- [384] Publishing Efficient On-device Models Increases Adversarial Vulnerability
- [385] Adversarial Robustness vs Model Compression, or Both
- [386] Unintended memorisation of unique features in neural networks
- [387] Planting and Mitigating Memorized Content in Predictive-Text Language Models
- [388] Can LLMs Keep a Secret Testing Privacy Implications of Language Models via Contextual Integrity Theory
- [389] The Privacy Onion Effect Memorization is Relative
- [390] Memorization of Named Entities in Fine-tuned BERT Models
- [391] On the Interaction Between Differential Privacy and Gradient Compression in Deep Learning
- [392] On the Effectiveness of Regularization Against Membership Inference Attacks
- [393] Investigating Membership Inference Attacks under Data Dependencies
- [394] A Cautionary Tale On the Role of Reference Data in Empirical Privacy Defenses
- [395] Learning with User-Level Privacy

- [396] Element Level Differential Privacy The Right Granularity of Privacy
- [397] Convergence and Privacy of Decentralized Nonconvex Optimization with Gradient Clipping and Communication Compression
- [398] On the Evaluation of User Privacy in Deep Neural Networks using Timing Side Channel
- [399] What Do Compressed Deep Neural Networks Forget
- [400] Compressed Models Decompress Race Biases What Quantized Models Forget for Fair Face Recognition
- [401] Understanding the Effect of Model Compression on Social Bias in Large Language Models
- [402] Can Model Compression Improve NLP Fairness
- [403] Are Compressed Language Models Less Subgroup Robust
- [404] Fairness Improves Learning from Noisily Labeled Long-Tailed Data
- [405] Does lossy image compression affect racial bias within face recognition
- [406] The Impact of Unstated Norms in Bias Analysis of Language Models
- [407] An Investigation of Representation and Allocation Harms in Contrastive Learning
- [408] Improved Quantization Strategies for Managing Heavy-tailed Gradients in Distributed Learning
- [409] Removing Disparate Impact of Differentially Private Stochastic Gradient Descent on Model Accuracy
- [410] SafetyPrompts a Systematic Review of Open Datasets for Evaluating and Improving Large Language Model Safety
- [411] AEGIS Online Adaptive AI Content Safety Moderation with Ensemble of LLM Experts
- [412] ALERT A Comprehensive Benchmark for Assessing Large Language Models' Safety through Red Teaming
- [413] Data Contamination Through the Lens of Time
- [414] An Open Source Data Contamination Report for Large Language Models
- [415] Estimating Contamination via Perplexity Quantifying Memorisation in Language Model Evaluation
- [416] Generalization or Memorization Data Contamination and Trustworthy Evaluation for Large Language Models
- [417] Assessing the Brittleness of Safety Alignment via Pruning and Low-Rank Modifications
- [418] Safety and Performance, Why Not Both Bi-Objective Optimized Model Compression against Heterogeneous Attacks Toward AI Software Deployment
- [419] The Art of Defending A Systematic Evaluation and Analysis of LLM Defense Strategies on Safety and Over-Defensiveness
- [420] The WMDP Benchmark Measuring and Reducing Malicious Use With Unlearning

- [421] CyberMetric A Benchmark Dataset for Evaluating Large Language Models Knowledge in Cybersecurity
- [422] CyberSecEval 2 A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models
- [423] SC-Safety A Multi-round Open-ended Question Adversarial Safety Benchmark for Large Language Models in Chinese
- [424] Modeling Lead-vehicle Kinematics For Rear-end Crash Scenario Generation
- [425] Beyond 1D and oversimplified kinematics A generic analytical framework for surrogate safety measures
- [426] Holistic Safety and Responsibility Evaluations of Advanced AI Models
- [427] Adversarial Fine-tuning of Compressed Neural Networks for Joint Improvement of Robustness and Efficiency
- [428] Pruning Adversarially Robust Neural Networks without Adversarial Examples
- [429] Sparsity Meets Robustness Channel Pruning for the Feynman-Kac Formalism Principled Robust Deep Neural Nets
- [430] FlatENN Train Flat for Enhanced Fault Tolerance of Quantized Deep Neural Networks
- [431] Parameter-Saving Adversarial Training Reinforcing Multi-Perturbation Robustness via Hypernetworks
- [432] Rebooting Neuromorphic Hardware Design – A Complexity Engineering Approach
- [433] Neuromorphic Intermediate Representation A Unified Instruction Set for Interoperable Brain-Inspired Computing
- [434] The Impact of On-chip Communication on Memory Technologies for Neuromorphic Systems
- [435] A Case for 3D Integrated System Design for Neuromorphic Computing & AI Applications
- [436] Nonuniform magnetic domain-wall synapses enabled by population coding
- [437] Thermal-Aware Compilation of Spiking Neural Networks to Neuromorphic Hardware
- [438] Cryogenic Neuromorphic Hardware
- [439] Hybrid Stochastic Synapses Enabled by Scaled Ferroelectric Field-effect Transistors
- [440] Design-Technology Co-Optimization for NVM-based Neuromorphic Processing Elements
- [441] Machine Learning Meets Quantitative Planning Enabling Self-Adaptation in Autonomous Robots
- [442] Slimmable Compressive Autoencoders for Practical Neural Image Compression
- [443] AdaBits Neural Network Quantization with Adaptive Bit-Widths
- [444] FIT A Metric for Model Sensitivity
- [445] Continual Test-Time Domain Adaptation
- [446] Layer-wise Auto-Weighting for Non-Stationary Test-Time Adaptation

- [447] AdaSpring Context-adaptive and Runtime-evolutionary Deep Model Compression for Mobile Applications
- [448] Need is All You Need Homeostatic Neural Networks Adapt to Concept Shift
- [449] Towards Self-Adaptive Machine Learning-Enabled Systems Through QoS-Aware Model Switching
- [450] SWITCH An Exemplar for Evaluating Self-Adaptive ML-Enabled Systems
- [451] Deep Compression Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding
- [452] Compact representations of convolutional neural networks via weight pruning and quantization
- [453] APack Off-Chip, Lossless Data Compression for Efficient Deep Learning Inference
- [454] Weight Fixing Networks
- [455] Rate Distortion For Model Compression From Theory To Practice
- [456] The Unreasonable Ineffectiveness of the Deeper Layers
- [457] Encoding Weights of Irregular Sparsity for Fixed-to-Fixed Model Compression
- [458] Lossless Compression with Probabilistic Circuits
- [459] Improving Prediction-Based Lossy Compression Dramatically via Ratio-Quality Modeling
- [460] Streaming Lossless Volumetric Compression of Medical Images Using Gated Recurrent Convolutional Neural Network
- [461] Surveying (Dis)Parities and Concerns of Compute Hungry NLP Research
- [462] Efficiency Pentathlon A Standardized Arena for Efficiency Evaluation
- [463] An energy-based comparative analysis of common approaches to text classification in the Legal domain
- [464] Utility is in the Eye of the User A Critique of NLP Leaderboards
- [465] Bridging Fairness and Environmental Sustainability in Natural Language Processing
- [466] Surveying Attitudinal Alignment Between Large Language Models Vs. Humans Towards 17 Sustainable Development Goals
- [467] Cheaply Evaluating Inference Efficiency Metrics for Autoregressive Transformer APIs
- [468] QualEval Qualitative Evaluation for Model Improvement
- [469] Divergent Token Metrics Measuring degradation to prune away LLM components – and optimize quantization
- [470] SMART Automatically Scaling Down Language Models with Accuracy Guarantees for Reduced Processing Fees
- [471] Use large language models to promote equity
- [472] On Hallucination and Predictive Uncertainty in Conditional Language Generation

- [473] Fact-Checking the Output of Large Language Models via Token-Level Uncertainty Quantification
- [474] Introspective Planning Guiding Language-Enabled Agents to Refine Their Own Uncertainty
- [475] Uncertainty-Based Abstention in LLMs Improves Safety and Reduces Hallucinations
- [476] Med-HALT Medical Domain Hallucination Test for Large Language Models
- [477] HypoTermQA Hypothetical Terms Dataset for Benchmarking Hallucination Tendency of LLMs
- [478] What if... Counterfactual Inception to Mitigate Hallucination Effects in Large Multimodal Models
- [479] Towards Mitigating Hallucination in Large Language Models via Self-Reflection
- [480] Retrieve Only When It Needs Adaptive Retrieval Augmentation for Hallucination Mitigation in Large Language Models
- [481] Wafer-scale Computing Advancements, Challenges, and Future Perspectives
- [482] Deep Neuromorphic Networks with Superconducting Single Flux Quanta
- [483] Considerations for neuromorphic supercomputing in semiconducting and superconducting optoelectronic hardware
- [484] Cryogenic Memory Technologies
- [485] Towards Reverse-Engineering the Brain Brain-Derived Neuromorphic Computing Approach with Photonic, Electronic, and Ionic Dynamicity in 3D integrated circuits